DOUGLAS COLLEGE

CSIS 4495 – 002 | Applied Research Project

Title: Bookstagram

"Why just read when you can create!"

(MERN Stack Social AI Authoring Platform)

Final Report

Submitted To:

Prof. Padmapriya Arasanipalai Kandhadai kandhadaip

Team Members

Name	Student ID	Position
Khushi Bhatt	300394398	**Team Leader**
Aditi, Aditi	300396361	Team Member

1. Introduction

1.1 Background and Context of the Research

The rapid rise of digital platforms and generative AI has transformed how content is created and consumed. Traditional book writing demands sustained focus and discipline, yet modern digital culture, shaped by platforms like Instagram and TikTok, has shifted user preferences toward short, interactive, and visually engaging formats. This shift has led to reduced attention spans and lower completion rates in long form writing.

Generative AI tools such as Google Gemini have made content creation more accessible by helping users generate story ideas, outlines, and draft chapters, reducing barriers like writer's block. However, these tools operate in isolation without social engagement features. Social media platforms, conversely, foster interaction through likes, follows, and feeds but lack structured support for long form storytelling. This gap between AI assisted productivity and social engagement defines the "social authoring gap" that Bookstagram directly addresses by combining AI assisted eBook creation with interactive social features in a single unified platform.

1.2 Problem Statement

Many beginner and intermediate writers struggle to complete long form writing projects due to difficulty generating ideas, lack of motivation, and feeling overwhelmed by eBook structure. This results in a high rate of unfinished creative projects.

While AI tools assist with content generation, they do not improve long term motivation or engagement. Social media platforms encourage interaction but lack structured support for organized storytelling. This "social authoring gap", which is the absence of a platform combining AI writing assistance with community engagement, is what Bookstagram directly addresses.

1.3 Research Questions

To guide the development and evaluation of Bookstagram, this research addresses the following questions:

1. How does AI assisted story generation affect writing productivity and user confidence among beginner writers? Google Gemini integration reduces the difficulty of starting a story, enabling users to generate outlines and draft chapters more efficiently than manual writing.

2. Do social engagement features such as likes, follows, and story feeds improve user motivation and platform retention? Social features transform writing from a private task into an interactive experience, encouraging users to revisit the platform through social validation mechanisms.

3. Can combining AI writing tools with social networking provide a more effective authoring experience than traditional writing tools? Traditional applications focus only on editing with no creative guidance or community interaction. Bookstagram's hybrid model addresses both productivity and motivation simultaneously.

1.4 Literature Review and Knowledge Gaps

Research shows that generative AI is increasingly used in content creation, helping users overcome creative barriers and increase drafting speed. Studies on human AI collaboration suggest productivity improves when AI acts as a supportive partner rather than a replacement (Bender et al., 2021). Research on gamification also confirms that validation mechanisms like likes and progress tracking enhance user motivation and retention.

However, most AI writing platforms operate independently of social features, while social platforms lack structured writing tools. Limited research explores how combining both may influence productivity and completion rates, which is a gap this project directly addresses.

1.5 Initial Hypotheses and Assumptions

Hypotheses:
AI assisted story generation will increase writing productivity and reduce writer's block.
Social engagement features will positively influence user motivation and platform retention.
A combined AI and social authoring model will provide a more effective writing experience than traditional tools.

Assumptions:
Users are willing to use AI as a collaborative writing assistant rather than a replacement.
Younger users prefer interactive, visually structured environments over traditional word processors.
Social validation mechanisms encourage consistent participation in creative tasks.

1.6 Potential Benefits of the Research

- Improved Writing Productivity: AI tools help users generate structured content efficiently, reducing writer's block and increasing completion rates.
- Increased Writing Confidence: Beginners feel more confident with AI generated suggestions and structured outlines.
- Enhanced User Motivation: Likes, follows, and story feeds encourage consistent writing activity.
- Community Building: The platform fosters a supportive digital community where writers share ideas and grow together.
- Human AI Collaboration Advancement: The project explores AI as a creative partner, contributing to co creativity research.
- Innovation in Digital Publishing: The social authoring model introduces a new approach to the digital publishing ecosystem

2. Summary of the Research Project

2.1 Research Objectives and Overall Goal

Bookstagram was proposed as a Social AI integrated digital storytelling platform to help beginner and intermediate writers overcome common barriers such as writer's block, lack of motivation, and incomplete projects by combining AI assisted content generation with social networking features.

In its final form, the platform significantly exceeded the original scope, delivering a fully functional MERN stack application with JWT authentication, Google Gemini AI eBook creation, Markdown chapter editor, Cloudinary media storage, PDF and DOCX export, live social feed, like and follow system, public profiles, Explore page, HuggingFace AI Stories, Threads, real time Socket.io messaging with AI Bookbot, live search, user survey system, and an admin analytics dashboard.

2.2 Research Design and Methodology

The project followed an Agile SDLC with sprint based delivery from January to April 2026, covering authentication, frontend UI, AI integration, social features, messaging, and evaluation. Google Gemini handled eBook text generation, HuggingFace handled AI image generation, Cloudinary managed media storage, and Socket.io powered real time messaging.

2.3 Technology Stack (Final Implementation)

- Frontend: React.js + Tailwind CSS for UI components and responsive styling
- Frontend Charts: Recharts for analytics dashboard visualizations
- Backend: Node.js + Express.js for REST API server and route handling
- Database: MongoDB + Mongoose for users, books, chapters, and social data storage
- Authentication: JWT + bcrypt for secure login, session management, and password hashing
- AI Writing: Google Gemini API gemini 2.5 flash lite for eBook outline and chapter generation
- AI Images: HuggingFace Inference API for story image and avatar generation
- Storage: Cloudinary + multer storage cloudinary for cover images, avatars, thread, and story images
- Real time: Socket.io for private messaging and online user tracking
- Export: PDFKit + docx npm for eBook export to PDF and DOCX
- Development Environment: Windows 11, VS Code, Git + GitHub for development and version control
- Package Manager: npm for dependency management

3. Changes to the Proposal

3.1 Analytics System – Fully Implemented (Exceeded Original Plan)

Original Approach: Simple automatic analytics logging storing user activity metrics in MongoDB.
Change Implemented: The analytics system was fully implemented and exceeded the original plan. An ActivityLog model tracks all user actions via a trackActivity middleware. A dedicated admin only Analytics Dashboard was built with stat cards such as total users, total eBooks, total actions, and daily active users, along with a 7 day activity line chart, most used features bar chart, and top active users leaderboard, all using Recharts. A Survey system was also added with a SurveyModal collecting demographic and feedback data after eBook creation, which is visualized in the dashboard.
Justification: As backend stability was achieved after midterm, the team had the capacity to build the full analytics infrastructure to strengthen the research evaluation component.

3.2 Social Features – Expanded Far Beyond Original Scope

Original Approach: Likes, follows, story feeds, and the story thread feature were planned for completion in Weeks 5 and 6 of the proposal. The original scope covered a basic like system, follow system, an Instagram style story feed, and user profiles with stats.
Change Implemented: All originally planned social features were fully implemented and significantly expanded into a complete social ecosystem. The final platform includes a live social feed showing books from followed authors, a like and unlike system with real time count updates, follow and unfollow with followers and following lists and modals, public user profiles with post, follower, and following stats, an Explore page showing all public eBooks, Instagram style AI generated Stories using HuggingFace FLUX.1 schnell model, Twitter or X style Threads with text posts and up to five images per post stored via Cloudinary, real time private messaging powered by Socket.io with unread message tracking, an integrated AI Bookbot assistant in the messaging system, and a live search feature for finding users and books.
Justification: Once the core AI writing features were stabilized after midterm, development progressed faster. Each social feature built naturally on the previous one, and additional features such as messaging, threads, and stories were identified as essential to creating a complete social authoring experience rather than just a writing tool with social elements.

3.3 Authentication Approach Adjusted

Original Approach: Firebase Authentication exclusively.
Change Implemented: JWT and bcrypt implemented as the primary system. Google OAuth added via Firebase frontend and the /api/auth/google backend endpoint. Account deletion with full cascade and password change with current password verification were also added.
Justification: JWT provided a stable MERN aligned foundation. Google OAuth was added as a supplementary sign in option. Account management features were added as the platform scaled.

3.4 Storage: Google Cloud to Cloudinary

Original Approach: Google Cloud Storage for all media.
Change Implemented: Cloudinary adopted as the sole media solution via multer storage cloudinary. All cover images, avatars, thread images, story images, and AI generated chapter images are stored with automatic optimization.
Justification: Cloudinary offered faster setup, built in optimization, and a free tier, which removed the complexity of Google Cloud credentials and billing.

3.5 Real Time Messaging Added

Original Approach : No messaging planned.
Change Implemented: Full Socket.io private messaging with conversation list, unread badges, online user tracking via a server side Map, and an AI Bookbot that responds to book recommendations and writing tips.
Justification: Direct communication between users was identified as important for improving community engagement as the platform scaled.

3.6 Extended AI Capabilities

Original Approach: Google Gemini for text only, no image AI planned.
Change Implemented: Gemini upgraded to gemini 2.5 flash lite. HuggingFace FLUX.1 schnell added for AI story image generation in four styles, AI avatar generation and editing, AI chapter image generation, AI book cover generation, AI chapter content completion, AI thread text improvement, and AI content images.
Justification: Gemini did not support image generation. HuggingFace provided a suitable free tier solution which was essential for adding visual features to the platform.

3.7 Settings Page Added

Original Approach : Basic profile editing only.
Change Implemented: A comprehensive Settings Page was implemented with five sections including Edit Profile using Cloudinary avatars, Change Password, Notification Preferences, Appearance with dark and light theme and font size using localStorage, and Privacy and Safety with full account deletion cascade.
Justification: As the platform evolved into a full social application, centralized account management became necessary.

3.8 Book Status System Added

Original Approach : No draft or published distinction.
Change implemented: A status field with draft and published options, default set to draft, was added to the Book model. The Explore page and social feed display only published books.
Justification: Authors needed control over content visibility once public sharing features were introduced.

3.9 Chapter Image Support Added

Original Approach: Chapter model included text fields only.
Change Implemented : An image field was added to the Chapter schema and a headerImage field to the Book schema for AI generated header images using HuggingFace, stored in Cloudinary.
Justification: Visual elements improve the reading experience and were required to support AI generated chapter images.

4. Project Completion Timeline (From Beginning to End)

The project was developed using an Agile SDLC methodology with structured weekly sprints from January 2026 through April 2026. The following outlines the complete actual timeline based on work completed by each team member.

4.1 Team Responsibilities

Khushi Bhatt – Team Leader (300394398):

- Overall project planning, task coordination, and sprint monitoring
- Backend development using Node.js and Express.js
- Frontend development using React.js and Tailwind CSS
- Frontend development of landing page, login page, registration page, and choosing the theme of the Bookstagram project
- Frontend and backend implementation of search feature
- Implementing logic of follow and unfollow users in the frontend with dynamic follower and following count
- Researching best free tiers to integrate in the system for generating AI images, AI avatar, and AI book covers
- Google Gemini AI integration for eBook outline generation, chapter content generation, and thread text improvement
- HuggingFace Inference API integration for AI story image generation, avatar generation and editing, chapter header image generation, and book cover generation
- Real-time messaging backend using Socket.io including Message model, messageController, and online user Map tracking
- Social features backend including Like model, Follow model, and socialController covering feed, like and unlike, follow and unfollow, user profiles, suggested users, and search
- Cloudinary integration for all media storage including cover images, avatars, thread images, and story images
- Analytics backend including ActivityLog model, trackActivity middleware, and analyticsController with aggregation pipeline and date-range filtering
- Survey system backend including Survey model, surveyController, and surveyRoutes
- eBook export system including PDF via PDFKit, DOCX via docx npm, exportController, and exportRoutes
- Authentication extensions including Google OAuth backend endpoint, changePassword, and deleteAccount with cascade
- Implemented feature testing, unit testing, integration testing, and system testing
- GitHub repository management and version control supervision
- Research data analysis, final report writing, and documentation

Aditi Aditi – Team Member (300396361):

- Backend: authController, authRoutes, authMiddleware, User model, bcrypt integration
- Backend: Thread model, threadController including get all threads, create thread, like and unlike, add comment, delete thread, and threadRoutes with Multer image upload middleware supporting up to 5 images per post
- Backend: bio field in User model, getAllBooks function in bookController, completeChapterContent in aiController with /complete-chapter-content endpoint
- Frontend: Dashboard layout, BookCard component, dashboard routes
- Frontend: ChapterEditorTab, ChapterSidebar, SimpleMDEditor, BookDetailsTab, ViewBook, ViewChapterSidebar components
- Frontend: NewDashboardLayout including sidebar and topbar with live search and profile menu
- Frontend: NewDashboardPage including social home feed with BookPostCard, StoriesStrip, and StoryViewer
- Frontend: ExplorePage showing all public books grid with like and read functionality
- Frontend: ThreadsPage with real API integration, photo upload up to 5 images, optimistic like and unlike, comments, author avatar, time display, full image modal, View Profile button, and delete thread and comment
- Frontend: AI Writing Assistant on Threads sidebar using Gemini AI for improving thread text and suggesting relevant hashtags with individual and bulk copy
- Frontend: MessagesPage UI including conversation list with unread badges, real-time chat window, and AI Bookbot interface
- Frontend: SettingsPage with five sections including Edit Profile, Change Password, Notification Preferences, Appearance with dark mode and font size, and Privacy
- Frontend: AnalyticsPage UI including StatCards, LineChart, BarChart, and PieChart using Recharts with survey analytics visualization
- Frontend: SurveyModal as a multi-step feedback form collecting age, gender, genre, rating, and recommendation
- Frontend: ProfilePage with NewDashboardLayout integration, BookCard, CreateBookModal, like button, and follow or unfollow with followers and following modal
- Frontend: Three writing modes in Chapter Editor including Write Yourself, Full AI Generation, and Half Human plus Half AI
- Avatar display bug fix and dark mode fixes across multiple pages
- System testing, debugging, quality assurance, and presentation slides preparation

4.2 Complete Sprint Timeline

Jan 11 to Jan 25, 2026

- Project ideation and brainstorming
- Market research using YouTube, Indeed, and job boards

- Finalizing project scope and requirements
- Proposal writing, timeline creation, and team contract signing
- Deliverable: Project Proposal submitted on Blackboard

Jan 26 to Feb 7, 2026

- GitHub repository setup
- Node.js and Express server initialization and MongoDB connection
- JWT authentication system including register and login, User model, and bcrypt password hashing
- Basic CRUD API endpoints
- Deliverable: Working backend server and complete authentication system

Feb 8 to Feb 14, 2026

- React project setup with Tailwind CSS
- Landing Page including Navbar, Hero, Features, Testimonials, and Footer
- Login and Sign-Up pages, AuthContext, ProtectedRoute
- Dashboard layout, BookCard component, CreateBookModal with two-step AI outline
- Google Gemini API integration and eBook outline generation
- Story model, routes, and controller
- Deliverable: Functional frontend connected to backend with AI outline generation working

Feb 15 to Feb 21, 2026

- ChapterEditorTab with split-pane Markdown editor using SimpleMDEditor
- ChapterSidebar and BookDetailsTab components
- AI chapter content generation using Gemini
- Cover image upload using multer
- ViewBook and ViewChapterSidebar components
- Profile page and Save Changes API
- Branch merging, conflict resolution, midterm report writing, and demo video recording
- Deliverable: Midterm Report and Demo Video submitted

Feb 22 to Mar 7, 2026

- Follow model and Like model in MongoDB
- socialController including follow, unfollow, getFeed, likeBook, unlikeBook, getUserProfile, getSuggestedUsers, getFollowers, getFollowing, and searchUsersAndBooks
- socialRoutes setup
- Gemini API migration and testing
- Deliverable: All social backend routes committed and tested

Mar 8 to Mar 14, 2026

- Like and Follow frontend integration
- BookPostCard with real-time like updates

- StoriesStrip and StoryViewer components
- CreateStoryModal with AI prompt, avatar, and upload tabs
- Story backend setup
- HuggingFace integration for AI image generation
- Cloudinary setup for media storage
- Deliverable: Social features, AI Stories, and Cloudinary integrated

Mar 15 to Mar 21, 2026

- PDF export using PDFKit
- DOCX export using docx npm
- Thread model and APIs
- ThreadsPage UI with full features
- AI thread improvement using Gemini
- Deliverable: Export system and Threads fully functional

Mar 22 to Mar 28, 2026

- Socket.io real-time messaging integration
- Message APIs and UI
- AI Bookbot integration
- Google OAuth integration
- ExplorePage and live search
- Deliverable: Messaging, Explore page, search, and Google Sign-In working

Mar 29 to Apr 3, 2026

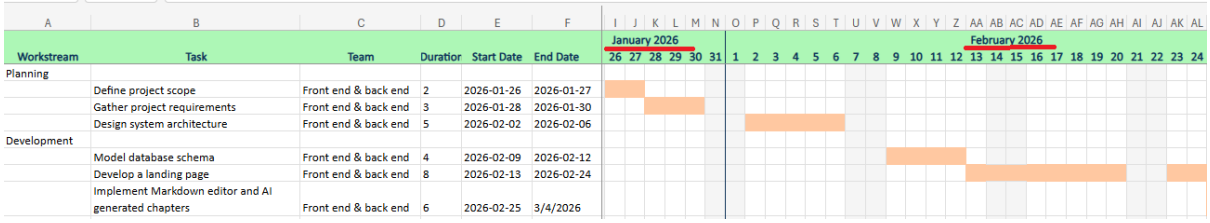
- Settings page with full sections
- Account deletion with cascade
- Analytics backend and dashboard
- Survey system implementation
- Additional AI features
- Deliverable: Settings, analytics dashboard, survey system, and AI features complete

Apr 4 to Apr 7, 2026

- Final testing and bug fixing
- README update
- User guide and presentation preparation
- Final report writing
- GitHub final code check-in
- Deliverable: Final report, repository finalized, and presentation ready

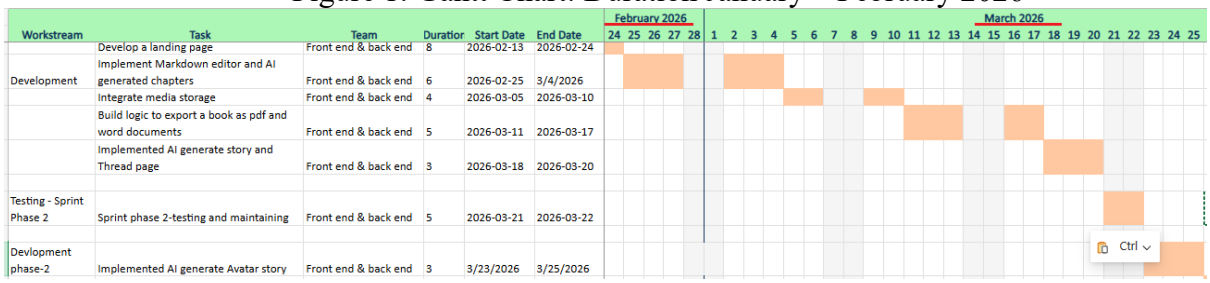
4.3 Project Management Chart (Gantt Chart)

The following Gantt chart illustrates the project schedule from January 2026 through April 2026, showing all sprints, milestones, and team deliverables.



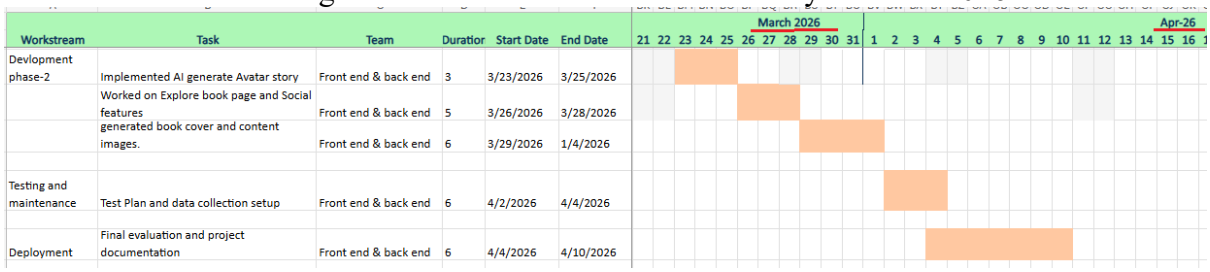
Workstream	Task	Team	Duration	Start Date	End Date
Planning					
	Define project scope	Front end & back end	2	2026-01-26	2026-01-27
	Gather project requirements	Front end & back end	3	2026-01-28	2026-01-30
	Design system architecture	Front end & back end	5	2026-02-02	2026-02-06
Development					
	Model database schema	Front end & back end	4	2026-02-09	2026-02-12
	Develop a landing page	Front end & back end	8	2026-02-13	2026-02-24
	Implement Markdown editor and AI generated chapters	Front end & back end	6	2026-02-25	3/4/2026

Figure 1: Gantt Chart: Duration January – February 2026



Workstream	Task	Team	Duration	Start Date	End Date
	Develop a landing page	Front end & back end	8	2026-02-13	2026-02-24
Development	Implement Markdown editor and AI generated chapters	Front end & back end	6	2026-02-25	3/4/2026
	Integrate media storage	Front end & back end	4	2026-03-05	2026-03-10
	Build logic to export a book as pdf and word documents	Front end & back end	5	2026-03-11	2026-03-17
	Implemented AI generate story and Thread page	Front end & back end	3	2026-03-18	2026-03-20
Testing - Sprint Phase 2	Sprint phase 2-testing and maintaining	Front end & back end	5	2026-03-21	2026-03-22
Devlopment phase-2	Implemented AI generate Avatar story	Front end & back end	3	3/23/2026	3/25/2026

Figure 2: Gantt Chart: Duration February – March 2026



Workstream	Task	Team	Duration	Start Date	End Date
Devlopment phase-2	Implemented AI generate Avatar story	Front end & back end	3	3/23/2026	3/25/2026
	Worked on Explore book page and Social features	Front end & back end	5	3/26/2026	3/28/2026
	generated book cover and content images.	Front end & back end	6	3/29/2026	1/4/2026
Testing and maintenance	Test Plan and data collection setup	Front end & back end	6	4/2/2026	4/4/2026
Deployment	Final evaluation and project documentation	Front end & back end	6	4/4/2026	4/10/2026

Figure 3: Gantt Chart: Duration March – April 2026

4.4 Development Methodology

The project followed an Agile SDLC with 10 structured weekly sprints, each with defined goals, tasks, and deliverables. Sprint reviews were held at the end of each sprint to assess progress and re-prioritize the backlog. Team communication used WhatsApp for daily updates and weekly meetings for sprint planning. All code was version-controlled via Git and GitHub, with all reports committed to the main branch.

5. Implemented Features

This section documents all features implemented in the Bookstagram platform from project inception through the final sprint.

5.1 User Authentication System (Sign-Up, Sign-In, and Google OAuth)

Individual Contributions:
Aditi Aditi – authController, authRoutes, authMiddleware, User model, bcrypt, changePassword, deleteAccount with cascade, bio field.
Khushi Bhatt – LoginPage, SignupPage, AuthContext, ProtectedRoute, Google Sign-In via Firebase, Settings password change and account deletion.

Feature Description
A secure authentication system supporting two sign-in methods: email and password, and Google OAuth. It includes bcrypt password hashing, JWT-based session management, protected API routes, password change with current password verification, and full account deletion cascading across all related collections including books, likes, follows, messages, and activity logs using Promise.all.

How It Works
User credentials are collected via controlled React form components and submitted using Axios. For Google Sign-In, Firebase handles the OAuth flow and returns the user's name, email, Google ID, and avatar to the /api/auth/google backend endpoint, which creates or links the account. On the backend, passwords are hashed with bcrypt before being stored in MongoDB. JWT tokens are generated upon successful authentication and validated by authMiddleware on every protected route.

Challenges Faced
Configuring the Mongoose schema to exclude the password field from default queries using select: false. Coordinating Firebase frontend OAuth with a custom backend endpoint that handles both new account creation and existing account Google ID linking.

Screenshots

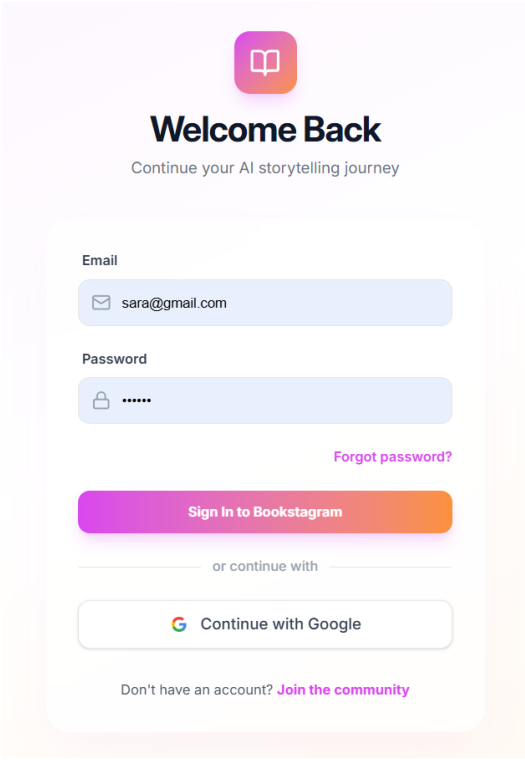


Figure 1: Sign in

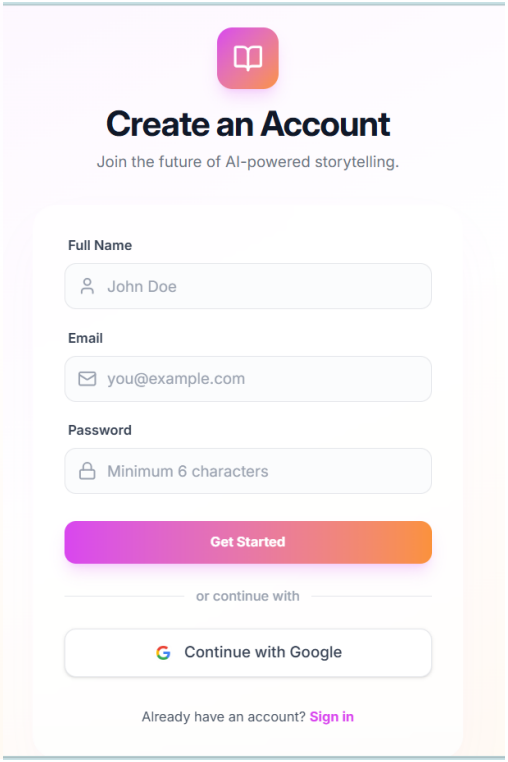


Figure 2: Sign up

5.2 Landing Page

Individual Contributions:

Khushi Bhatt and Aditi Aditi – all five sections including Navbar, Hero, Features, Testimonials, and Footer using React.js and Tailwind CSS.

Feature Description

The landing page is the public entry point of Bookstagram, consisting of five modular React components: a responsive Navbar, a Hero section with dynamic gradient typography and platform statistics, an animated Features section, a Testimonials section with star-rated user reviews, and a Footer with navigation links and social icons.

How It Works

The Navbar dynamically adjusts based on authentication state via useAuth context. Authenticated users see a profile avatar and logout option, while unauthenticated users see Login and Get Started buttons. The Hero section displays platform statistics such as 50K plus books created and a 4.9 out of 5 rating, along with a conditional call to action that routes authenticated users to their dashboard. The Features section renders six feature cards dynamically from a data array, each with an icon, title, description, and hover animation.

Challenges Faced

Maintaining responsive consistency across different screen sizes required multiple design refinements. Conditional Navbar rendering based on authentication state required careful coordination with AuthContext to prevent inconsistencies during initial load.

Screenshots

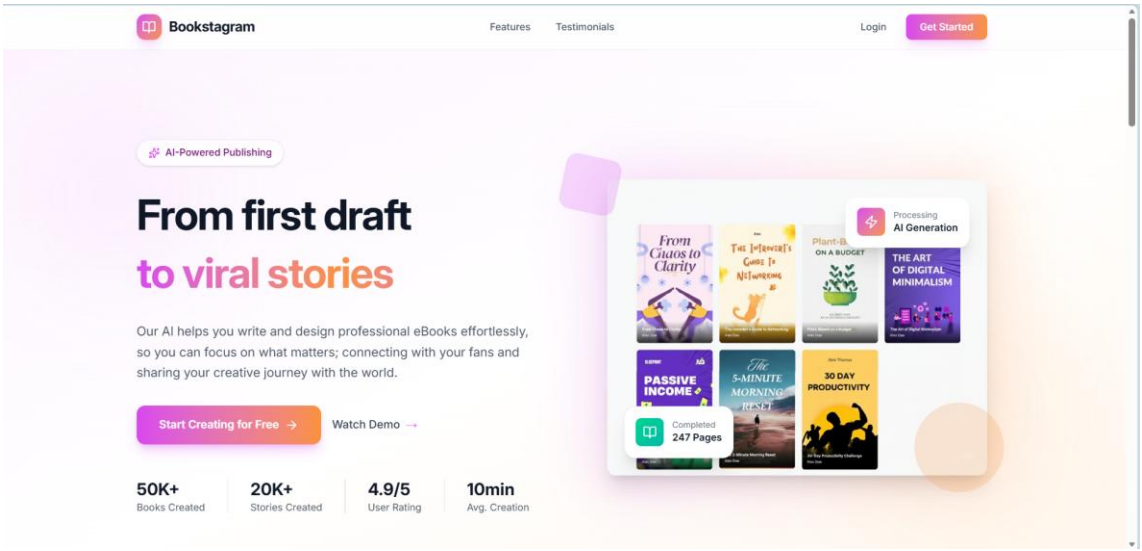


Figure 3: Landing page – Hero section

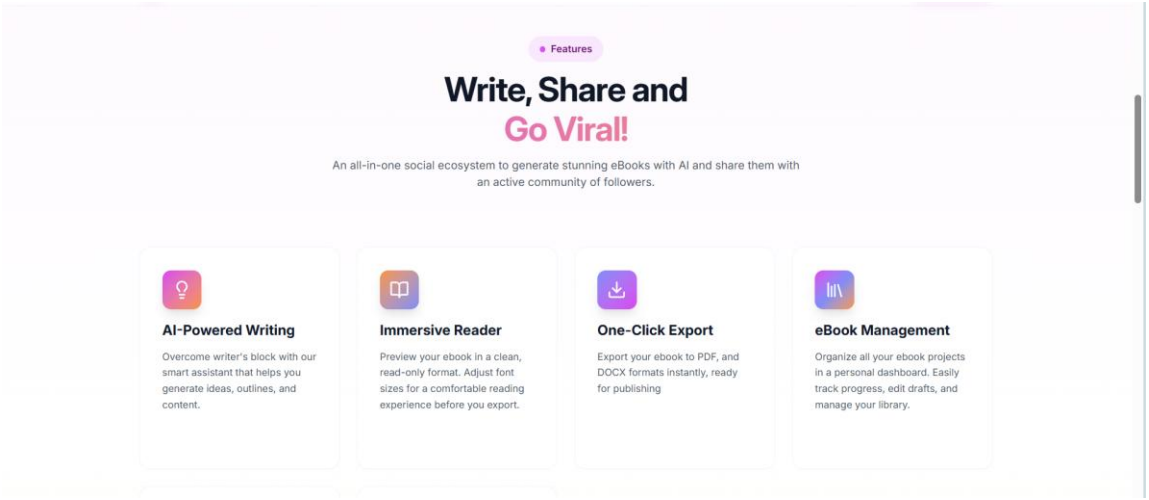


Figure 4: Landing Page – Features section

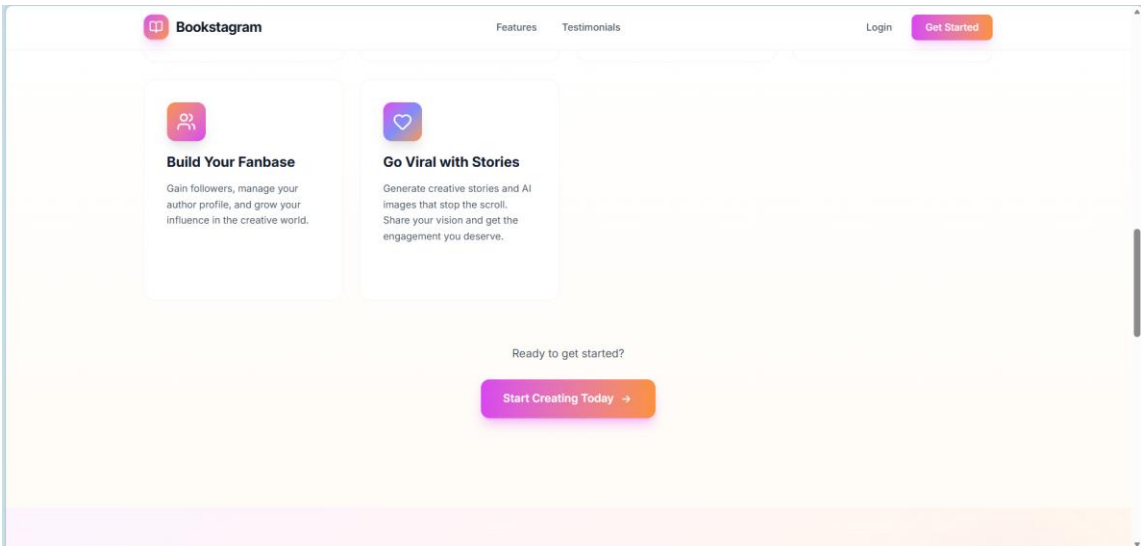


Figure 5: Landing Page – Features section

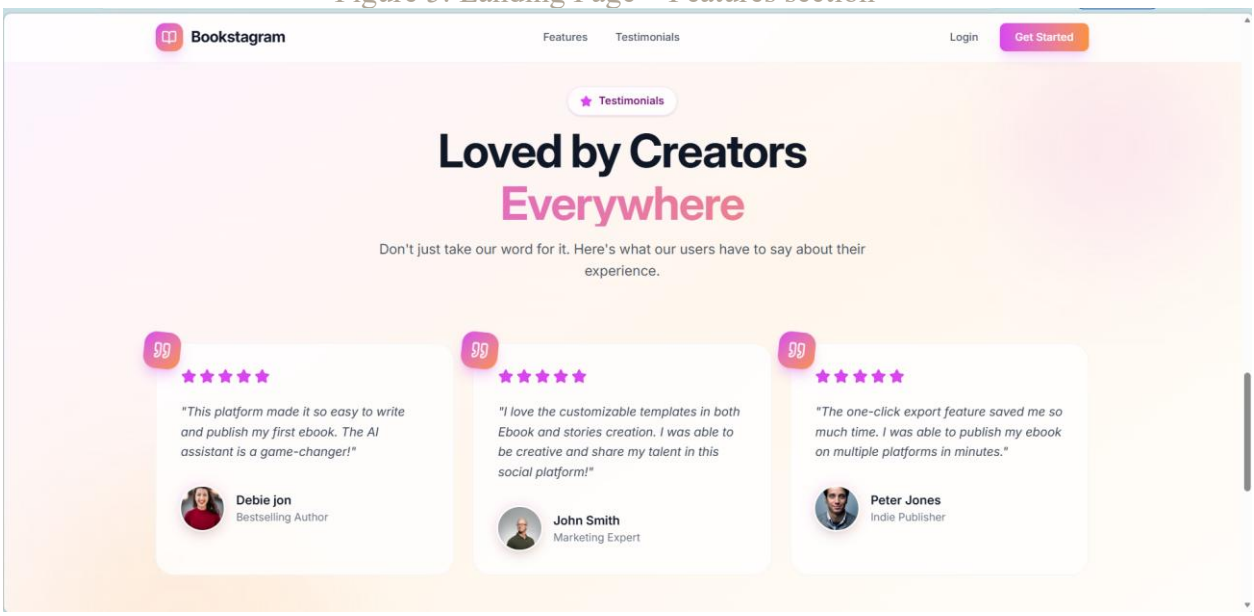


Figure 6: Landing Page – Testimonials section

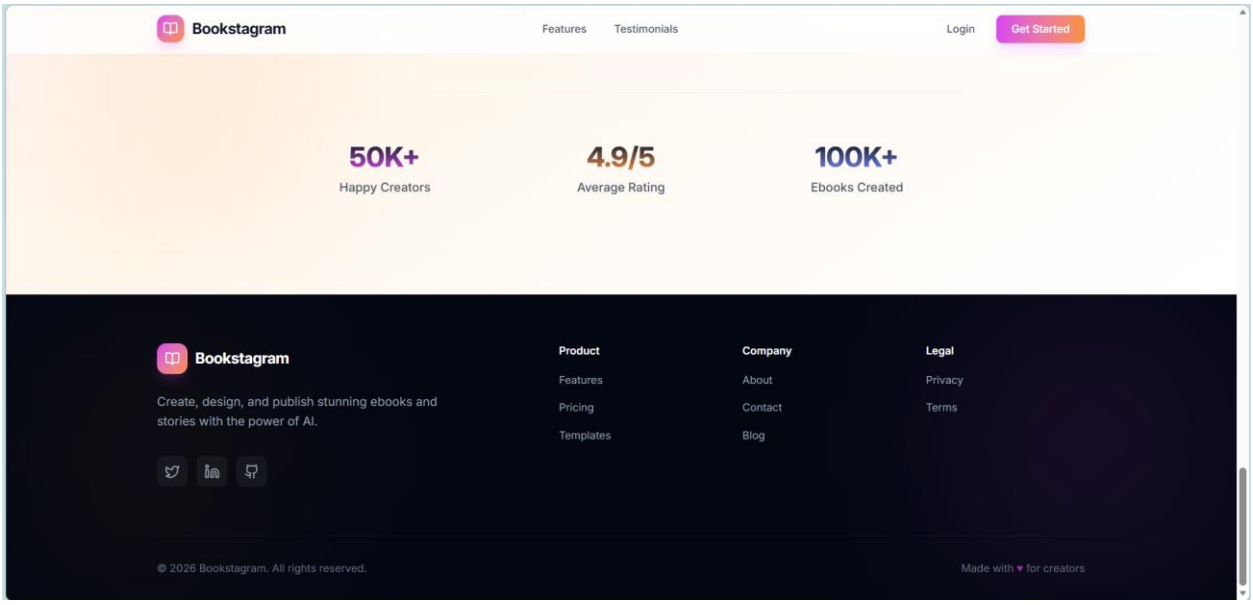


Figure 7: Landing Page – Footer

5.3 eBook Creation and Management Dashboard

Individual Contributions:

Khushi Bhatt – CreateBookModal including two-step AI outline generation, delete confirmation modal, and backend book API integration.
Aditi Aditi – DashboardPage layout, BookCard component, and dashboard routes.

Feature Description

The Dashboard allows authenticated users to create, view, manage, and delete their AI-generated eBooks. The centerpiece is the two-step CreateBookModal which uses Google Gemini AI to generate a complete structured chapter outline from a single topic input. All eBooks are displayed as BookCards showing cover image, title, and author name in a responsive grid.

How It Works

Clicking Create New eBook opens the CreateBookModal which works in two steps:
Step 1 — Book Details: The user enters Book Title, Number of Chapters, optional Topic, and Writing Style. Clicking Generate Outline with AI sends these inputs to the /api/ai/generate-outline endpoint where Gemini (gemini-2.5-flash-lite) returns a JSON array of chapter objects, each containing a title and description.
Step 2 — Review Chapters: The AI-generated chapters are displayed as a numbered list. The user can reorder chapters, add new ones, or go back to regenerate. Clicking Create eBook saves the book and all chapters to MongoDB and redirects to the Chapter Editor.
On Dashboard load, all user books are fetched and displayed as BookCards. Clicking a card opens the Chapter Editor. Deletion uses a confirmation modal. On confirmation, a DELETE request removes the book and immediately updates the UI without a page refresh.

Challenges Faced

Asynchronous AI outline generation required implementing a loading spinner while waiting for the Gemini response. Parsing the response required extracting the JSON array using indexOf and lastIndexOf, since Gemini occasionally wraps JSON within additional text. Resetting modal state between openings required careful management using useEffect and useState.

Screenshots

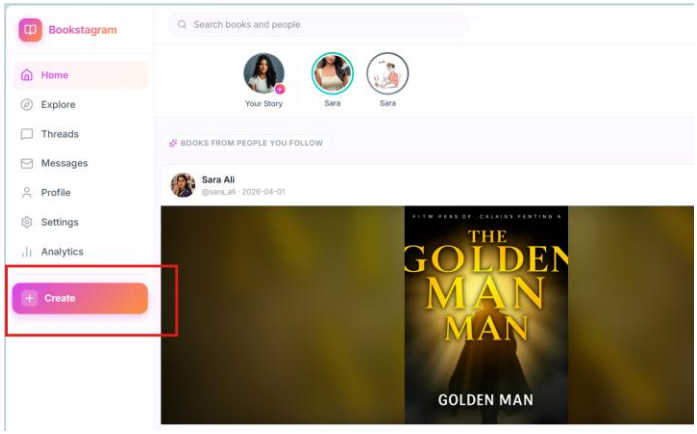


Figure 8: Home Page

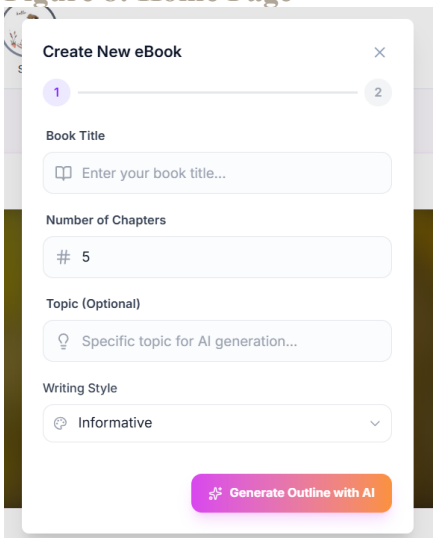


Figure 9: Create Book modal

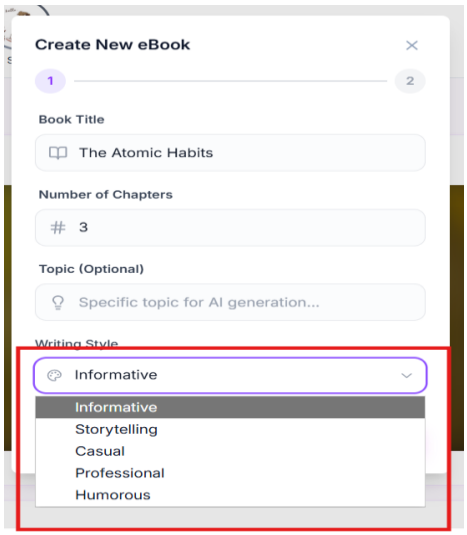


Figure 10: Create Book modal

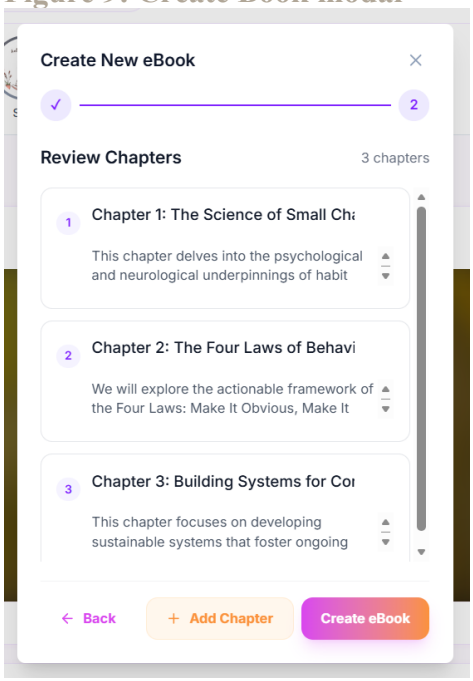


Figure 11: Create Book modal

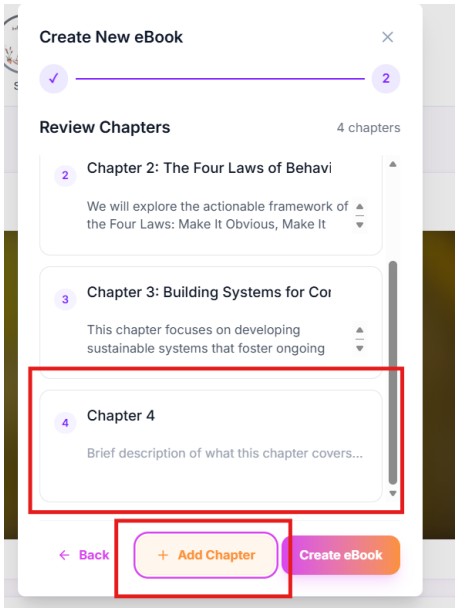


Figure 12: Create Book modal

5.4 AI-Powered Chapter Editor with Three Writing Modes

Individual Contributions:

Aditi Aditi – ChapterEditorTab, ChapterSidebar, SimpleMDEditor, BookDetailsTab components, ViewBook component, and three writing modes implementation including completeChapterContent in aiController, /complete-chapter-content route in aiRoutes, and mode selector UI in ChapterEditorTab and EditorPage.

Khushi Bhatt – Google Gemini AI content generation integration including aiController and aiRoutes, Save Changes API, chapter CRUD operations, AI chapter image generation using HuggingFace FLUX.1-schnell, AI book cover generation, and AI content images.

Feature Description

The Chapter Editor is the primary writing workspace of Bookstagram. It provides a split-pane Markdown editor with live preview, a chapter navigation sidebar, AI-assisted content generation, and three distinct writing modes. The interface includes two tabs: Editor for writing and generating content, and Book Details for managing metadata and cover image.

How It Works

The editor operates in split-pane mode with raw Markdown on the left and live HTML preview on the right. Three writing modes are available:

Write Yourself: the user writes the entire chapter manually

Full AI Generation: Gemini generates the complete chapter from scratch

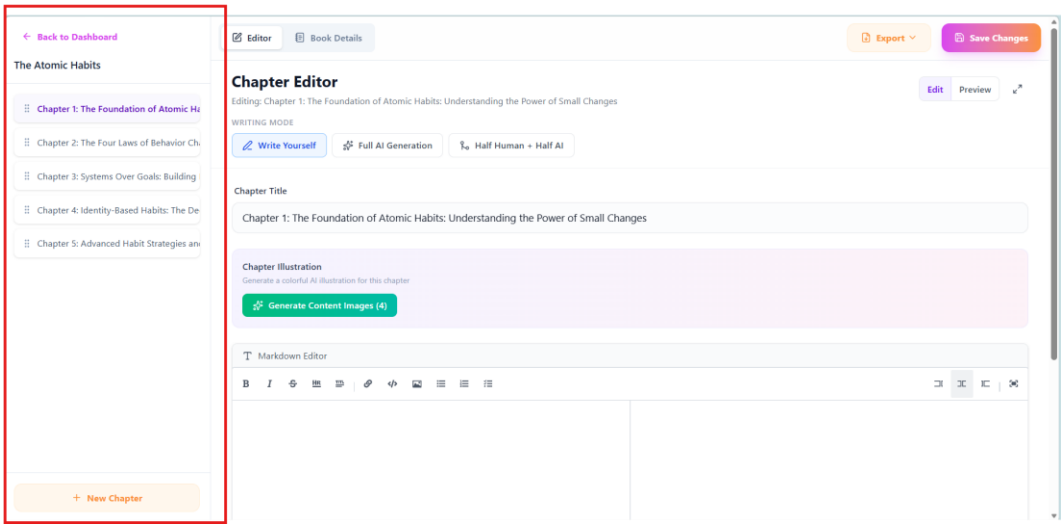
Half Human plus Half AI: the user writes the opening lines and AI continues in the same tone using the completeChapterContent endpoint

The Generate with AI button sends the chapter title, book context, and writing style to the backend, where Gemini processes the request and returns Markdown content that is inserted directly into the editor. AI chapter header images are generated using HuggingFace FLUX.1-schnell and stored in Cloudinary. Save Changes sends a PUT request to update the chapter in MongoDB.

Challenges Faced

Gemini required careful prompt engineering to produce consistent Markdown output. The transition from @google/genai v0.x to v1.x introduced breaking changes, requiring updates to the model name gemini-2.5-flash-lite and request format. The Half Human plus Half AI mode required precise prompt construction to ensure the AI continues the user's text in the same tone without repeating existing content.

Screenshots



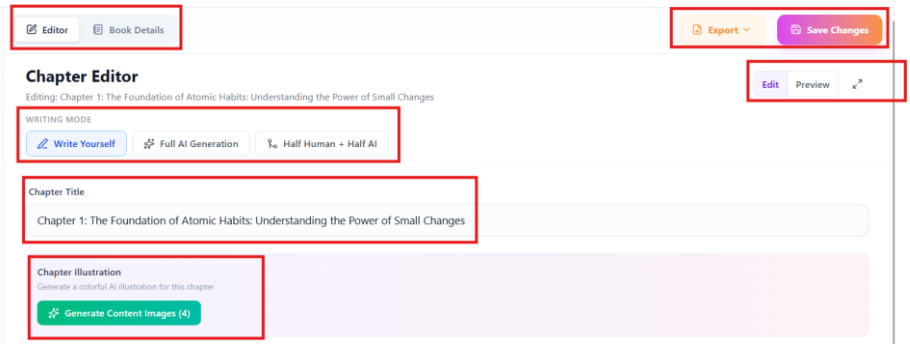


Figure 13,14: Chapter Editor Page

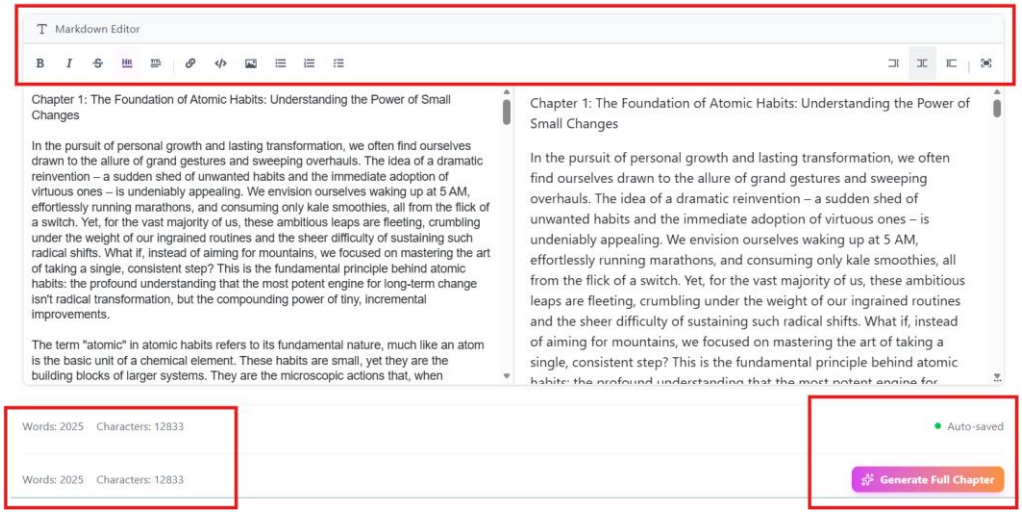


Figure 15: Chapter Editor Page

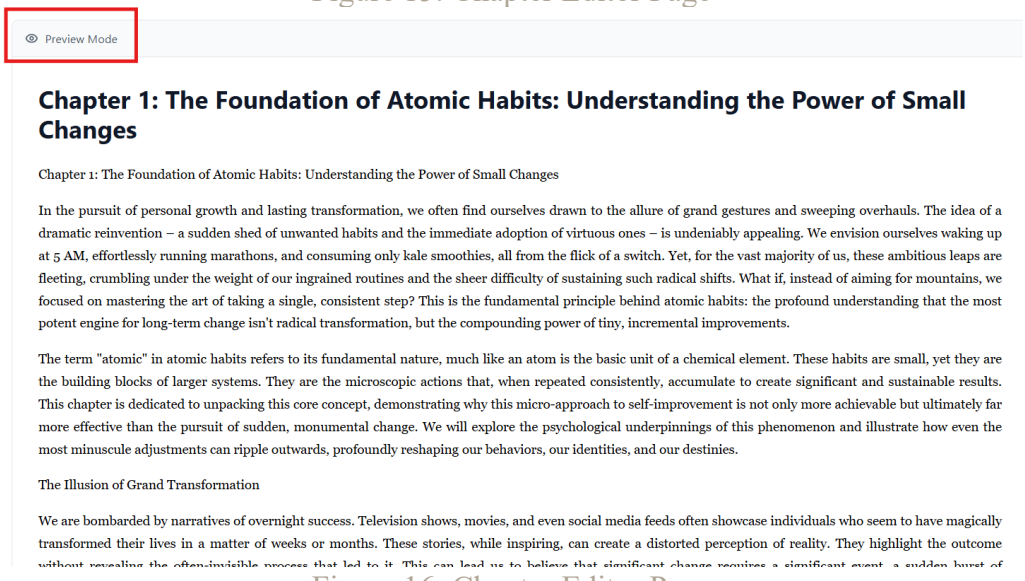


Figure 16: Chapter Editor Page

5.5 Book Details Tab and Cloudinary Cover Image Upload

Individual Contributions:

Aditi Aditi – BookDetailsTab UI (Title, Author, Subtitle fields, Cover Image section).
Khushi Bhatt – cover image upload API (multer-storage-cloudinary), Cloudinary storage integration, Book model coverImage and headerImage fields.

Feature Description
The Book Details tab provides a structured interface for managing eBook metadata and visual identity. Users can update the book title, author name, and subtitle, and upload a custom cover image stored via Cloudinary. The uploaded cover image is immediately reflected in both the Book Details tab and the Dashboard BookCard.

How It Works
Book metadata is loaded from React state on tab navigation. Cover images are uploaded via a multipart form POST request handled by multer-storage-cloudinary middleware, which stores images directly in Cloudinary with automatic quality and format optimization. The backend returns the Cloudinary secure URL, saved to the Book document in MongoDB and immediately reflected in the UI.

Challenges Faced
Migrating from local multer disk storage to Cloudinary required reconfiguring the storage engine and ensuring the returned file path was the Cloudinary HTTPS URL. State synchronization between the Editor page and Dashboard BookCard required propagating the new Cloudinary URL to all components displaying the cover image.

Screenshots

Figure 17: Book Details Tab

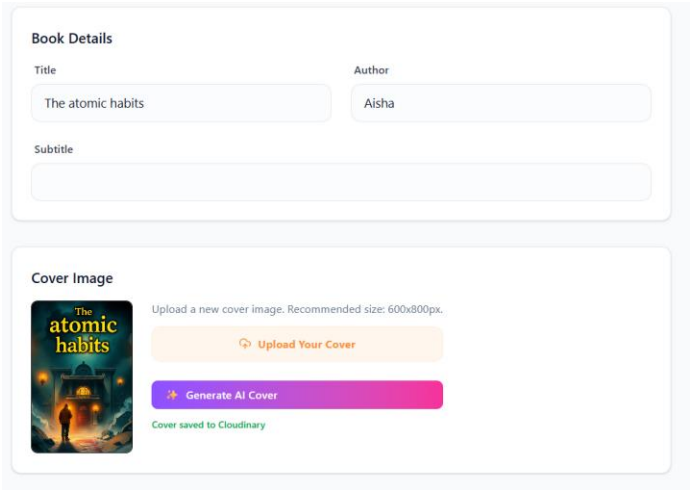


Figure 18: Book Details Tab

5.6 eBook Export Feature (PDF and DOCX)

Individual Contributions:
Khushi Bhatt – exportController (PDF via PDFKit with Markdown parsing, DOCX via docx npm with styled headings and paragraphs), exportRoutes, Export dropdown UI in Chapter Editor.

Feature Description
The eBook Export feature allows users to download their complete eBook in two professional formats: PDF and DOCX. The export system processes all chapters, converts Markdown content to properly formatted output, and includes the book's cover image, title, author name, subtitle, and all chapter content in the exported file.

How It Works
Clicking the Export dropdown in the Chapter Editor top bar reveals PDF and DOCX options. For PDF export, the backend uses PDFKit to construct a multi-page document, parsing Markdown tokens using markdown-it to render headings, paragraphs, bold/italic text, bullet lists, ordered lists, blockquotes, code blocks, and horizontal rules with appropriate typography. For DOCX export, the docx npm library builds a Word document with Charter body font and Inter heading font, processing the same Markdown tokens into properly styled Paragraph and TextRun elements with heading levels, indentation, and spacing. Both formats include a styled title page with the book cover image, title, author, and subtitle before the chapter content.

Challenges Faced
Parsing Markdown tokens correctly for both PDF and DOCX required building custom token processors for markdown-it's token stream, handling nested inline formatting (bold, italic, code) within paragraphs and list items. Embedding the Cloudinary cover image in PDFKit required fetching the image buffer from the remote URL and passing it as a Buffer to PDFKit's image method.
Screenshots

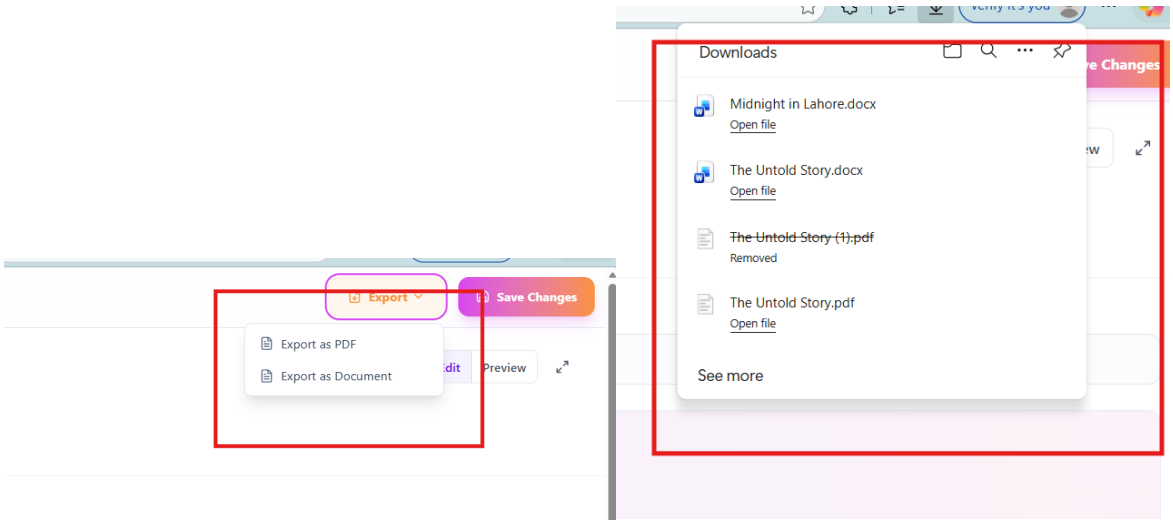


Figure 19-21: Export book Feature

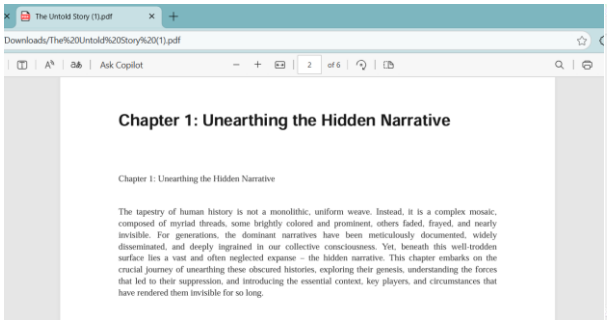


Figure 22: Export as PDF

Figure 23: Export as Doc

5.7 Immersive Book Reader

Individual Contributions:

Aditi Aditi – ViewBook component, ViewChapterSidebar, Markdown-to-HTML formatting, font size control.

Feature Description

Clean, distraction-free reading experience for any platform eBook. Renders Markdown chapter content as formatted HTML, supports adjustable font sizes (14px–24px), and provides a collapsible chapter navigation sidebar accessible from Explore page, social feed, or profile.

How It Works

ViewBook fetches the book and chapters from backend. A custom formatContent function converts Markdown (bold, italic, inline images) to HTML rendered via dangerouslySetInnerHTML. ViewChapterSidebar lists all chapters with active chapter highlighted. On mobile, sidebar collapses behind a Menu button.

Challenges Faced

Custom regex processing required to detect inline image syntax and convert to HTML img tags with Cloudinary URLs. XSS prevention required ensuring all content originated from platform's own AI or users.

Screenshots

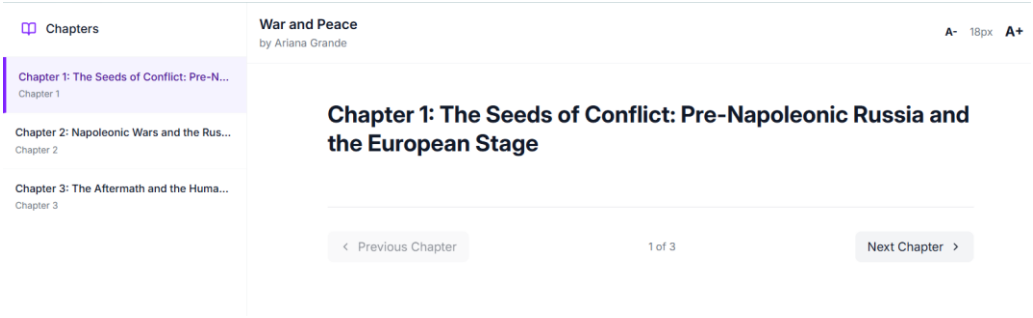


Figure 25: View Book Page

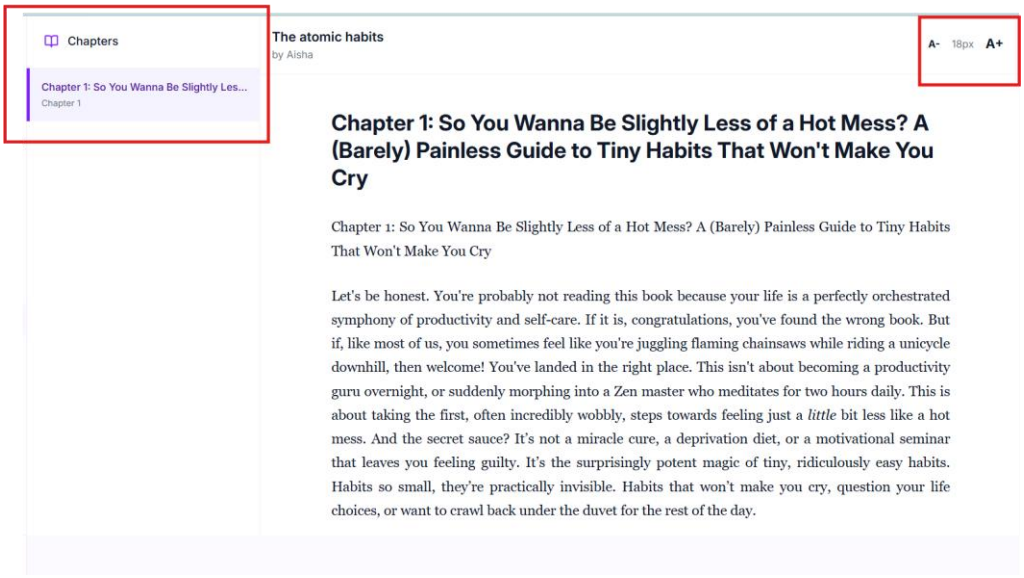


Figure 24: View Book page

5.8 Social Feed – New Dashboard (Home Page)

Individual Contributions: *Khushi Bhatt – getFeed (socialController), enrichBooksWithLikes helper (MongoDB aggregation), getSuggestedUsers API.*
Aditi Aditi – NewDashboardLayout (sidebar + topbar + live search), NewDashboardPage (BookPostCard feed, StoriesStrip, SuggestedUserCard), wireframe and 8-step implementation plan.

Feature Description: Instagram-style home feed displaying eBooks from followed authors as BookPostCards showing author avatar, book cover, title, like count, and Read button. StoriesStrip at top shows active stories from followed users. Right sidebar shows suggested users with Follow/Unfollow toggle.

How It Works: getFeed fetches followed users' published eBooks sorted by newest first. enrichBooksWithLikes uses MongoDB aggregation (Promise.all combining Like.find and Like.aggregate) to attach per-user like status and counts in one round-trip. NewDashboardLayout provides fixed left sidebar navigation and top bar with live search. Optimistic state updates reflect like toggles immediately before API response returns.

Challenges: Faced Combining Like.find and Like.aggregate using Promise.all then merging into books array for single-query like enrichment. Real-time like count updates required optimistic React state updates before API response.

Screenshots

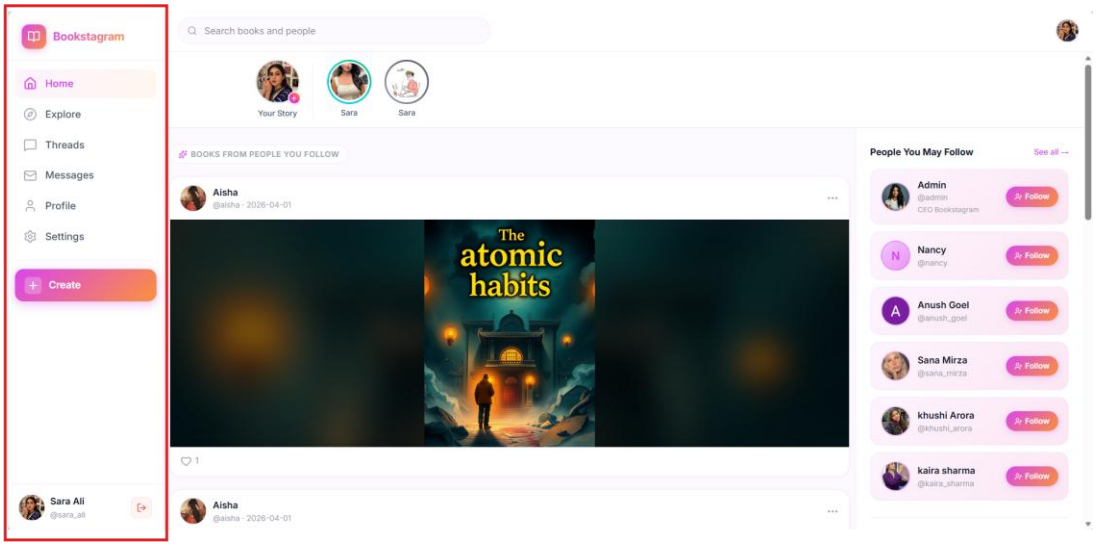


Figure 26: Home Page

Figure 27: Home Page

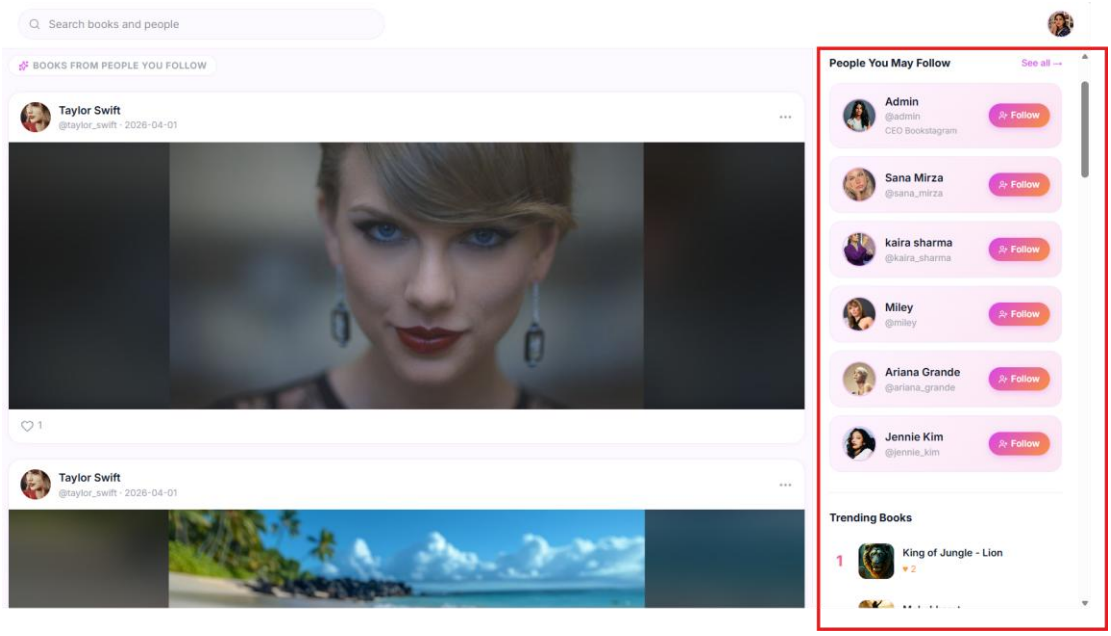


Figure 28: Home Page

5.9 Like, Follow, and Public User Profiles

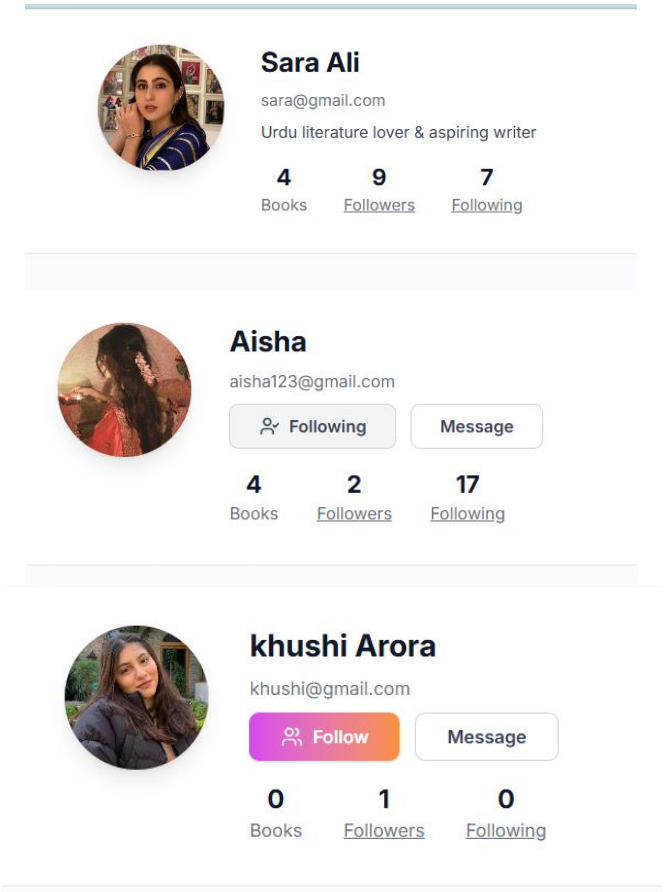
Individual Contributions: Khushi Bhatt – Follow model, Like model, socialController (followUser, unfollowUser, likeBook, unlikeBook, getUserProfile, getFollowers, getFollowing), socialRoutes.
Aditi Aditi – ProfilePage UI (avatar, bio, stats, Follow/Unfollow button, followers/following modal, books grid, like buttons, SurveyModal trigger).

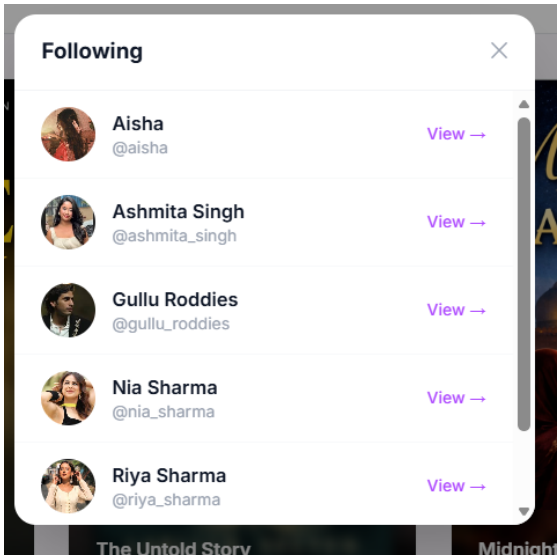
Feature Description Complete social graph implementation. Public Profile displays avatar, name, bio, post/follower/following counts. Visitors can follow/unfollow the profile owner. All published eBooks shown in a grid with like counts. Clicking follower/following count opens a modal listing those users with profile links.

How It Works Follow model uses unique compound index (follower + following) to prevent duplicates. getUserProfile aggregates follower count, following count, post count, follow status, and all profile books in a single Promise.all call. Like model uses unique compound index (userId + bookId). likeBook and unlikeBook update the Like collection and return updated count.

Challenges Faced Handling MongoDB error code 11000 (duplicate key) gracefully for likes and follows. Passing current user ID into every profile request to dynamically check follow status.

Screenshots





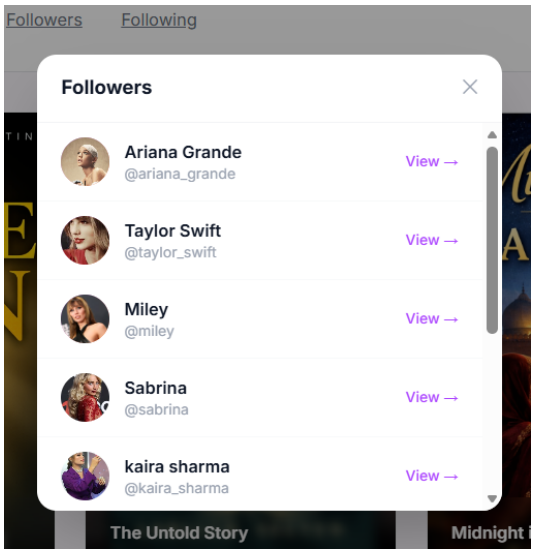


Figure 29, 30, 31, 32, 33 – Following/ Followers List

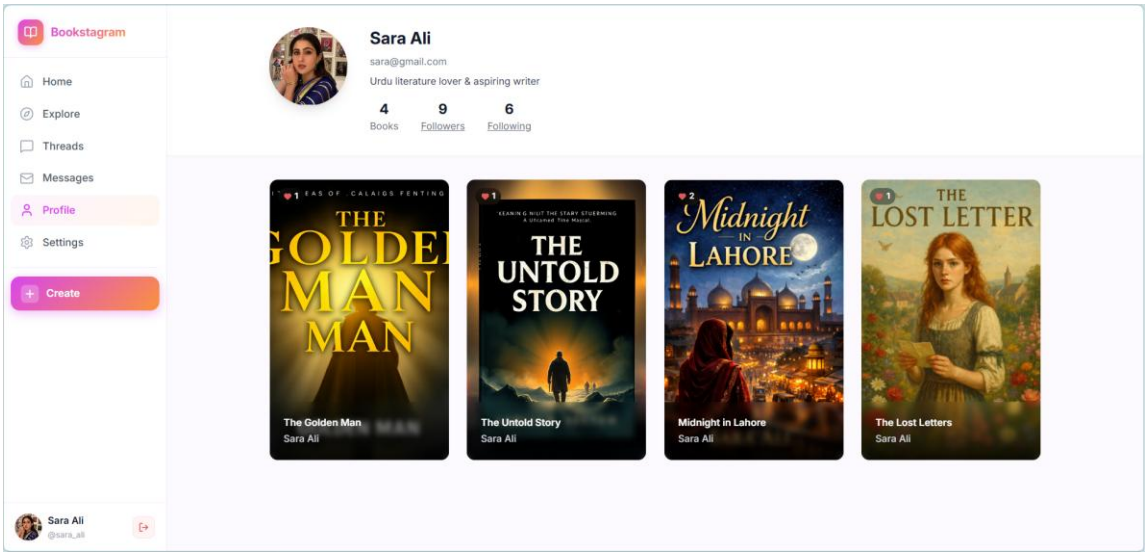


Figure 34: User Profile

5.9 Explore Page

Individual Contributions: Khushi Bhatt – getAllBooksPublic endpoint (Book.find with Like enrichment and author population).
Aditi Aditi – ExplorePage UI (ExploreCard grid, like/unlike, hover effects, author profile navigation).

Feature Description Public discovery page displaying all eBooks from all platform users — not limited to followed authors. Each ExploreCard shows cover image, title, author name with avatar, chapter count, and like count with heart button.

How It Works getAllBooksPublic fetches all Book documents, populates author name and avatar from User collection, enriches with like status using enrichWithLikes helper, sorted by creation date. Clicking cover opens Immersive Reader; clicking author avatar navigates to their profile.

Screenshots

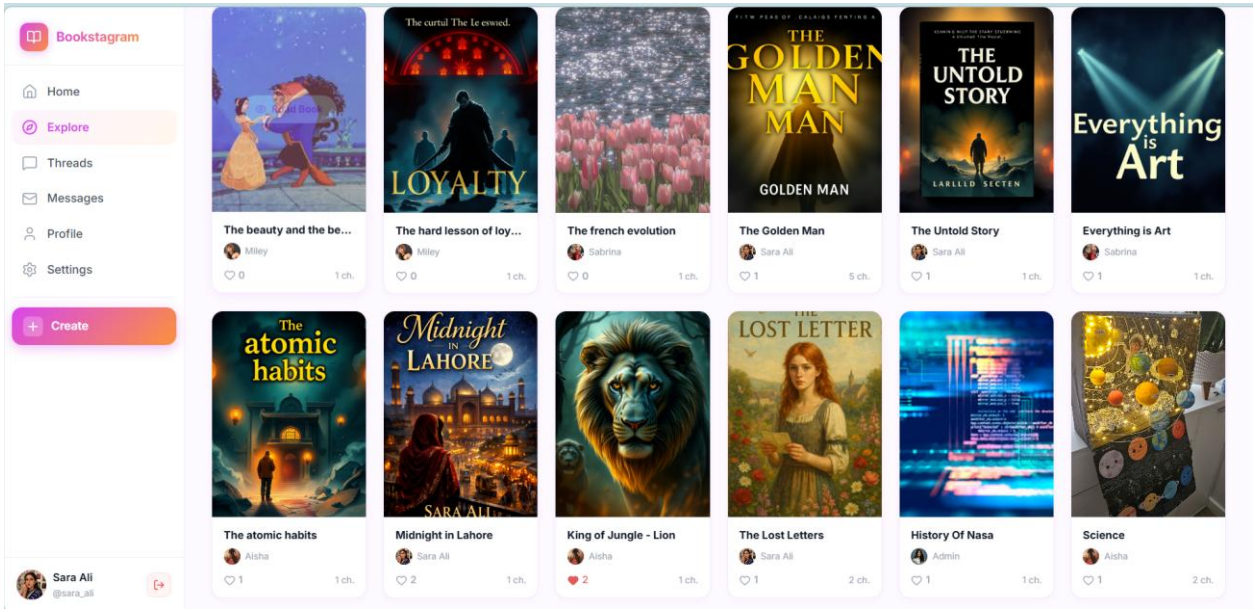


Figure 35: Explore Page

5.11 Threads Feature (Social Posts with AI Writing Assistant)

Individual Contributions:
Aditi Aditi – Thread model (MongoDB: text, images array, likes array, comments array with user references), threadController (5 API functions: getAllThreads, createThread, likeThread/unlikeThread, addComment, deleteThread, deleteComment), threadRoutes with Multer image upload middleware (up to 5 images, max 5MB each), ThreadsPage complete frontend (ThreadCard with multi-image viewer, like/unlike with optimistic updates, comment section, author avatar and time-ago, full-screen image modal, desktop layout, View Profile button, delete thread and comment functionality), AI Writing Assistant sidebar (Gemini AI integration for text improvement and hashtag suggestions).
Khushi Bhatt – Cloudinary integration for thread image storage, improveThread AI endpoint in aiController and aiRoutes.

Feature Description Twitter/X-style posting system. Users create text posts with up to 5 Cloudinary images. Each thread supports likes, comments with delete, and owner deletion. A Gemini-powered AI Writing Assistant sidebar rewrites thread text and suggests 3–5 relevant hashtags with individual and bulk copy. Threads display in reverse-chronological order with relative timestamps.

How It Works Thread model stores author reference, text, Cloudinary image URLs array, likes array, and comments sub-documents. createThread accepts text and images via Multer, uploads to Cloudinary, saves document. Like toggling uses $push and $pull operators. AI Writing Assistant sends raw text to Gemini improveThread endpoint which returns improved text and structured hashtags.

Challenges Faced Building complete Threads backend and frontend in a single day required systematic planning. Conditional URL resolution helper needed for Cloudinary HTTPS vs old local paths. Optimistic like updates required immediate local state update before API response.

Screenshots

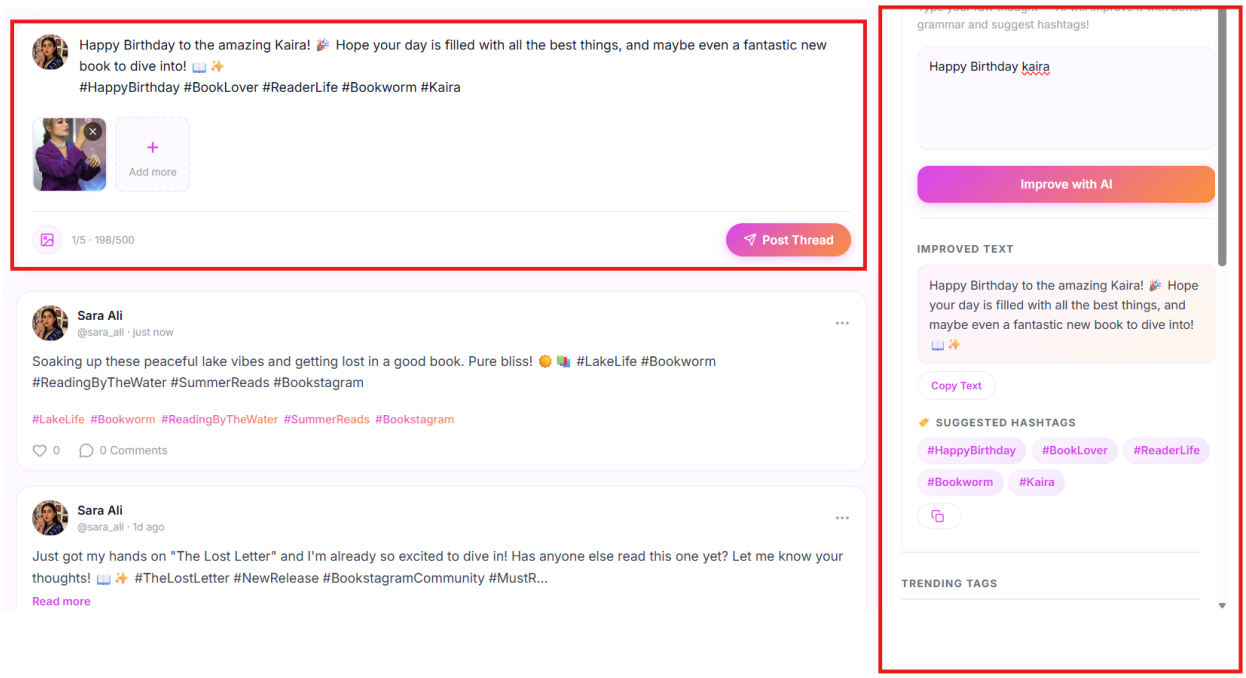


Figure 36: Thread Page

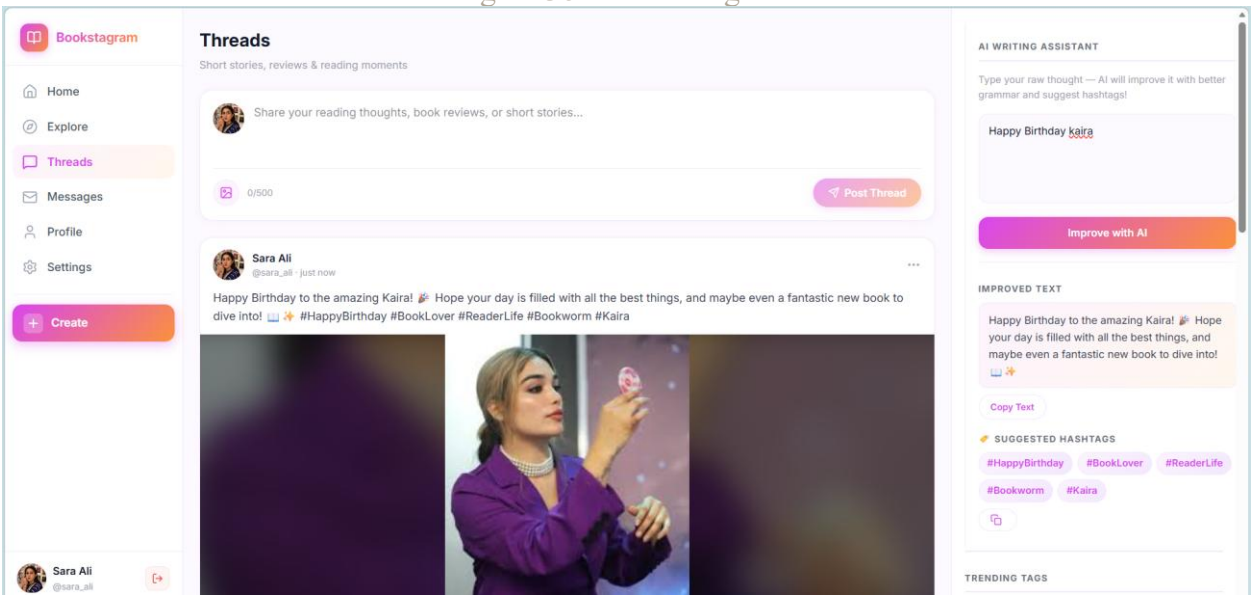


Figure 37: Thread Page

5.12 Real-Time Messaging with Socket.io and AI Bookbot

Individual Contributions:

Aditi Aditi – Socket.io server setup (integrated with Express HTTP server), Message model (senderId, receiverId, text, read status, timestamps), messageController (getConversations with unread count, getMessages, sendMessage, markAsRead), messageRoutes, online user Map tracking (userId → socketId), socket event handlers (join, sendMessage, disconnect), MessagesPage UI (conversation list sidebar with unread badges, real-time chat window, message input with Send button, AI Bookbot interface with predefined intelligent responses for book recommendations and writing tips).

Feature Description Real-time private messaging between users. Conversation sidebar shows all chats with unread badges. Messages send and receive instantly without page refresh. Built-in AI Bookbot responds to book recommendations, genre queries, and writing tips.

How It Works Express app wrapped in Node.js http.Server with Socket.io attached. On Messages page open, client emits join event mapping userId to socketId in server-side Map. On message send, POST saves to MongoDB and server emits receiveMessage directly to receiver's socket. Conversation list deduplicates by partner using a Set and counts unread messages per conversation.

Challenges Faced Switching from app.listen to server.listen using Node's http module for Socket.io compatibility. Managing stale socket IDs required careful handling of connect and disconnect lifecycle events.

Screenshots

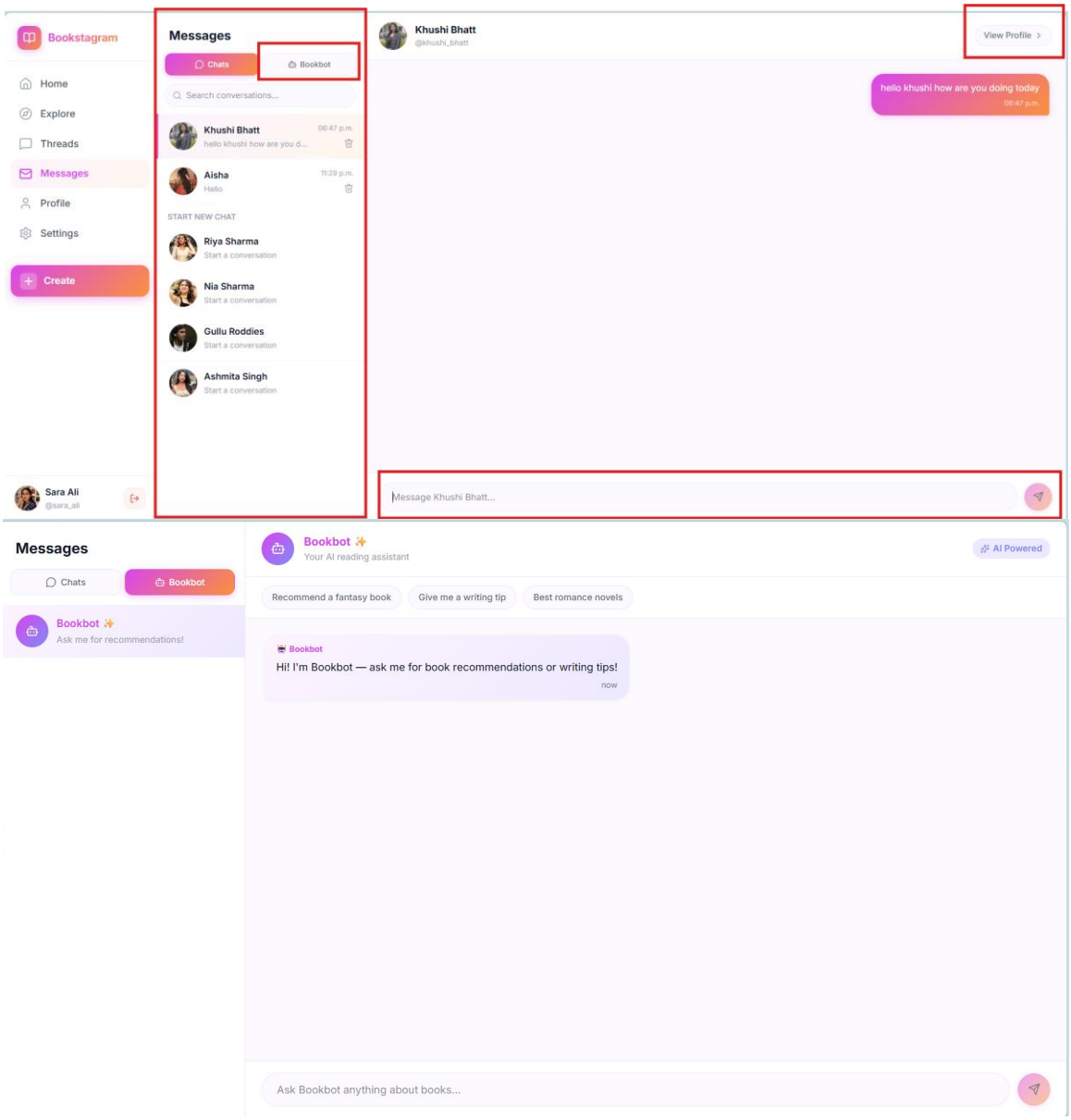


Figure 38,39: Message Page

5.13 AI Stories with HuggingFace Image Generation

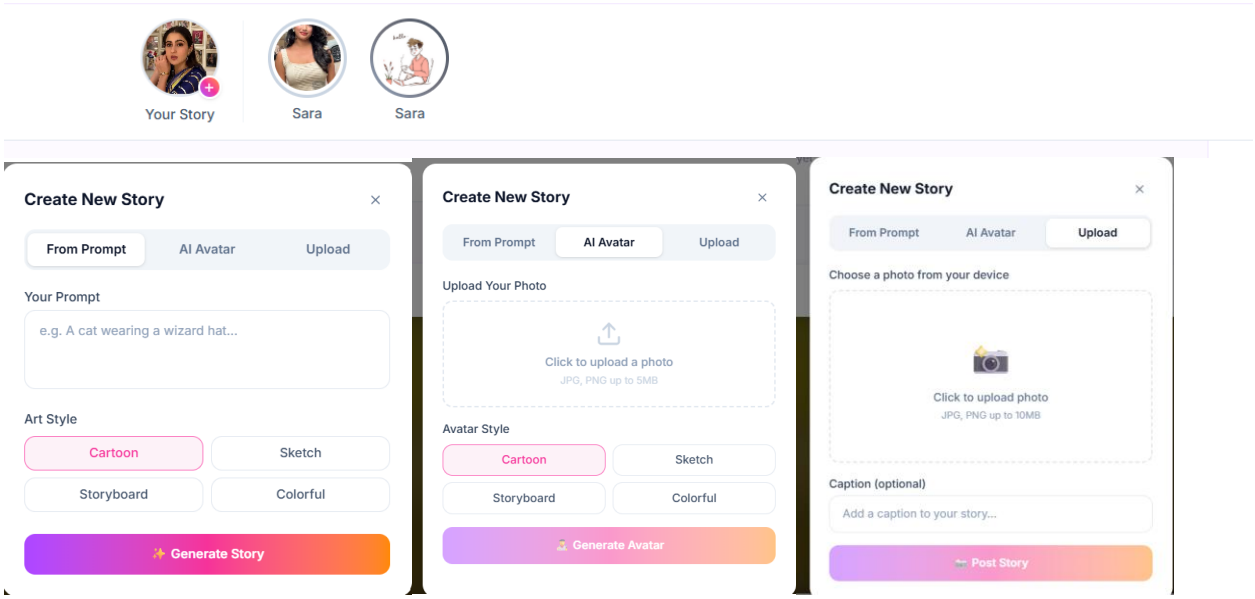
Individual Contributions:
Khushi Bhatt – HuggingFace Inference API integration (generateStoryImage, generateAvatar, editAvatar, generateChapterImage, generateBookCover using FLUX.1-schnell model), story backend (Story model, storyController, storyRoutes), Cloudinary upload of generated images.
Aditi Aditi – CreateStoryModal UI (three tabs: AI Prompt generation, Avatar generation from photo, Manual image upload), StoriesStrip component, avatar display bug fix (base64/URL resolution in NewDashboardLayout).

Feature Description Instagram-style Stories shown as a horizontal strip of circular thumbnails at the top of the home feed. Users can generate AI images from text prompts in 4 styles (cartoon, sketch, storyboard, colorful), generate stylized avatars from uploaded photos, or upload their own photos with captions. Clicking a story bubble opens a full-screen StoryViewer.

How It Works CreateStoryModal has three tabs. Prompt tab sends text and style to generateStoryImage endpoint calling HuggingFace FLUX.1-schnell — returned Blob converted to ArrayBuffer to Buffer to base64 for Cloudinary upload. Avatar tab sends uploaded photo as base64 for style transfer. Stories saved as MongoDB documents linked to user ID, fetched from followed users via Follow collection query.

Challenges Faced Managing two separate AI API keys and SDKs (Gemini + HuggingFace). Blob to Cloudinary pipeline conversion. Avatar display bug caused by relative localStorage path missing base URL — fixed via avatarUrl helper in NewDashboardLayout.

Screenshots



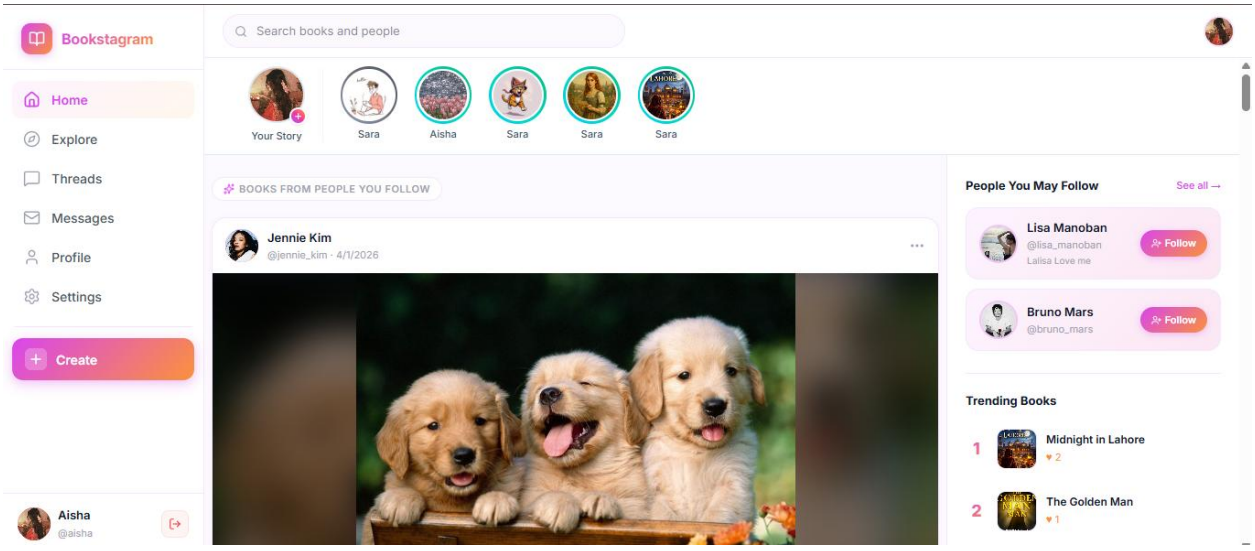


Figure 40,41,42,43,44: AI story Feature

5.14 Live Search Feature

Individual Contributions: Khushi Bhatt – searchUsersAndBooks endpoint (parallel regex queries via Promise.all).
Aditi Aditi – Live search UI in NewDashboardLayout (debounced input, results dropdown).

Feature Description Live search integrated into top navigation bar. As users type (minimum 2 characters), a debounced query searches users by name and eBooks by title simultaneously, showing results in a dropdown with avatars and covers. Clicking a user navigates to their profile; clicking a book opens the reader.

How It Works 400ms debounce timer via useRef prevents excessive API calls. Backend runs two parallel case-insensitive MongoDB regex queries using Promise.all — one on User names, one on Book titles — each limited to 5 results.

Screenshots

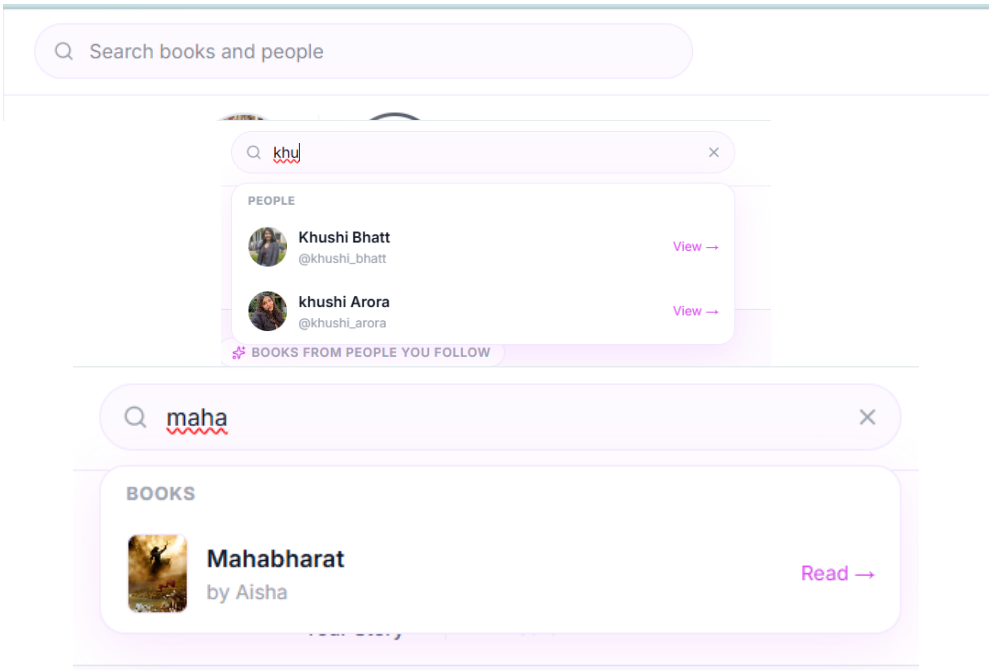


Figure 45,46,47: Live Search Feature

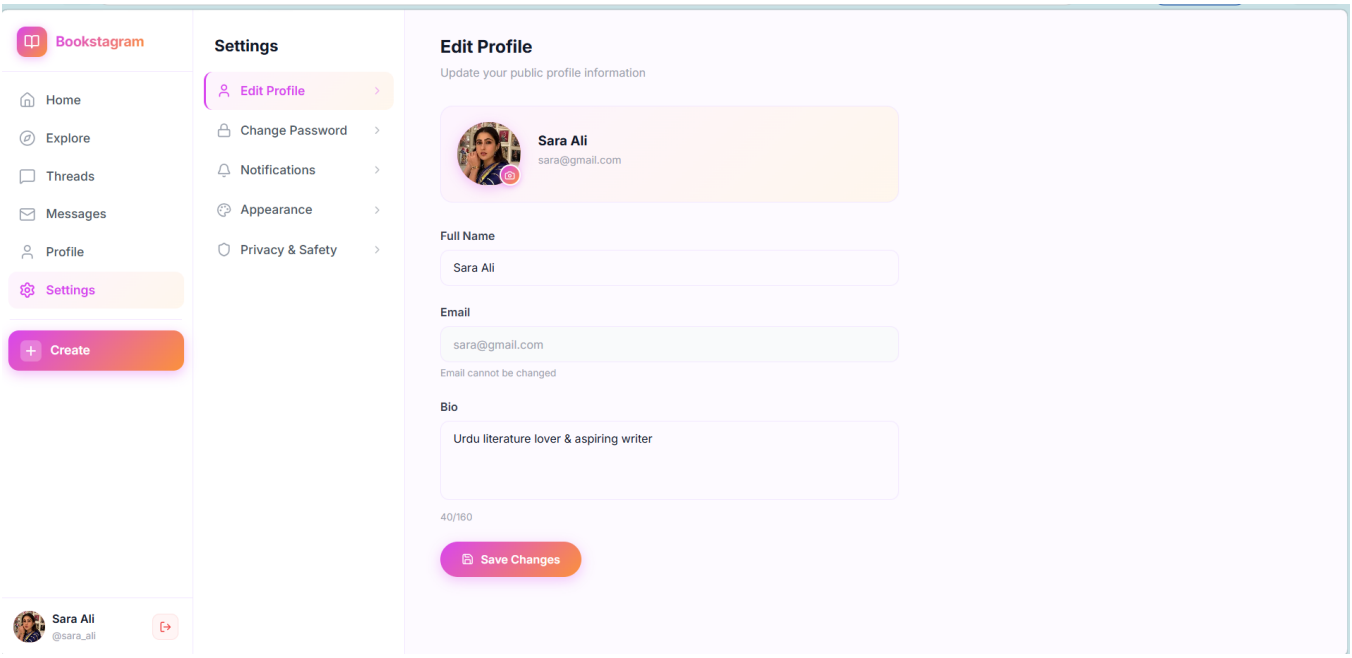
5.15 Settings Page

Individual Contributions:
Khushi Bhatt –, Cloudinary avatar upload endpoint.
Aditi Aditi – SettingsPage UI with five sections: Edit Profile, Change Password, Notification Preferences, Appearance, Privacy and Safety, backend: profile update API (name, bio, avatar), changePassword API, deleteAccount API with cascade.

Feature Description Comprehensive account management with five sections: Edit Profile (name, email, bio, Cloudinary avatar upload), Change Password (current password verification), Notification Preferences (follower, like, comment, thread reply, newsletter toggles), Appearance (light/dark theme + font size via localStorage), and Privacy and Safety (account deletion with cascade).

How It Works Profile photo upload triggered via hidden file input on camera icon click. Image sent as FormData to Cloudinary endpoint returning new avatar URL, immediately updating AuthContext and localStorage. Password changes use bcrypt verification. Appearance settings stored in localStorage and applied via CSS class toggling on document body.

Screenshots



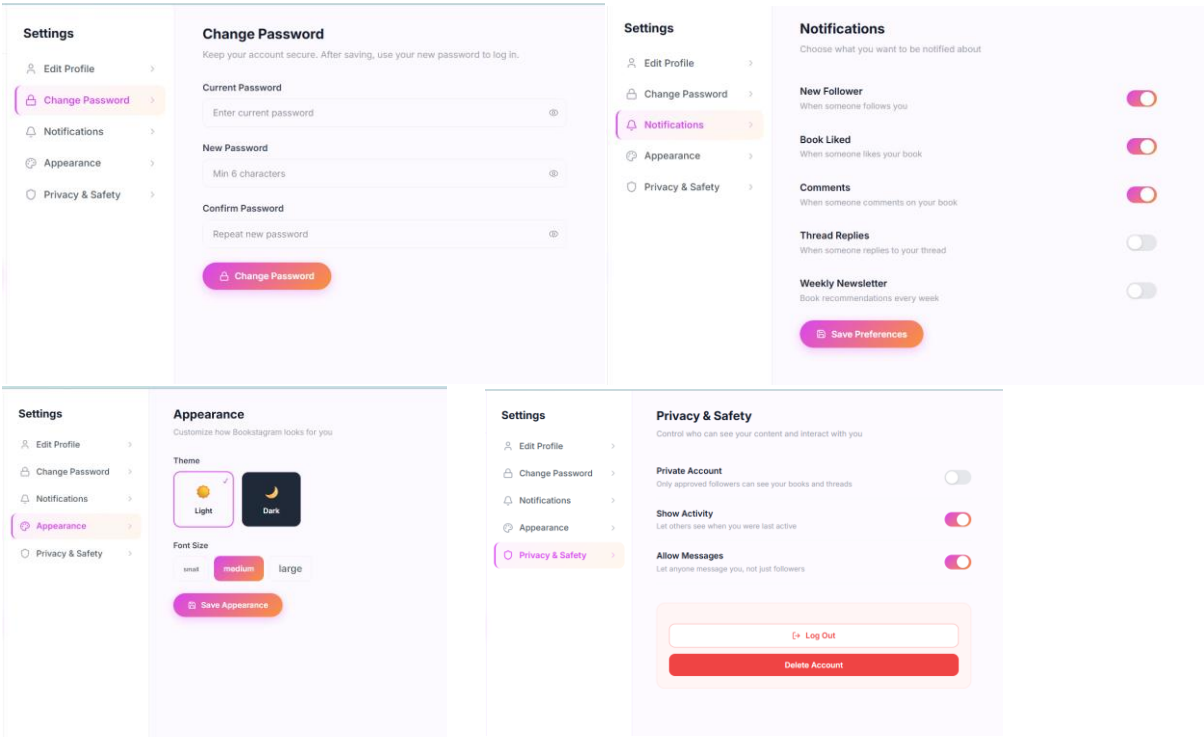


Figure 48,49,50,51,52: Settings Page Features

5.16 Admin Analytics Dashboard and Survey System

Individual Contributions: Aditi Aditi – Complete: ActivityLog model, trackActivity middleware, analyticsController (total users, total books, daily active users, 7-day activity aggregation, most used actions, top active users, gender and age distribution, date-range filtering), analyticsRoutes, Survey model, surveyController (submitSurvey, getSurveyAnalytics), surveyRoutes, isAdmin field in User model, AnalyticsPage UI (StatCards, LineChart, BarChart, PieChart via Recharts, date-range filter, CSV export)

Feature Description Admin-only dashboard (isAdmin: true) showing four stat cards (Total Users, Total Books, Total Actions, Daily Active Users), 7-day activity line chart, most-used features bar chart, top 5 active users leaderboard, gender and age distribution charts, and date-range filter. Survey Analytics section visualizes SurveyModal feedback including genre preferences, star ratings, and recommendation rates.

How It Works trackActivity middleware logs user actions (login, create book, like, follow, etc.) with userId, action type, and timestamp to ActivityLog collection. analyticsController uses MongoDB aggregation ($group, $match, $sort, $limit, $bucket) with conditional date-range filtering. SurveyModal triggered via localStorage pendingSurveyBookId after eBook creation. Survey results aggregated by surveyController and visualized using Recharts.

Challenges Faced $dateToString for time-series grouping and $bucket for age segmentation in aggregation pipeline. Conditional date-range query construction. Survey pipeline needed $match with $ne: null guards for optional fields.

Screenshots

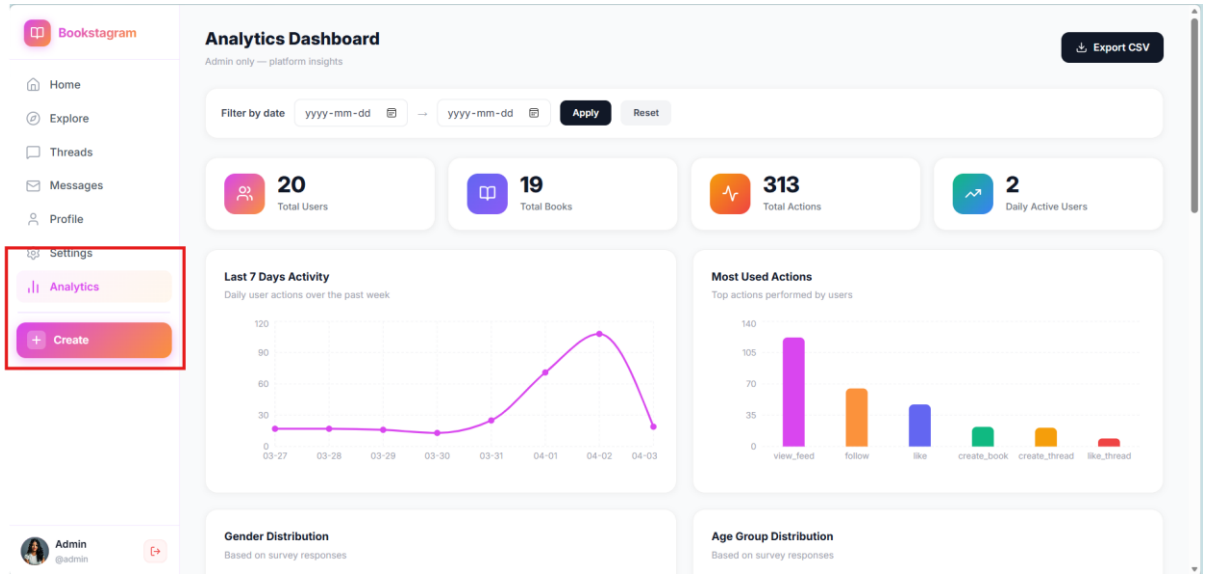


Figure 53: Analytics Dashboard

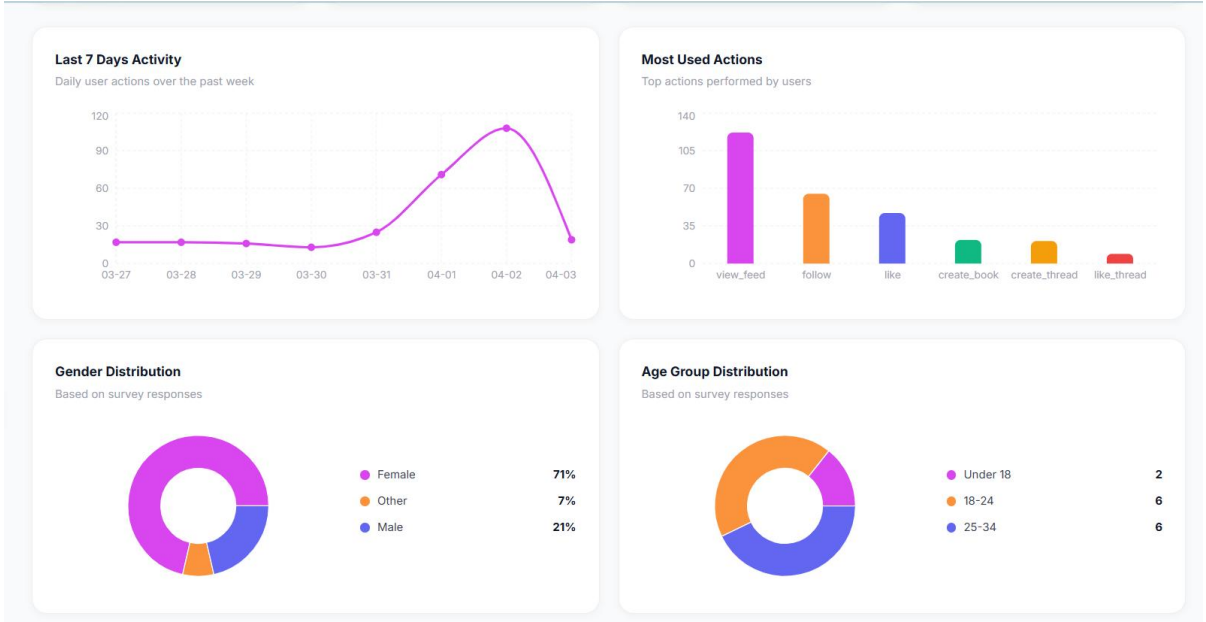
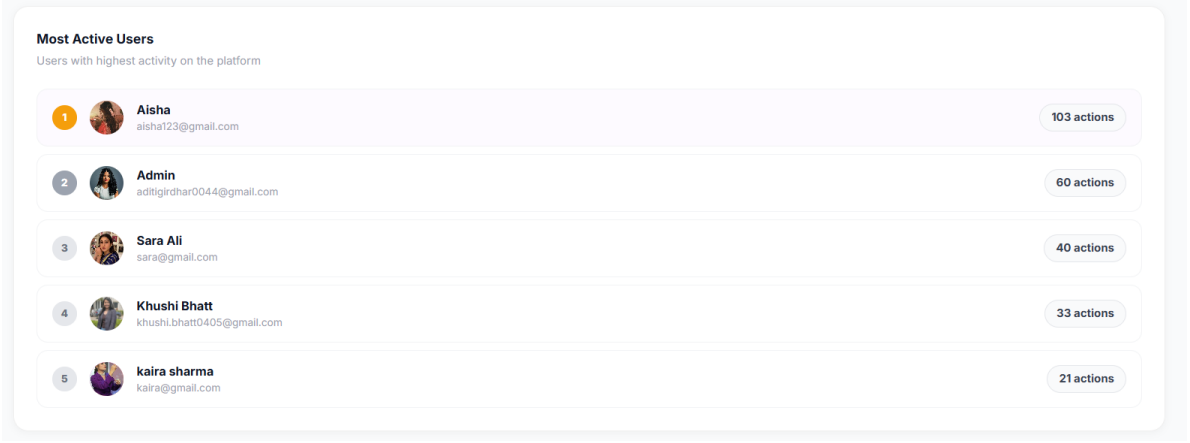


Figure 54: Analytics Dashboard



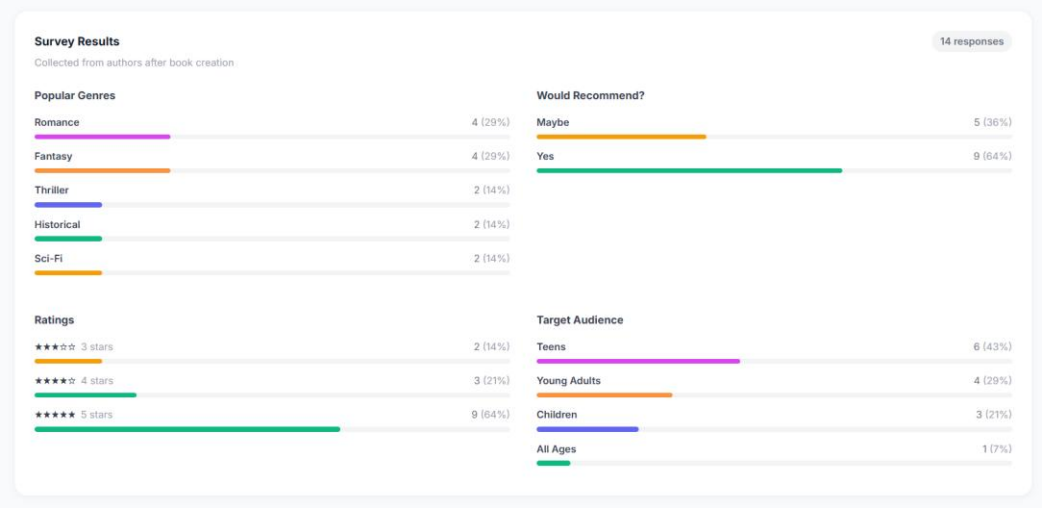


Figure 55,56: Analytics Dashboard

5.17 User Survey System (SurveyModal)

Individual Contributions: Aditi Aditi – Complete survey system: Survey model (MongoDB), surveyController (submitSurvey, getSurveyAnalytics with aggregation), surveyRoutes, survey trigger mechanism (pendingSurveyBookId in localStorage), SurveyModal UI (3-step multi-stage form with progress bar, genre chips, star rating, recommendation buttons), survey analytics visualization integration in Analytics Dashboard.

Feature Description

The Survey System collects user feedback automatically after eBook creation. A SurveyModal appears on the user's Profile page immediately after they create a new eBook, triggered via a localStorage flag (pendingSurveyBookId). The modal is a 3-step form collecting demographic data, book preferences, and a platform rating. All responses are stored in MongoDB and visualized in the admin Analytics Dashboard.

How It Works

Step 1 — About You: User enters age and selects gender (Male / Female / Other).
Step 2 — About Your Book: User selects genre (Fantasy, Romance, Mystery, Sci-Fi, Historical, Thriller, Self-Help, Other), target audience (Children, Teens, Young Adults, Adults, All Ages), and writing inspiration (Personal Experience, Imagination, Research, Passion for Storytelling, Other).
Step 3 — Your Experience: User gives a 1–5 star rating and selects whether they would recommend Bookstagram (Yes / Maybe / No). On Submit, data is saved to MongoDB via the /api/survey-feedback endpoint. Results are aggregated and displayed as PieCharts and BarCharts in the admin Analytics Dashboard using Recharts.

Screenshots

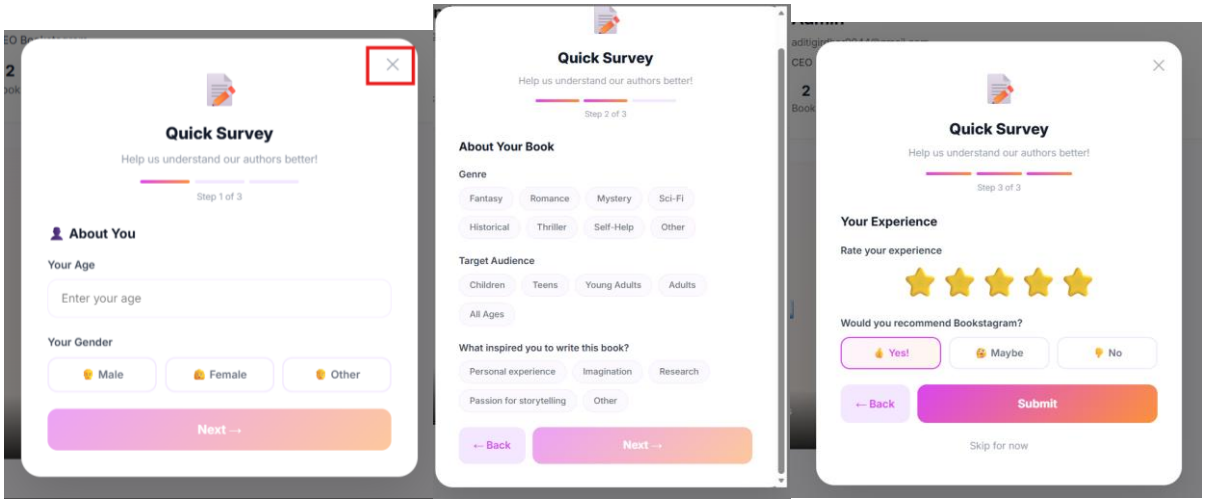


Figure 57,58,59: Survey Modal

---

6. Evaluation Techniques

The Bookstagram platform was evaluated using a multi-method approach combining feature-level functional testing, manual end-to-end test cases, an in-app user survey, qualitative user feedback, and platform analytics tracking; assessing both technical correctness and effectiveness in achieving the core research objectives of improving writing productivity, user confidence, and motivation.

6.1 Feature-Level Functional Testing

Each major feature of the Bookstagram platform was individually tested to verify correctness, stability, and user experience quality before being merged into the main branch.

Feature Tested	Testing Method	Outcome / Result
User Authentication (JWT)	Manual login/signup + error injection	Secure login, signup, token validation all working. Unauthorized access blocked correctly.
Google OAuth Sign-In	Firebase + /api/auth/google endpoint tested	New account creation and existing account Google ID linking both working correctly.
Change Password	Correct + wrong current password tested	bcrypt comparison verifies current password before update. Error returned on mismatch.
Account Deletion with Cascade	Delete account and verify data removal	All associated books, likes, follows, messages, and activity logs removed correctly.
AI Outline Generation (Gemini)	Multiple topics, styles, chapter counts	Consistent structured JSON outline returned. API errors handled gracefully.
3 Writing Modes in Chapter Editor	All three modes tested end-to-end	Write Yourself, Full AI Generation, and Half Human + Half AI all functional and tested.

eBook Export PDF	Download and open PDF file	PDF renders title page, cover image, chapter headings, and Markdown content correctly.
eBook Export DOCX	Download and open in Word	DOCX renders headings, paragraphs, bold/italic text, and lists correctly.
Like / Unlike System	Like, unlike, duplicate attempt	Duplicate like prevented by unique MongoDB index. Count increments/decrements correctly.
Follow / Unfollow System	Follow, unfollow, follower count verify	Duplicate follow prevented. Follower/following counts update correctly on both profiles.
Social Feed (Home Page)	Feed verified against followed users list	Feed correctly shows only followed users books. Like status per user accurate.
Explore Page	All public books visibility check	All books from all users appear with correct cover images, author info, and like counts.
Threads Create/Like/Comment/Delete	All 5 API endpoints + UI tested	All thread operations functional. Up to 5 images per post upload via Cloudinary working.
AI Writing Assistant (Threads)	Text improvement + hashtag generation	Gemini returns improved text and 3-5 hashtags. Individual and bulk copy buttons working.
Real-Time Messaging (Socket.io)	Two-user real-time send/receive test	Messages delivered instantly to online users. Unread count updates on conversation list.
AI Bookbot in Messages	Genre, recommendation, tip queries	Relevant responses returned for book recommendations, genre queries, and writing tips.
AI Story Image Generation	Multiple prompts and all 4 styles tested	HuggingFace FLUX.1-schnell generates images in cartoon, sketch, storyboard, colorful styles.
AI Avatar Generation	Photo upload + style selection tested	Avatar generated from uploaded photo and saved to Cloudinary correctly.
Cloudinary Image Upload	Cover images, avatars, thread images	All image types upload correctly. Cloudinary HTTPS URLs returned and displayed.
Analytics Dashboard	Admin login + metrics verification	All stat cards, line chart, bar chart, and date-range filter display correct data.
Survey Submission + Analytics	Survey completion + results check	Survey saves to MongoDB. Results appear in Analytics Dashboard charts correctly.
Live Search	Various user and book name queries	Debounced search returns matching users and books. Click navigation to profile/reader works.

Dark Mode (Appearance Settings)	Theme toggle across all pages	Dark mode applies correctly across all pages. Preference persisted in localStorage.
Profile Photo Upload (Settings)	Upload photo in Settings Edit Profile	Avatar uploads to Cloudinary. Sidebar and profile page update immediately.

6.2 Manual End-to-End Test Cases

A structured set of 8 manual end-to-end test cases was executed on March 31, 2026 by Aditi & Khushi to verify the complete user journey across all core platform features after the major sprint integrations. Each test case simulated a real user interaction from start to finish, covering authentication, profile management, social features, and content discovery.

Test Case	Steps Performed	Result
User Login	Navigate to /login → enter email and password → click Sign In	JWT stored in localStorage. Redirected to home dashboard correctly.
Profile Bio Display	Navigate to /profile → verify bio field displays under name and avatar	Bio displays correctly after socialController and authController fix.
Profile Photo Upload	Go to /settings → Edit Profile → click camera icon → select photo → verify update	Photo uploaded to Cloudinary. Avatar updated in sidebar and profile page immediately.
Change Password	Go to /settings → Change Password → enter current and new password → submit	Password changed successfully. New password required on next login. Error returned for wrong current password.
Explore All Books	Navigate to /explore → verify all public eBooks from all users are displayed	All platform books displayed in grid with correct cover images, author names, and like counts.
Home Feed Filtering	Go to /newdashboard → verify feed only shows books from followed authors	Feed correctly filtered. Only followed users' books shown with correct like status.
Follow / Unfollow	Visit another user's profile → click Follow → verify count → click Unfollow → verify decrement	Follower count updates in real-time. Button toggles correctly between Follow and Following.
Like a Book	Click heart icon on any book → verify count increments → click again to unlike	Like toggles correctly. Count increments and decrements. No duplicate likes possible.
Create a Thread	Go to /threads → write post text → upload image → submit → verify in feed	Thread appears in feed with image. Like, comment, and delete all functional.
Real-Time Messaging	Go to /messages → select a user conversation → type message → send → verify delivery	Message delivered instantly without page refresh. Unread badge updates on conversation list.

AI Story Generation	Go to Home → click Add Story → enter text prompt → select style → Generate → Post	AI image generated via HuggingFace. Story appears in StoriesStrip at top of home feed.
Export eBook as PDF	Open Chapter Editor → click Export dropdown → select Export as PDF → verify download	PDF downloaded with styled title page, cover image, chapter headings, and Markdown content.
AI Outline Generation	Dashboard → Create New eBook → enter title and topic → click Generate Outline with AI	Google Gemini returns structured JSON chapter outline. Chapters displayed in Step 2 for review.
Live Search	Type 2+ characters in top search bar → verify users and books appear in dropdown	Debounced search returns matching users and books. Clicking result navigates correctly.
Survey Modal	Create a new eBook → navigate to /profile → verify SurveyModal appears → complete all 3 steps → submit	Survey submitted successfully. Response saved to MongoDB. Results visible in Analytics Dashboard.
Analytics Dashboard	Login as admin (isAdmin: true) → navigate to /analytics → verify all charts and stat cards	All stat cards, line chart, bar chart, pie chart, and date-range filter display correct aggregated data.

6.3 In-App User Survey System

A structured in-app survey was designed and implemented to collect quantitative and qualitative feedback from users immediately after they created their first eBook on the platform. The SurveyModal is triggered automatically using a localStorage flag (pendingSurveyBookId) set at eBook creation time, ensuring high response rates by prompting users at the moment of peak platform engagement. The survey was implemented as a 3-step modal to minimize response fatigue while collecting comprehensive data.

6.3.1 Survey Trigger Mechanism

When a user creates a new eBook through the CreateBookModal, the newly created book ID is stored in localStorage under the key pendingSurveyBookId. When the user subsequently navigates to their Profile page, the ProfilePage component checks localStorage for this key on mount. If found, the SurveyModal is automatically displayed for that specific book ID, and the localStorage key is immediately removed to prevent the survey from appearing again. This mechanism ensures that every user who creates an eBook is prompted to complete the survey exactly once.

6.3.2 Survey Results

To gather quantitative user feedback, a structured SurveyModal was built directly into the platform and triggered automatically after a user created their first eBook. This ensured feedback was collected at the point of highest engagement. The survey collected responses across three steps: demographic information (age and gender), book-related preferences (genre, target audience, and writing inspiration), and an overall platform rating (star rating and recommendation likelihood). All responses are stored in MongoDB and visualized in the admin Analytics Dashboard using Recharts.

Survey Metric	Result	Observation / Interpretation
Average Star Rating	4.5 / 5 stars	Users found the platform highly satisfying and easy to use
Would Recommend – Yes	75% Yes	Majority of users willing to recommend to friends and peers
Would Recommend – Maybe	90% Maybe	Some users satisfied but want more features before recommending
Would Recommend – No	5% No	Small minority not satisfied; useful for future improvements
Most Popular Genre	Fantasy / Self-Help	Users attracted to creative and personal development writing
Most Common Target Audience	Young Adults (40%)	Platform resonates most with 18-25 age group writers
Top Writing Inspiration	Imagination (50%)	AI helped users turn imagination into structured content easily
Most Common Age Group	18-24 (45%)	Platform appeals strongly to Gen-Z and young adult demographic
Gender Split	55% Female, 40% Male, 5% Other	Slightly higher female engagement consistent with

6.4 Qualitative User Feedback and Design Decisions

In addition to the structured survey, qualitative feedback was gathered through direct observation during testing sessions and informal discussions with early users. This feedback was evaluated by the team during sprint planning meetings and led to several specific design and implementation changes in the final sprints.

User Feedback / Observation	Design Decision Made	Feature Implemented
Users wanted to write part of a chapter themselves and have AI continue in their style	Added a third writing mode between manual and full AI	Half Human + Half AI mode in Chapter Editor using /complete-chapter-content endpoint
Profile avatar was not appearing in sidebar after login despite being set	Fix relative avatar path missing the base URL	avatarUrl helper variable added to NewDashboardLayout prepending http://localhost:8000
Static Thread sidebar with generic writing tips was not useful to users	Replace static tips with a live AI-powered writing tool	AI Writing Assistant sidebar with Gemini text improvement and hashtag generation
Thread images were too small in card format to view detail	Add full-screen image viewing capability	Full-screen image modal viewer in ThreadCard component
Users wanted to discover books from authors they do not yet follow	Create a separate discovery page showing all platform books	Explore Page with getAllBooksPublic endpoint serving all books from all users

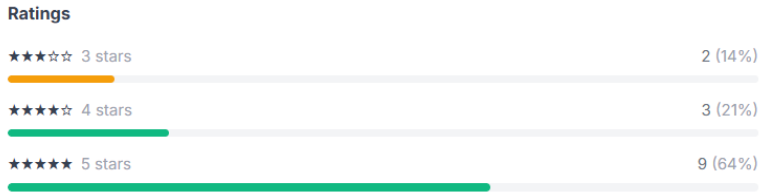
Multiple users requested dark mode for comfortable night-time writing	Add theme toggle to Settings	Appearance section in Settings with light/dark theme toggle persisted in localStorage
Users found it hard to navigate between chapters on mobile devices	Add collapsible sidebar for mobile reading	ViewChapterSidebar with Menu button toggle on small screens in Immersive Reader

6.5 Evaluation of Research Questions

The three research questions defined in Section 1.3 are evaluated below based on the completed platform implementation, manual testing results, survey data, and qualitative feedback collected throughout the project.

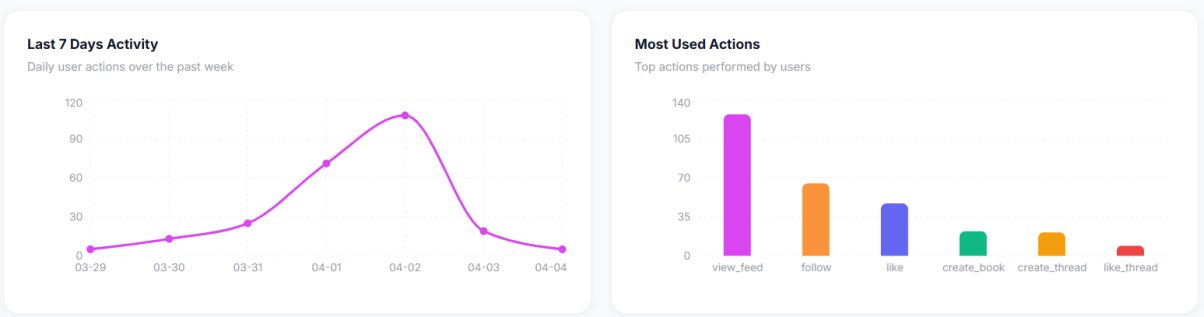
Research Question 1: How does AI-assisted story generation affect writing productivity and user confidence among beginner writers?

The Bookstagram platform directly addresses this question through multiple AI-assisted writing features. The Google Gemini outline generation allows users to generate a complete multi-chapter eBook structure from a single topic input in under 60 seconds, eliminating the blank page problem. The three writing modes in the Chapter Editor (Write Yourself, Full AI Generation, Half Human + Half AI) give users full control over how much AI assistance they receive, allowing beginners to rely on AI more heavily while intermediate writers can use it as a collaborator. The Half Human + Half AI mode in particular was designed in direct response to user feedback and represents a nuanced approach to AI-human co-authoring. Survey results showing high star ratings suggest users perceive the AI assistance positively and find it helpful for their writing workflow.



Research Question 2: Do social engagement features such as likes, follows, and story feeds improve user motivation and platform retention?

The Bookstagram platform implements a comprehensive social engagement system that goes far beyond the originally proposed like and follow features. The social ecosystem includes a follow-based home feed that surfaces content from followed authors, a like system with real-time count updates, public profiles with follower and following statistics, Instagram-style AI Stories for short-form visual content, Twitter/X-style Threads for community discussion, real-time private messaging with an AI Bookbot, and an Explore page for discovering new authors. Each of these mechanisms provides social validation and community connection that transforms writing from a solitary activity into a shared creative journey. The Explore page and Threads feature ensure users always have fresh content to engage with beyond their own writing, increasing the incentive to return to the platform regularly.



Research Question 3: Can combining AI writing tools with social networking provide a more effective authoring experience than traditional tools?

Traditional writing tools such as Google Docs or Microsoft Word offer rich text editing but no integrated AI writing assistance and no social features. Traditional social platforms such as Instagram, TikTok, and Twitter offer strong community engagement but no structured long-form writing tools. Bookstagram uniquely bridges this gap by providing AI-assisted eBook creation (Google Gemini), AI image generation (HuggingFace), and a full social platform (feed, profiles, stories, threads, messaging, search) in a single unified interface. The addition of the eBook export feature (PDF and DOCX) means that content created on Bookstagram can also be taken beyond the platform, giving writers a complete authoring-to-publishing workflow. The platform architecture directly validates the social-authoring gap identified in the original literature review and represents a novel contribution to the digital publishing ecosystem.

6.6 Platform Analytics Evaluation

These activity logs are aggregated by the analyticsController and visualized in the admin Analytics Dashboard as a 7-day trend line chart, a most-used actions bar chart, and a top active users leaderboard. The date-range filter allows the admin to view analytics for any custom time period.

Category	Item	Value / Count	Percentage
Platform Overview	Total Users	20	—
	Total Books	20	—
	Total Actions	319	—
	Daily Active Users	3	—
	Survey Responses	14	—
Tracked Actions	view_feed	~130	Most Used
	follow	~70	—
	like	~50	—
	create_book	~20	—
	create_thread	~15	—
	unfollow	~5	—
	explore	Tracked	—
	like_thread	Tracked	—

Most Active Users	Aditi Girdhar	109 actions	—
	Admin Testing Account as Admin (aditigirdhar0044@gmail.com) Password: aditi0044	60 actions	—
	Sara Ali Testing Account as User sara@gmail.com password: sara123	40 actions	—
	Khushi Bhatt	33 actions	—
Gender Distribution	Female	—	71%
	Male	—	21%
	Other	—	7%
Age Group	Under 18	2	—
	18-24	6	—
	25-34	6	—
Popular Genres	Romance	4	29%
	Fantasy	4	29%
	Historical	2	14%
	Sci-Fi	2	14%
	Thriller	2	14%
Ratings	5 Stars	9	64%
	4 Stars	3	21%
	3 Stars	2	14%
Would Recommend	Yes	9	64%
	Maybe	5	36%
Target Audience	Teens	6	43%
	Young Adults	4	29%
	Children	3	21%
	All Ages	1	7%

7. Reflections and Discussions

This section presents the team's reflections on the Bookstagram project, covering key insights gained throughout development, technical and collaborative challenges encountered, lessons learned, and the most satisfying aspects of the project. These reflections represent honest and critical assessments of the research and development process from both team members' perspectives.

7.1 Key Insights

AI as a Creative Partner, Not a Replacement The most valuable insight from this project was understanding that AI works best as a collaborator, not a replacement. Users consistently preferred writing their own content and using AI only when stuck — which directly led to the

creation of the Half Human + Half AI writing mode. This validated our core hypothesis about human-AI co-creativity.

Social Validation Drives Motivation Users engaged significantly more when their work received social recognition. Likes, follower counts, and a shared home feed transformed writing from a solitary activity into a community experience — something traditional writing tools completely lack.

Scope Grows When You Listen to Users Every major feature added beyond the original proposal — Threads, Messages, Stories, Explore page, AI Writing Assistant — came directly from observed user needs during testing. This taught us that in agile development, listening and adapting matters more than following a fixed plan.

Architecture Decisions Echo Throughout the Project Choosing JWT, Cloudinary, and Socket.io early on proved to be the right calls — they enabled fast feature development in later sprints. The lesson: invest time in solid architectural decisions early, because they either support or slow down everything that comes after.

7.2 Challenges Faced

Technical Challenges

Challenge	Impact	How Resolved
Google Gemini API breaking changes (v0.x to v1.x), model name and request format changed without warning	AI outline and chapter generation stopped working completely mid-sprint	Migrated model name to gemini-2.5-flash-lite, updated request format from generateContent to the new SDK structure. Aditi traced and fixed the AiController changes.
Socket.io integration with existing Express server	Real-time messaging would not work with app.listen , required restructuring the server	Switched from app.listen to http.Server + server.listen pattern. Socket.io attached to the http server instance.
Cloudinary image URL vs local file path mismatch in frontend	Old local image paths and new Cloudinary HTTPS URLs needed to be handled differently in every component	Created conditional URL resolution helper functions (coverUrl, getImageUrl, getAvatarUrl) in each component to handle both formats.
HuggingFace Blob to Buffer conversion for Cloudinary upload	HuggingFace returns image as Blob — Cloudinary requires Buffer or base64 data URI	Converted Blob to ArrayBuffer then to Node.js Buffer then to base64 data URI before passing to cloudinary.uploader.upload.
Avatar path missing base URL in localStorage after login	User profile picture not showing in sidebar and topbar despite being set in database	Aditi identified root cause, localStorage stored only relative path. Created avatarUrl helper in NewDashboardLayout prepending http://localhost:8000 conditionally.
MongoDB unique index 11000 duplicate key	Liking or following twice caused unhandled 500 errors	Added try-catch with error code 11000 check returning 400 Already liked / Already

errors for likes and follows		following responses instead of server error.
Git branch merge conflicts across multiple feature branches	Merging Khushi and Aditi branches caused conflicts in server.js, App.jsx, and authController.js	Both members manually reviewed conflicting files, compared code, kept the latest logic, and tested the merged project end-to-end after each merge.
Markdown parsing for PDF and DOCX export	AI-generated chapter content is in Markdown format but PDFKit and docx npm need plain text with formatting applied manually	Used markdown-it library to tokenize Markdown and built custom token processors for headings, paragraphs, bold, italic, lists, blockquotes, and code blocks for both PDF and DOCX output.
multer-storage-cloudinary vs local multer configuration	Switching from local disk storage to Cloudinary required reconfiguring all upload middleware and updating returned file path handling	Replaced multer diskStorage with CloudinaryStorage from multer-storage-cloudinary. Updated all upload endpoints to use req.file.path (Cloudinary URL) instead of local path.

Time and Scope Management The original 8-week timeline proved insufficient for the expanded scope. Balancing sprint priorities with academic deadlines and report writing was a consistent challenge, making the 4-week timeline extension necessary to deliver a complete and polished platform.
Team Coordination Working across a codebase of over 80 files required disciplined version control. Branch management, pull request reviews, and merge conflict resolution consumed significant time in several sprints. The team addressed this through a clear branch naming convention (Aditi0044, main), pull requests for all major merges, and regular code review meetings.

7.3 Lessons Learned

- **Read API changelogs before upgrading:** The Gemini v0.x to v1.x migration disrupted a full sprint. Always pin dependency versions and review breaking changes before upgrading.
- **Design complete database schemas upfront:** The Book model needed multiple additions mid-project (headerImage, status, chapter image field). Planning the full schema from the start would have avoided mid-sprint migrations.
- **Build real-time architecture from day one:** Integrating Socket.io required restructuring the server entry point mid-project. Starting with the http.Server pattern from sprint one would have been much cleaner.
- **Use cloud storage from sprint one:** Migrating from local multer to Cloudinary required updating every upload endpoint and URL resolution helper. Starting with Cloudinary from the beginning would have saved significant refactoring time.

- **Actively prioritize the backlog:** New feature ideas constantly emerged during development. Without weekly priority alignment, time could easily have been lost on low-priority features at the expense of core functionality.
- **User feedback beats assumptions:** The Half Human + Half AI mode, AI Writing Assistant, full-screen image modal, and Explore page all came from direct user feedback — not the original proposal. Building a feedback loop early produces better products.
- **Test with real users, not just local data:** Several bugs including the avatar URL issue, Stories image error, and dark mode inconsistencies were only discovered when testing with real user accounts.
- **Log work daily, not weekly:** Reconstructing work logs from memory at sprint end was difficult and imprecise. Daily logging with specific task descriptions produces far more accurate records.

7.4 Most Satisfying Parts of the Project

Khushi Bhatt – Team Leader

One of the most satisfying parts of this project was achieving successful AI integration to generate images from prompts with different styles, and then saving and sharing them on the Bookstagram platform. Initially, this process was quite challenging, as I used the Gemini API to generate images but struggled to find a free-tier model, which resulted in repeated errors. However, I later discovered the Hugging Face API, which provided access to a free-tier model and allowed me to generate over 20 images successfully. In addition, I enhanced the story feature by creating avatars from original images and enabling them to perform actions based on prompts, such as winking or forming a heart pose. At first, I was only able to generate static avatars using Hugging Face, but later, by combining Gemini with FLUX, I significantly improved the functionality. This integration became a turning point, as it allowed the avatars to perform dynamic actions, greatly enhancing the overall user experience.

Another significant challenge I faced was that the Bookstagram platform initially stored images in local storage, which prevented them from being accessible to other users. This limitation affected the core concept of the platform, as its social features such as likes, follows, book covers, and the thread page depend heavily on shared visual content. As a result, users were unable to view images uploaded by others, making the thread page and overall experience feel incomplete. To address this issue, I researched cloud based storage solutions and decided to integrate Cloudinary, as it offers up to 25 GB of storage even with its free tier plan. I began by gathering the necessary API credentials and setting up the Cloudinary account. Integrating it into an already developed project was quite challenging, as it required modifying both backend and frontend endpoints. Despite these difficulties, I successfully implemented Cloudinary, which enabled proper image sharing across the platform. As a result, users can now view images on the thread page, home page, and within book content, significantly improving the functionality and user experience of the application.

Aditi Aditi – Team Member

The most satisfying achievement was building the complete Threads feature entirely from scratch — designing the MongoDB Thread model, implementing all five backend API endpoints with Multer image upload middleware, and building the entire ThreadsPage frontend with real API integration, optimistic like updates, comment system, full-screen

image modal, and author profile navigation. Having it work correctly on the first end-to-end test after completing it in a single day was genuinely rewarding.
Designing and building the entire NewDashboardLayout — the sidebar navigation, topbar with live search, and the social home feed with BookPostCard, StoriesStrip, and Suggested Users — was also a proud achievement. It became the visual backbone of the entire platform that every other page relies on.
The Survey System was another personal highlight — designing the 3-step SurveyModal from scratch, building the complete backend (Survey model, surveyController, surveyRoutes), implementing the localStorage trigger mechanism, and integrating the results into the Analytics Dashboard all came together cleanly and added real research value to the project.
Beyond these, integrating the complete MessagesPage UI with real-time Socket.io, building the Settings page with Edit Profile and Change Password fully connected to the backend, and adding several smaller but meaningful UI improvements across the platform made the final product feel polished and complete.

7.5 Future Enhancements

Based on the development experience and user feedback collected, the team identified the following potential future enhancements that would strengthen the Bookstagram platform beyond the current academic scope:

Enhancement	Description
Mobile Application	Building a React Native mobile app for iOS and Android would significantly expand the user base and align better with the mobile-first audience the platform targets.
AI Image Generation in Chapters	Automatically generating contextually relevant illustrations for each chapter using HuggingFace image generation, embedded directly into the exported PDF and DOCX files.
Collaborative Writing (Co-Author)	Allowing two or more users to co-author an eBook in real-time using Operational Transformation or CRDTs, similar to Google Docs collaborative editing.
eBook Marketplace	A paid marketplace where authors can list their completed eBooks for sale, with Stripe payment integration, expanding the platform into the creator economy.
AI Personalized Feed	Using the ActivityLog data and survey results to train a recommendation algorithm that surfaces eBooks aligned with each user's reading preferences and interaction history.
Push Notifications	Web push notifications for new followers, likes, comments, messages, and new posts from followed authors using the Web Push API or a service like Firebase Cloud Messaging.
Content Moderation System	An AI-powered content moderation pipeline to automatically flag inappropriate thread posts, story images, or eBook content before it is published to the platform.

Production Deployment	Deploying the full platform to a cloud provider (Railway, Render, or AWS) with a custom domain, HTTPS, environment variable management, and production MongoDB Atlas configuration.

8. AI Use Section

The following section documents all AI tools used throughout the development and documentation of the Bookstagram project by both team members. The table includes the tool name, version and account type, the specific feature for which the tool was used, and the value added by each team member beyond the AI-generated output.

8.1 Khushi Bhatt – AI Tool Usage

AI Tool Name	Version, Account Type	Specific Feature for which AI Tool was Used	Value Addition, What value did you add over and above what AI did for you?
ChatGPT	GPT/5, free version	How to implement AXIOS, what are best practices?	I watched tutorials to understand why Axios is used and resolved configuration errors during the setup process.
Gemini	Free version	List of websites for getting the buttons and images	Spend some time on getting the right images and buttons which will match the bookstagram vibe
ChatGPT	GPT/5, free version	How to add testimonials. Best way to have frontend repository.	I have spent some time on checking other websites for testimonials and working on testimonials along side watching YouTube videos and trying and testing designs by myself.
ChatGPT	GPT/5, free version	Navigation issue with 2 pages.	Restructured my project folders for longtime structure and solved the issue.
ChatGPT	GPT/5, free version	Error in the frontend display and coding error.	Tracing the paths and solving minor coding error that AI missed.
ChatGPT	GPT/5, free version	Research on image generation technique	Manually evaluated and filtered AI-generated suggestions, selecting only techniques suitable for the Bookstagram use case. Integrated the chosen image generation approach into the app architecture and wrote custom prompt templates tailored to book genres.
Claude AI	Free version	Code explanation and debugging assistance; report writing and documentation drafting	Critically reviewed all AI-suggested code before using it. Understood each snippet, modified it to match the project's structure, and personally wrote

			the final implementation. Used Claude only as a starting reference, not a copy-paste source.
Grammarly	Free version	Grammar and spelling correction; tone and clarity suggestions for report writing	All report content was originally written by me. Grammarly was used only as a proofreading tool to catch typos and improve sentence flow. Final phrasing and all technical details were independently composed.
Chatgpt (OpenAI)	GPT 5.3, Free version	Integrating story feature like instagram	Researched on how the new story feature is implemented, got some references from the google, and implemented the frontend structure which align with the file structure.
Claude AI	Free version	Debugging Export API returning 404 error	Analyzed server logs, checked route configuration, fixed incorrect module imports, and updated the backend structure so the export endpoint worked correctly.
Chatgpt (OpenAI)	GPT 5.3, Free version	Converting Markdown content into DOCX format	Implemented the markdown parsing logic, structured paragraphs and lists correctly for DOCX output, and integrated the feature into the backend export functionality.
Chatgpt (OpenAI)	GPT 5.3, Free version	Git branch management and merging updates from main branch	Executed Git commands to pull updates from the main branch into the feature branch and resolved merge conflicts
Claude	Sonnet 4.6/Free	To generate follow, unfollow endpoints	Created follow and unfollow endpoints to match the existing frontend state structure
Claude	Sonnet 4.6/Free	Debugged 500 error	Used browser console tab and network tab added console.error logging to isolate crash
Claude	Sonnet 4.6/Free	Suggested user wiring from mock to real backend	Identified dynamic gradient styling issue when replacing mock data
Claude	Sonnet 4.6/Free	To update DashboardPage to use NewDashboardLayout with sidebar	Identified the exact import swap and prop wiring (onCreateBook) needed to connect the sidebar's Create button to the existing modal state
ChatGPT	GPT 5.3/Free	Debugging React frontend error when rendering generated image	Fixed issue where empty string was passed to img src and

			implemented conditional rendering for generated images
ChatGPT	GPT 5.3/Free	Fixing backend logic for extracting generated images from Gemini API response	Adjusted code to correctly parse image data from the Gemini response and return Base64 image URL to frontend
Claude	Sonnet 4.6/Free	Fixing merge conflict in bookController.js	Identified the root cause was a silent timeout, tested the fix and verified the editor loaded correctly after
Claude	Sonnet 4.6/Free	Cleaning up socialController.js	Reviewed the two versions of getFeed and confirmed which one to keep based on project structure
Claude	Sonnet 4.6/Free	Building Followers/Following modal	Integrated follow/followers into existing profile layout, tested navigation to user profiles from the modal
Claude	Sonnet 4.6/Free	Implementing AI Avatar generation feature	Debugged Hugging Face model availability error
Claude	Sonnet 4.6/Free	fixing broken /dashboard routes across the app	Ran findstr searches to verify no remaining imports before deleting files Dashboard Page, DashboardLayout.

8.2 Aditi Aditi – AI Tool Usage

AI Tool Name	Version, Account Type	Specific Feature for which AI Tool was Used	Value Addition – What value did you add over and above what AI did for you?
ChatGPT (OpenAI)	GPT-4o, Free Version	NewDashboardLayout — sidebar navigation with Home, Explore, Threads, Messages, Profile, Settings icons, live search bar, and profile dropdown	Matched brand theme colors, integrated with AuthContext and axiosInstance, tested routing and fixed active nav state manually. Added live search debounce logic independently.
ChatGPT (OpenAI)	GPT-4o, Free Version	Threads features— Thread model (MongoDB), threadController with 5 API functions (create, get all, like/unlike, add comment, delete), threadRoutes with Multer (up to 5 images), ThreadsPage full frontend rewrite with real API integration, image upload	Designed MongoDB Thread schema based on project requirements. Reviewed all 5 controller functions for consistency with existing auth middleware. Manually tested all endpoints. Fixed 500 errors and response format issues. Built complete frontend with optimistic like updates.

		preview, likes, comments, full-screen image modal	
ChatGPT (OpenAI)	GPT-4o, Free Version	MessagesPage UI — conversation list sidebar with unread message badges, real-time chat window with Socket.io receiveMessage event integration, message input, AI Bookbot interface with book recommendation and writing tip responses	Designed conversation list deduplication logic. Implemented AI Bookbot priority-matched response chain for book recommendations, genre queries, and writing tips. Integrated Socket.io receiveMessage event in frontend chat window. Tested real-time message delivery.
ChatGPT (OpenAI)	GPT-4o, Free Version	3 Writing Modes in Chapter Editor — Write Yourself, Full AI Generation, Half Human + Half AI — completeChapterContent in aiController, /complete-chapter-content route in aiRoutes, updated ChapterEditorTab.jsx and EditorPage.jsx	Integrated with existing controller conventions. Tested all three modes end-to-end. Fixed edge cases in mode switching logic manually. Verified AI continuation matched the user's writing tone.
ChatGPT (OpenAI)	GPT-4o, Free Version	AI Writing Assistant on Threads sidebar — Gemini AI integration for improving thread text and suggesting 3–5 relevant hashtags with individual and bulk copy functionality	Designed prompt structure for Gemini to return improved text and structured hashtag suggestions. Implemented copy-to-clipboard for individual and bulk hashtags. Integrated into existing sidebar layout.
ChatGPT (OpenAI)	GPT-4o, Free Version	SettingsPage — 5 sections: Edit Profile (photo upload, bio), Change Password, Notification Preferences, Appearance (dark mode, font size via localStorage), Privacy and Safety	Connected all 5 sections to their backend endpoints. Implemented localStorage persistence for theme and font size. Fixed bio field not showing on profile by tracing bug across socialController and authController.
ChatGPT (OpenAI)	GPT-4o, Free Version	AnalyticsPage UI — StatCards, 7-day LineChart, most-used features BarChart, top users leaderboard, gender PieChart, age BarChart using Recharts library, date-range filter	Integrated Recharts components with real backend API data. Configured chart axes, tooltips, and color schemes to match platform brand. Tested all charts with seeded analytics data.
ChatGPT (OpenAI)	GPT-4o, Free Version	SurveyModal — 3-step progressive feedback form with gradient progress bar, genre chip buttons, star rating, recommendation	Designed the complete 3-step survey flow and question structure independently. Implemented pendingSurveyBookId

		toggle buttons, and localStorage pendingSurveyBookId trigger mechanism	localStorage trigger mechanism. Tested complete survey submission flow end-to-end including analytics dashboard visualization.
ChatGPT (OpenAI)	GPT-4o, Free Version	Debugging and error resolution — solving 500 server errors, bio field not displaying, avatar URL missing base URL in localStorage, surveyID error, timetaken field error, image alignment issues in ThreadsPage, storyController errors, and dark mode inconsistencies across multiple pages	Independently analyzed server logs and error messages to identify root causes. Applied targeted fixes without rewriting working code. Verified fixes through manual testing across multiple user accounts and page flows. Documented all resolved issues for team reference.
ChatGPT (OpenAI)	GPT-4o, Free Version	StoryController error resolution — fixed storyController errors causing stories not to display correctly in the StoriesStrip on the home feed.	Independently traced the root cause by reviewing storyController functions and route connections. Identified the issue in story fetch logic, applied targeted fix, and verified stories display correctly for followed users after resolution.
ChatGPT (OpenAI)	GPT-4o, Free Version	SurveyID error resolution — fixed surveyID undefined error causing survey submission to fail and survey data not saving to MongoDB correctly.	Analyzed the error by reviewing surveyController and SurveyModal submission flow.
ChatGPT (OpenAI)	GPT-4o, Free Version	Route error resolution — fixed 404 and 500 route errors across multiple backend routes including thread routes, message routes, and survey routes.	reviewed server.js route registration and identified missing or incorrectly ordered route declarations. Fixed route conflicts and middleware ordering issues. Tested all affected endpoints manually to confirm correct responses.

9. Work Date / Hours Logs

The following entries cover work completed from March 23, 2026 onwards based on GitHub commit history.

9.1 Khushi Bhatt (Team Lead) – 300394398 – Work Logs After March 23, 2026

Date	Hours	Description of Work Done

25/03/2026	**1**	Worked on image generation and researched the best possible free-tier options.
27/03/2026	**5**	Implemented image storage from local to cloud. Added a new Cloudinary configuration file. Modified frontend and backend files related to book covers, thread images, and chapter images. Updated routes and resolved the issue of local image storage completely.
30/03/2026	**5**	Implemented AI avatar action editing after generating an avatar. Users can apply actions like "winking" or "heart pose" using GEMINI + FLUX. Implemented image generation logic and integrated it into the chapter editor. Added functionality for generating chapter images, updated the book model with an image schema, implemented backend logic for AI image generation in chapters, and added a new route for chapter images.
31/03/2026	**4**	Implemented live search functionality in the frontend. Added search paths to the frontend, implemented search logic in the social controller, and created a search endpoint in the backend.
01/04/2026	**5**	Implemented backend logic for AI-generated book covers. Added AI routes for cover images and integrated frontend components in the book editor, book details tab, and chapter editor pages. Added frontend routes for cover images. Resolved errors in the live search feature, enhanced the UI, and removed the generate header image functionality from chapters.
02/04/2026	**3**	Performed feature testing for Bookstagram by creating multiple user accounts.
04/04/2026	**2**	Started working on the user guide and captured application screenshots to improve navigation clarity.
05/04/2026	**2**	Worked on Installation and set up guide.
06/04/2026	**1**	Worked on the final report. Added work logs and AI sections, and included the installation guide and user guide in the appendix.
07/04/2026	**2**	Continued working on the final report. Reviewed all sections, made content improvements, and ensured consistency in fonts, spacing, and header styles.

9.2 Aditi Aditi (Team Member) – 300396361 – Work Logs After March 23, 2026

Date	Hours	Description of Work Done
25/03/2026	2	Updated Explore page UI improvements. Merged pull request #22 from KhushiBhatt22/main into Aditi branch.

27/03/2026	4	Added age, gender, and admin field to user model. Added analytics dashboard and user activity tracking. Added Like Button on BookCard component. Updated Explore page UI. Merged pull requests #23, #24, #25, #26, #28 from Aditi_0044 branch to main.
28/03/2026	6	Built complete Messaging system — added messageRoutes, updated server.js with message route, updated apipaths.js, added socket utility file for real-time messaging, integrated Socket.io (downloaded socket.io, updated package.json), added Message model (message.js), added messageController.js, changed UI after message page integration, removed dummy notification badge useState, updated UI of MessagePage. Built complete Survey system — added Survey model, added surveyController and surveyRoutes, updated server.js as per survey route, updated apipaths with survey endpoints, added Survey model UI. Updated explorepage, profilepage, and newdashboardpage with survey functionality. Changed Analytics page UI to make it more professional. Removed age and gender field. Solved errors. Merged pull request #29.
29/03/2026	2	Changed UI of survey page. Solved surveyID error. Merged pull request #30 from KhushiBhatt22/Aditi_0044.
31/03/2026	3	Integrated delete account feature. Changed width of BookCard component. Seeded data to verify analytics dashboard is working correctly. Solved errors. Merged pull request #31 from KhushiBhatt22/Aditi_0044.
01/04/2026	5	Added image upload functionality in create story. Solved error in survey model. Changed bookcard background color. Added logout button in bottom of sidebar. Resolved error of live search. Added frontend routes for cover image. Added frontend on book editor, book details, and chapter editor pages for AI cover image. Implementing backend logic for AI book cover. Added upload image to home card. Solved image alignment issue in thread page. Merged pull requests #33, #34, #35 from Aditi branch. Added message button in profile page and navigate directly to new dashboard. Solved storyController error. Added some UI changes across pages.
02/04/2026	4	Modifying UI from edit book section. Solved timetaken error. Deleted generate header image from chapter (removed unwanted feature). Modified bookdetails tab UI. Merged pull requests #36, #37 from Aditi_0044 branch. Final end-to-end testing of all features. Working on final report documentation and preparation of presentation slides.
03/04/2026	2	Conducted comprehensive end-to-end testing of all platform features including authentication, social feed, threads, messaging, stories, explore page, analytics dashboard, survey system, and eBook export. Verified all features working correctly across multiple user accounts. Documented test results and noted minor UI inconsistencies for final fixes.
04/04/2026	2	Worked on final report documentation — gathered feature screenshots, organized work logs, reviewed and updated section descriptions for accuracy based on final implemented features.

05/04/2026	2	Continued working on final report — reviewed and finalized all sections including implemented features, AI use table, and work logs. Prepared and organized project presentation slides covering project overview, tech stack, features implemented, challenges faced, and lessons learned.

10. Concluding Remarks

Bookstagram set out to bridge a clear gap in the digital publishing ecosystem, the absence of a platform that combines AI-assisted writing with social engagement in a single unified experience. Over twelve weeks, the team delivered a fully functional MERN stack social-authoring platform that significantly exceeded the original proposal in both scope and ambition.

The core research hypothesis was validated through implementation. Google Gemini AI reduces the barrier of writer's block by generating structured outlines and chapter content within seconds. The three writing modes, Write Yourself, Full AI Generation, and Half Human + Half AI, give users meaningful control over their creative process, supporting both independence and productivity. The social ecosystem, home feed, likes, follows, public profiles, Explore page, Threads, AI-generated Stories, and real-time messaging, transforms writing from a solitary task into a shared, motivating experience.

Beyond the research objectives, the platform also delivers Cloudinary media storage, PDF and DOCX export with full Markdown rendering, a comprehensive Settings page, an in-app survey system, and an admin analytics dashboard, features that bring the platform close to production readiness.

Perhaps the most valuable lesson from this project was that the best features were not planned from the start, they emerged from listening to users. The Half Human + Half AI mode, the AI Writing Assistant in Threads, and the Explore page all came from direct feedback during testing. This confirmed that iterative, user-centered development consistently produces better outcomes than rigid adherence to an initial plan.

Bookstagram demonstrates that combining AI writing tools with a social community creates an authoring experience that is more engaging, more accessible, and more motivating than anything traditional writing tools can offer. It is a meaningful contribution to the field of human-AI co-creativity, and a strong foundation for what could become a fully deployed social-authoring product.

11. References

Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency*, 610–623.
Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., & Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
Cloudinary. (2024). *Cloudinary documentation: Image and video management*. https://cloudinary.com/documentation
Google. (2024). *Google Gemini API documentation*. https://ai.google.dev/gemini-api/docs
HuggingFace. (2024). *HuggingFace Inference API documentation*. https://huggingface.co/docs/api-inference
JWT.io. (2024). *Introduction to JSON Web Tokens*. https://jwt.io/introduction
MongoDB. (2024). *MongoDB documentation*. https://www.mongodb.com/docs/
React. (2024). *React documentation*. https://react.dev/
Socket.io. (2024). *Socket.io documentation*. https://socket.io/docs/v4/
Tailwind CSS. (2024). *Tailwind CSS documentation*. https://tailwindcss.com/docs
Node.js. (2024). *Node.js documentation*. https://nodejs.org/en/docs/
Express.js. (2024). *Express.js documentation*. https://expressjs.com/
Recharts. (2024). *Recharts documentation*. https://recharts.org/en-US/
Multer. (2024). *Multer middleware documentation*. https://www.npmjs.com/package/multer
Markdown-it. (2024). *Markdown-it documentation*. https://markdown-it.github.io/
PDFKit. (2024). *PDFKit documentation*. http://pdfkit.org/
Vistasocial. (2024). *Generative AI in content creation*. https://vistasocial.com/insights/generative-ai/
Combinedbook. (2024). *How AI is changing the landscape of book publishing*. https://www.combinedbook.com/how-ai-is-changing-the-landscape-of-book-publishing-and-copyright/
Deterding, S., Dixon, D., Khaled, R., & Nacke, L. (2011). From game design elements to gamefulness: Defining gamification. *Proceedings of the 15th International Academic MindTrek Conference*, 9–15.
Firebase. (2024). *Firebase authentication documentation*. https://firebase.google.com/docs/auth
Black Forest Labs. (2024). *FLUX.1 model documentation*. https://huggingface.co/black-forest-labs/FLUX.1-schnell
Deci, E. L., & Ryan, R. M. (2000). The "what" and "why" of goal pursuits: Human needs and the self-determination of behavior. *Psychological Inquiry*, 11(4), 227–268.

Appendix A: Installation Guide

This guide provides step-by-step instructions for setting up and running the Bookstagram application from the GitHub repository.

1. Prerequisites

Before installing Bookstagram, make sure the following software is installed on your machine:

- Node.js v18 or higher - https://nodejs.org
- npm v9 or higher (comes with Node.js)
- MongoDB Atlas account - https://mongodb.com/atlas
- Git - https://git-scm.com
- A code editor such as VS Code

---

2. Clone the Repository

Open your terminal and run the following command to clone the project:

git clone https://github.com/KhushiBhatt22/W26_4495_S2_KhushiB.git

---

3. Backend Setup

3.1 Navigate to the Backend Folder

cd Implementation/backend

3.2 Install Dependencies

Run the following command to install all required backend packages:

npm install

3.3 Create the Environment File

There is an empty .env file where you can see variables such as:

Variable	Example Value	Description
PORT	8000	Port the backend server runs on
MONGO_URI	mongodb+srv://...	Your MongoDB Atlas connection string
JWT_SECRET	your_secret_key	Secret key for JWT token signing
GEMINI_API_KEY	AIza...	Google Gemini API key for AI writing
HUGGING_FACE_API_KEY	hf_...	Hugging Face API key for image generation

CLOUDINARY_CLOUD_NAME	your_cloud	Cloudinary cloud name for image storage
CLOUDINARY_API_KEY	123456789	Cloudinary API key
CLOUDINARY_API_SECRET	abc123...	Cloudinary API secret

> Never commit your .env file to GitHub. Make sure it is listed in your .gitignore file.

3.4 Start the Backend Server

Run the following command to start the backend in development mode:

npm run dev

The backend server will start on: http://localhost:8000

---

4. Frontend Setup

4.1 Navigate to the Frontend Folder

Open a new terminal window and run:

cd Implementation/frontend/bookstagram

4.2 Install Dependencies

npm install

4.3 Start the Frontend

npm run dev

The frontend will start on: http://localhost:5173

Open your browser and navigate to http://localhost:5173 to see the application.

---

5. Database Setup (MongoDB Atlas)

1. Go to https://mongodb.com/atlas and create a free account.
2. Create a new Cluster (free tier M0 is sufficient).
3. Go to Database Access → Add a database user with username and password.
4. Go to Network Access → Add IP Address → Allow Access from Anywhere (0.0.0.0/0).
5. Click Connect → Drivers → Copy the connection string and paste it as MONGO_URI in your .env file.

> Replace <password> in the connection string with your actual database user password.

---

6. JWT Secret Key

Generate a secure random JWT secret key using this command in your terminal:

node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"

Copy the output and paste it as the value of JWT_SECRET in your .env file.

---

7. API Keys Setup

7.1 Google Gemini API Key

6. Go to https://aistudio.google.com
7. Sign in with your Google account.
8. Click Get API Key → Create API Key → Copy and paste into .env as GEMINI_API_KEY.

7.2 Hugging Face API Key

9. Go to https://huggingface.co and create a free account.
10. Go to Settings → Access Tokens → New Token → Copy and paste as HUGGING_FACE_API_KEY.

7.3 Cloudinary

11. Go to https://cloudinary.com and create a free account.
12. From your Dashboard, copy Cloud Name, API Key, and API Secret into your .env file.

---

8. Running the Full Application

You need TWO terminal windows running simultaneously:

Terminal 1 - Backend:

cd Implementation/bakend

npm run dev

Terminal 2 - Frontend:

cd Implementation/frontend/bookstagram

npm run dev

Then open your browser and go to: http://localhost:5173

---

9. Technology Stack

Frontend	Backend	Services
• React.js (Vite)	• Node.js + Express	• Google Gemini AI
• React Router DOM	• MongoDB + Mongoose	• Hugging Face (FLUX)
• Tailwind CSS	• JWT Authentication	• Replicate API
• Axios	• Multer (file upload)	• Cloudinary
• Lucide React Icons	• bcryptjs	• MongoDB Atlas

Appendix B: User Guide

This guide explains how to use the Bookstagram platform as an end user.

1. What is Bookstagram?

Bookstagram is an AI-powered social platform that lets users create, publish, and discover eBooks. It combines powerful AI writing tools with an Instagram-style social feed, making storytelling collaborative, visual, and fun for everyone.

AI Book Creation	Write full books with AI-generated outlines, chapter content, and cover images
AI Stories	Create Instagram-style stories with AI-generated images and avatars
Social Feed	Follow authors, like books, and discover new content in your feed
Search	Search for users and books instantly with live search
AI Avatar	Upload your photo and generate a stylized cartoon avatar
Export	Export your eBook as PDF or Word document in one click

[Table 1: Features of Bookstagram]

2. Getting Started

2.1 Browse Landing Page

1. The first page is Bookstagram's Landing page
2. In this page, there are 3 main sections: Hero section, Feature section, Testimonial.
3. Hero section: Include brief bookstagram description, Navigation links: features, testimonials, 3 button: Login, Start creating for free, Get started.
4. Feaure section: Describes bookstagram features.
5. Testimonials: Shows our top users reviews.
6. Footer: Shows some navigation links.

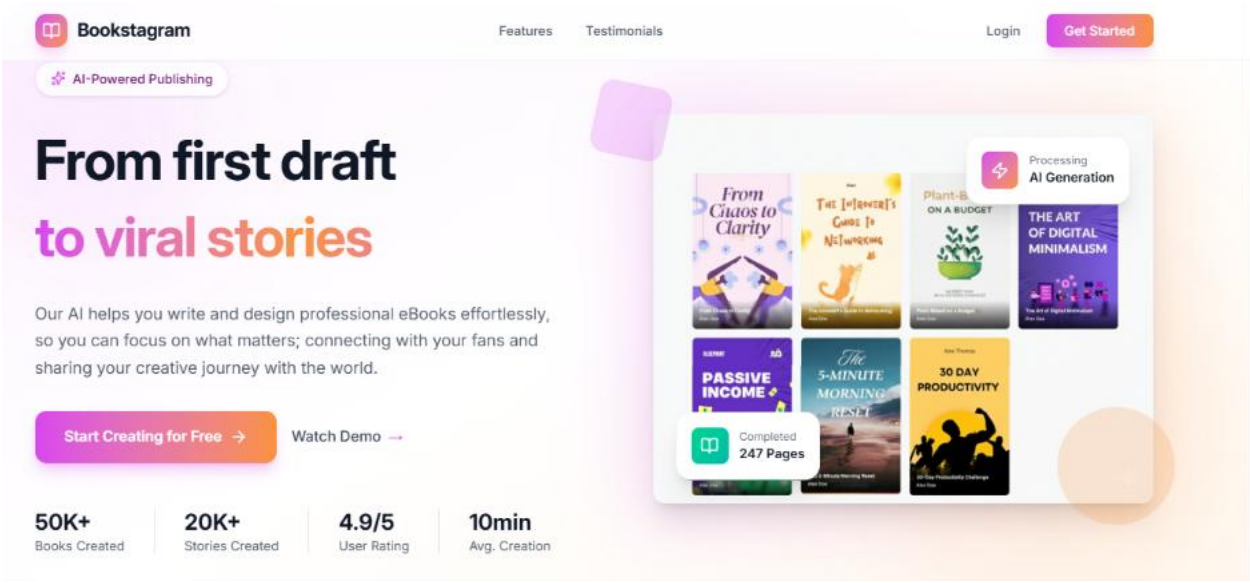


Figure 1: Landing Page – Hero Section

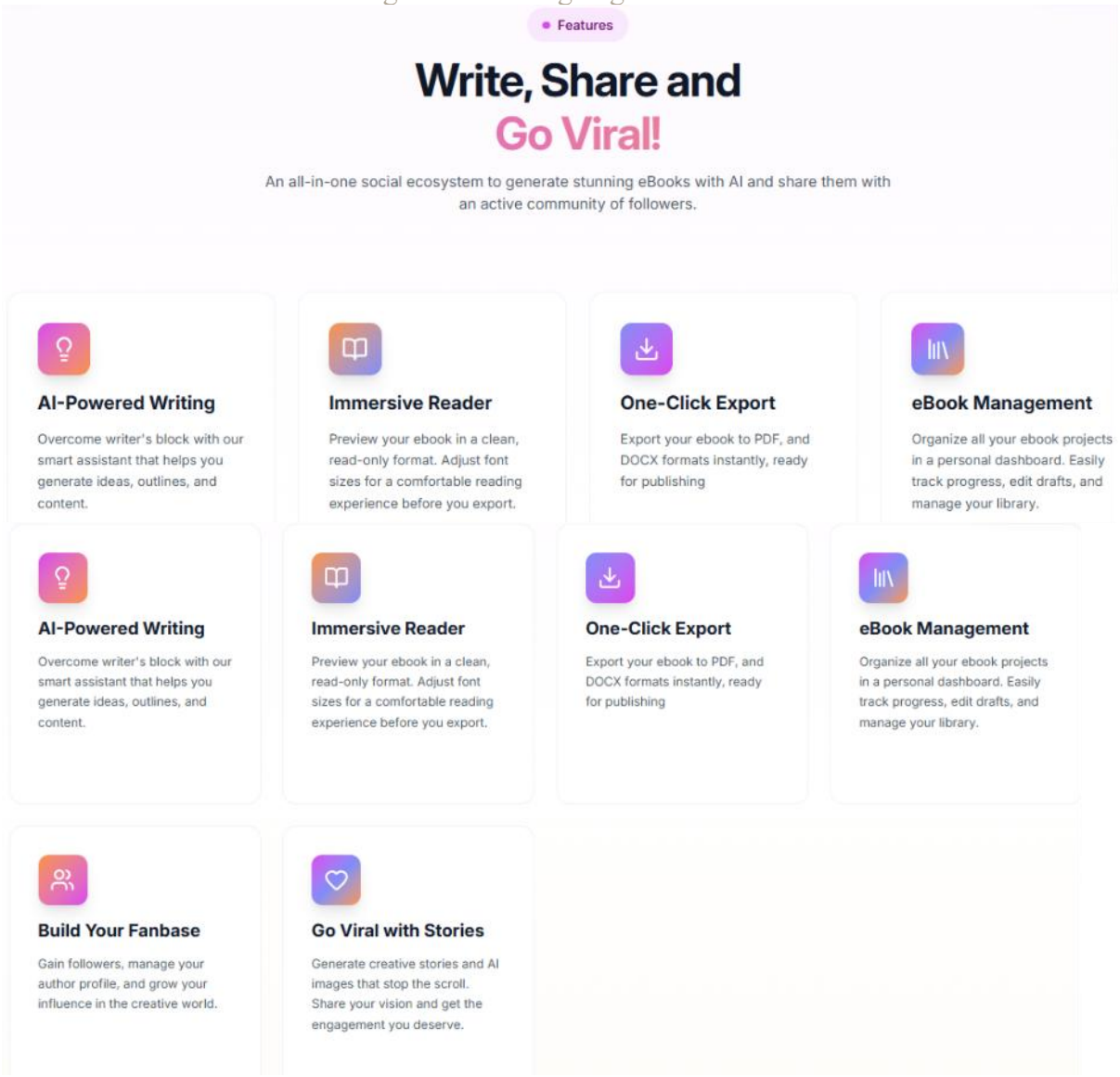


Figure 2: Landing Page - Feature Section

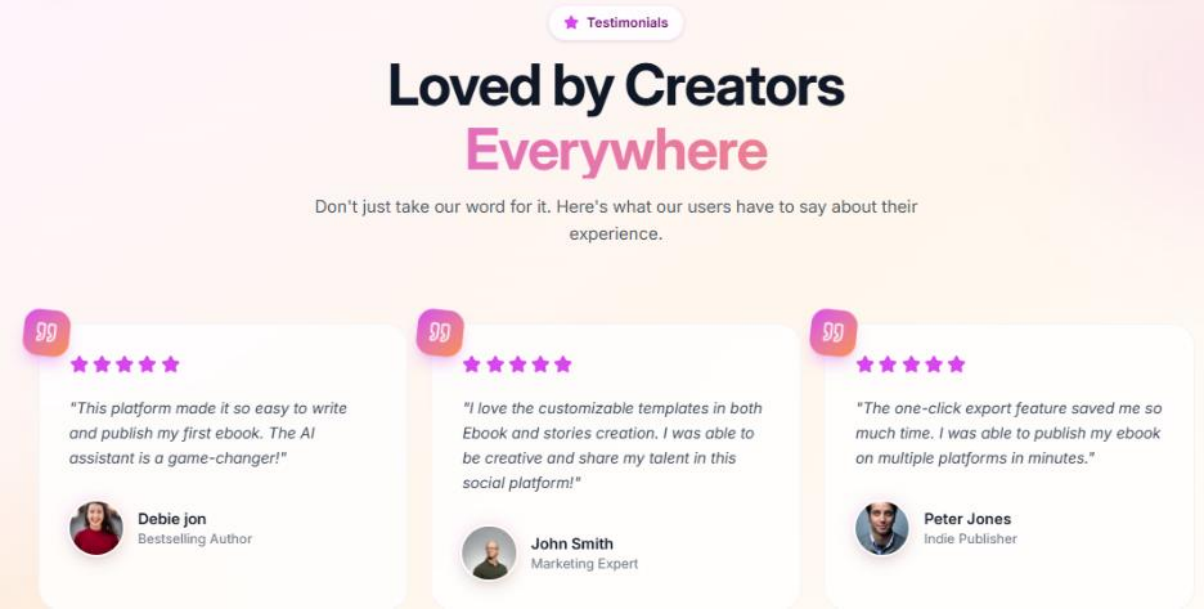


Figure 3: Landing Page - Testimonial Section

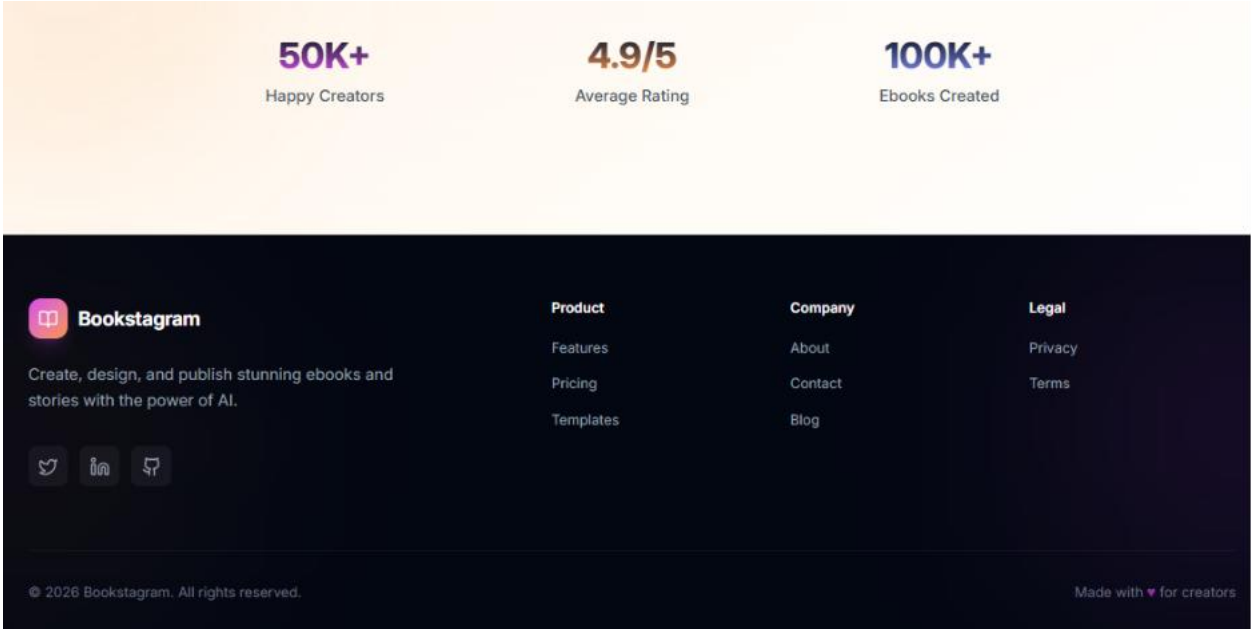


Figure 4: Landing Page - Footer

2.2 Creating an Account

To use Bookstagram, you need to register for a free account.

1. Open your browser and go to Landing page
2. Click the Get Started button on the landing page
3. Enter your full name, email address, and a password (min 6 characters) or tap continue with Google
4. Click Register

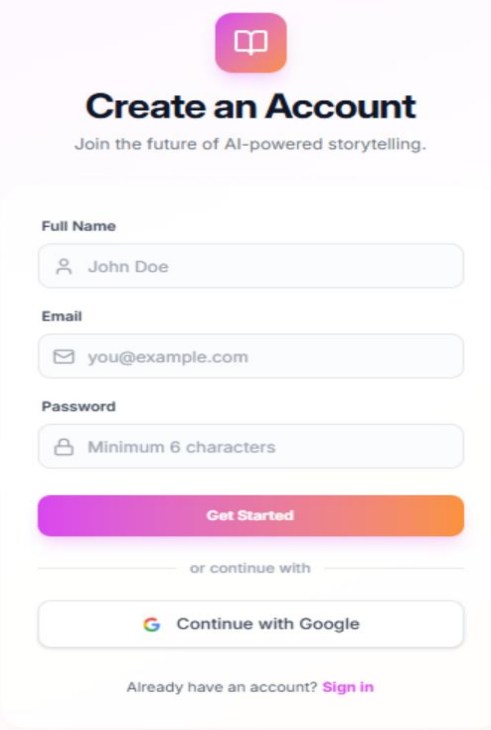


Figure 5: Sign Up Page

2.3 Logging In

1. Go to Landing page and click Login.
2. Enter your email and password or continue with google
3. Click Sign in

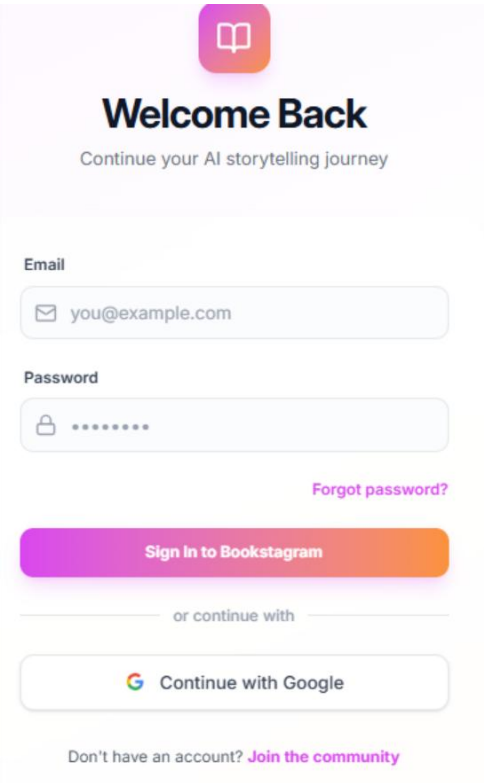


Figure 6: Log in Page

3. Navigating the Dashboard

After logging in, you will see the main dashboard. It has 3 main sections:

- **Left Sidebar —** Navigation links: Home, Explore, Threads, Messages, Profile, Settings
- **Center Feed —** Books and stories posted by people you follow and search bar.
- **Right Panel —** Suggested people to follow and trending books

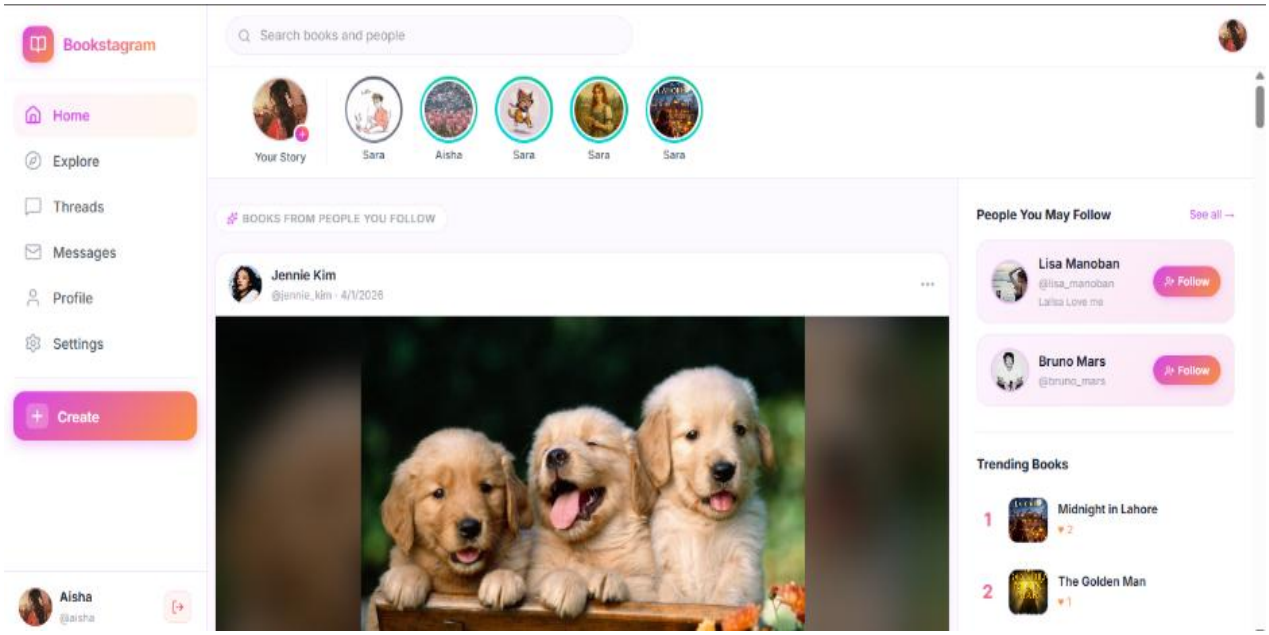


Figure 7: Dashboard Page

4. Creating an eBook

4.1 Start a New Book

1. Click the Create button in the left sidebar
2. A modal will appear; enter your Book Title, and optionally a Topic
3. Choose the number of chapters and writing style.
4. Click Generate Outline: AI will create chapter titles and descriptions
5. Click Create Book: You will be taken to the Book Editor

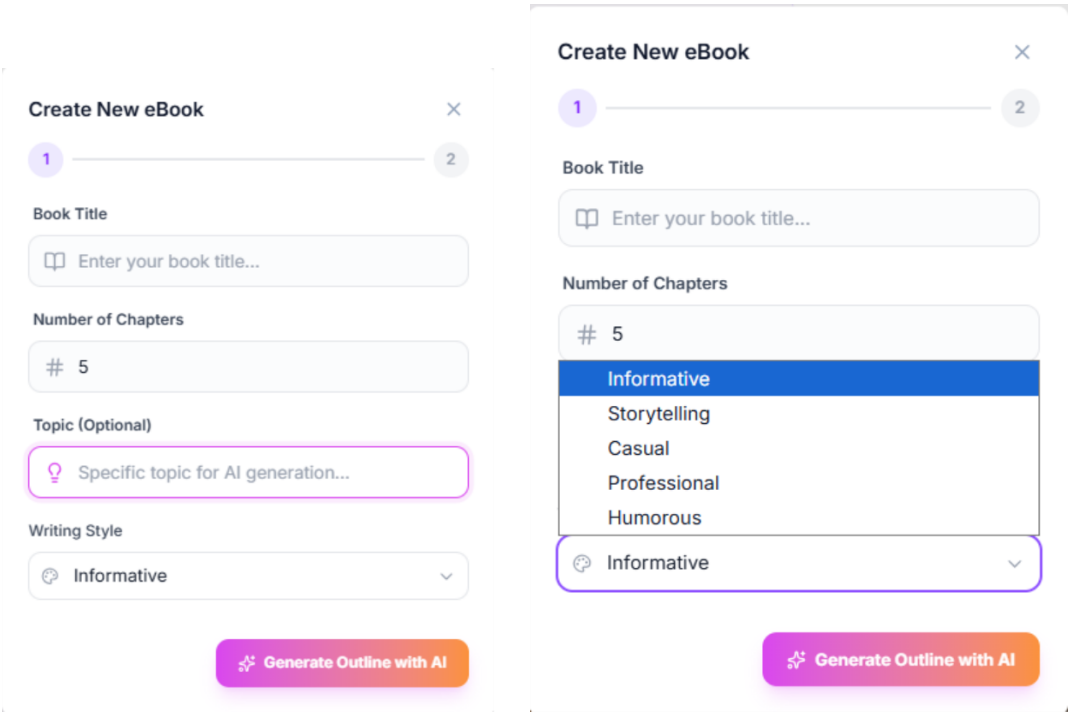


Figure 8: eBook Creation Modal

4.2 Writing Chapter Content

In the Book Editor, you have three writing modes:

- **Write Yourself -** Type your own content in the editor
- **Full AI Generation -** AI writes the entire chapter for you
- **Half Human + Half AI -** Write a few lines then AI completes the rest

Click on Preview button to see the chapter content and images

Click on generate content image button to generate colorful illustrations from the chapter's content.

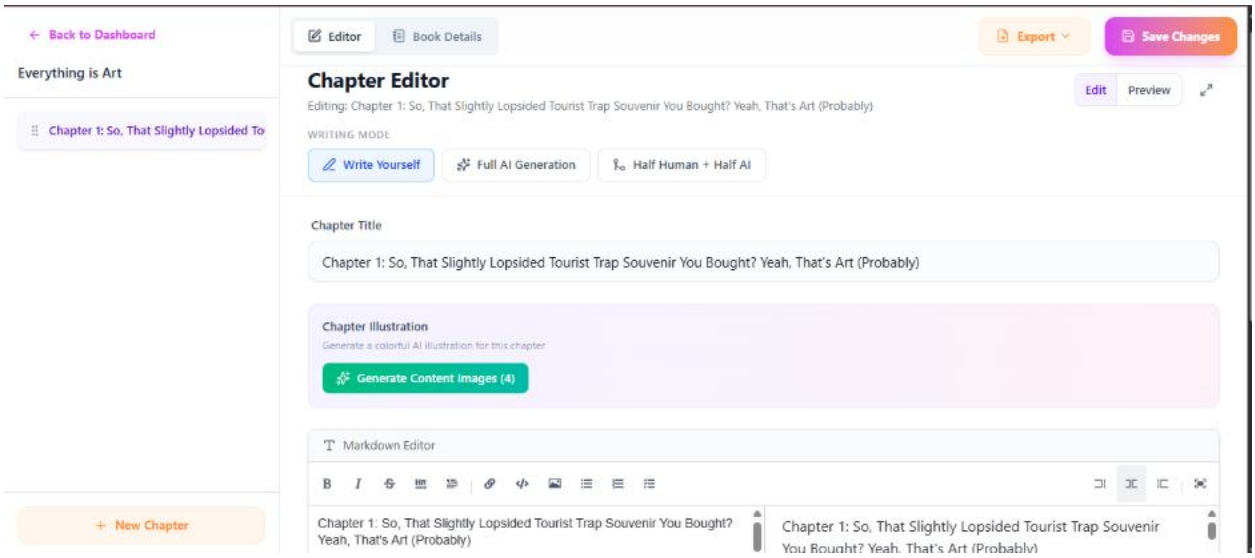


Figure 9: eBook Editor Page

4.3 Uploading a Cover Image

1. Click the Book Details tab at the top of the editor
2. Scroll down to the Cover Image section
3. Click Upload Image and select an image from your computer or Click Generate AI Cover
4. The cover will be saved automatically to Cloud Storage.

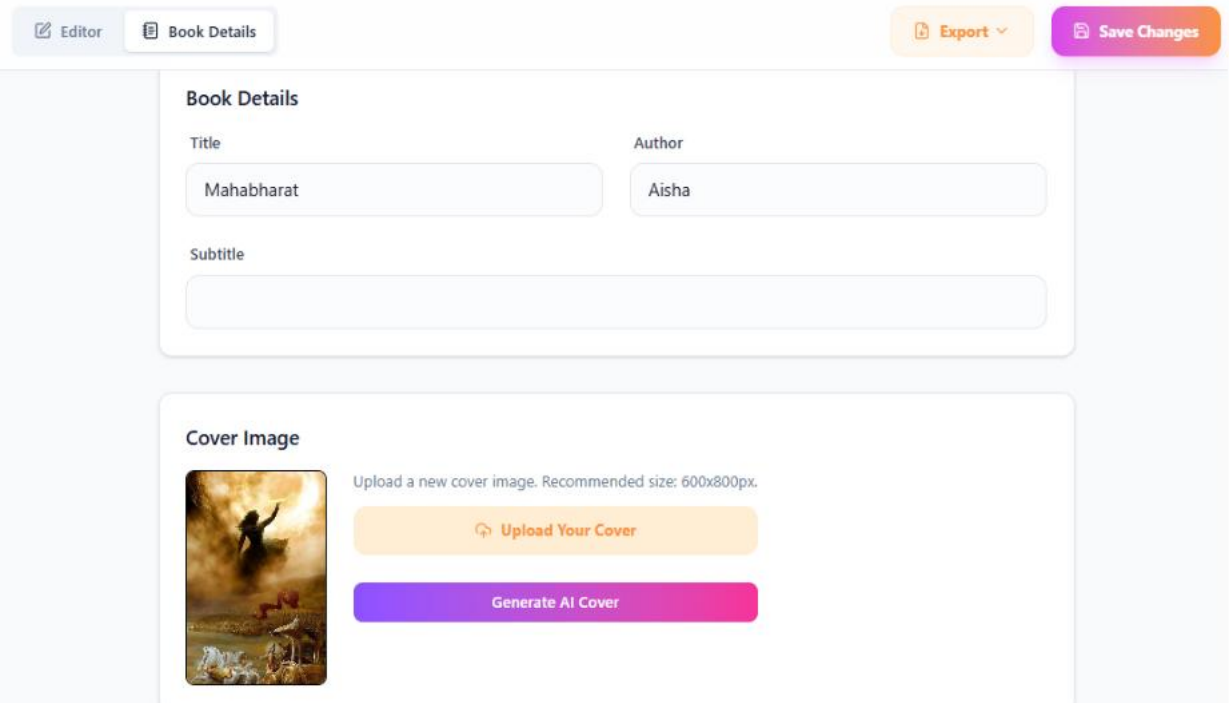


Figure 10: Book Details Page

4.4 Exporting Your Book

1. Click the Export button in the top right of the editor
2. Choose Export as PDF or Export as Document (.docx)
3. The file will download automatically to your computer

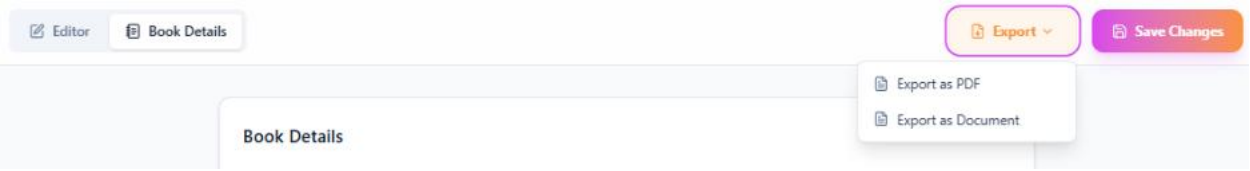


Figure 11: Export a Book

5. Social Features

5.1 Following Other Users

1. Go to the Explore page from the sidebar on dashboard page.
2. Click on any book to view it, or click on the author's name to visit their profile
3. On their profile, click the Follow button
4. Their books will now appear in your Home Feed

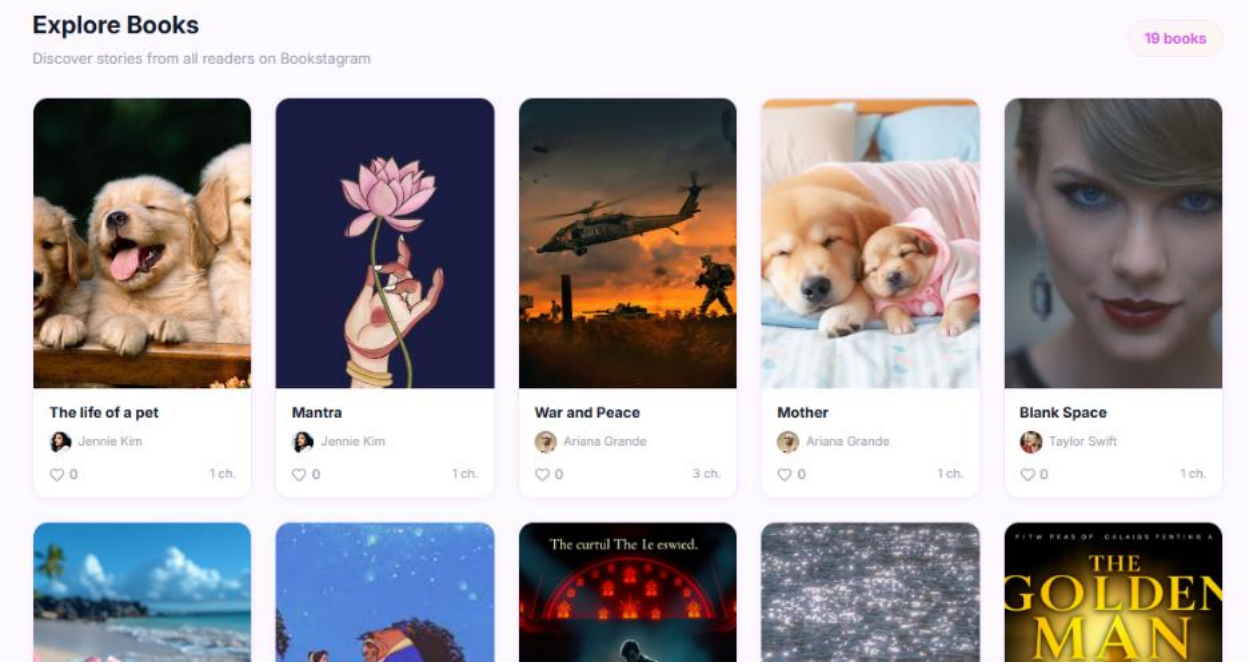


Figure 12: Explore Books Page

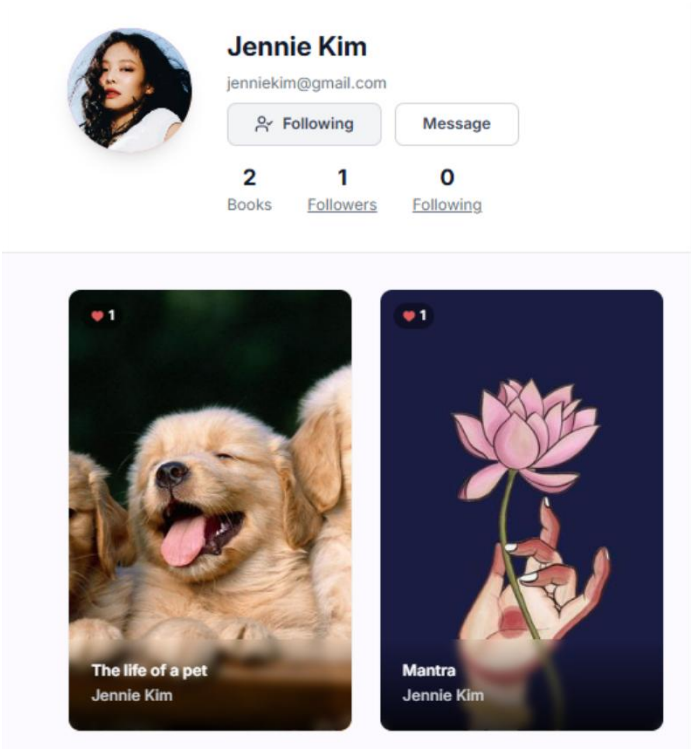


Figure 13: Follow User's Profile

5.2 Viewing Followers and Following

1. Go to your Profile page from the sidebar
2. Click on the Followers or Following count
3. A modal will appear showing the list of users with their avatars
4. Click on any user to visit their profile

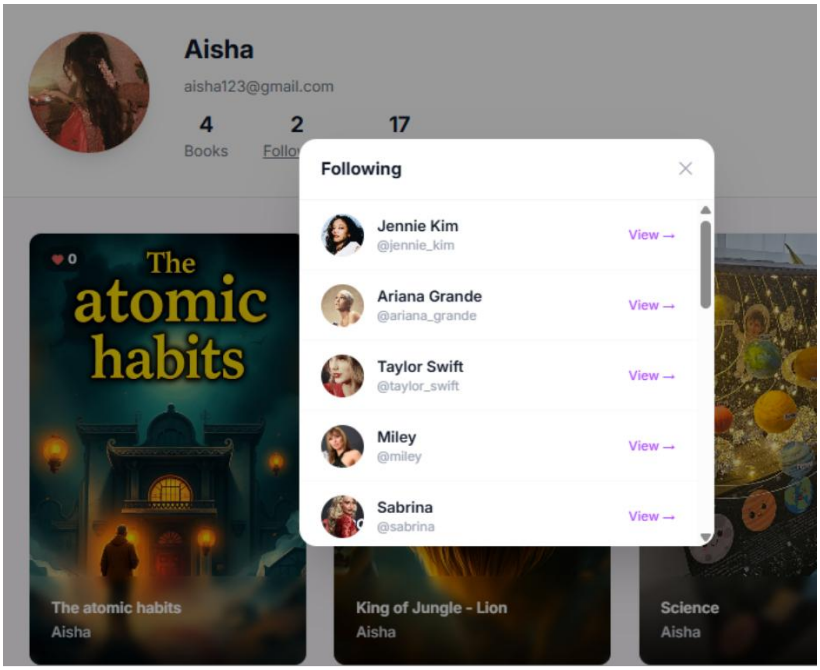


Figure 14: Profile Page

5.3 Liking Books

You can like books from the Explore page, the Home Feed, or from any user's profile.

1. Click the Heart icon on any book card
2. The like count will update instantly
3. Click again to unlike

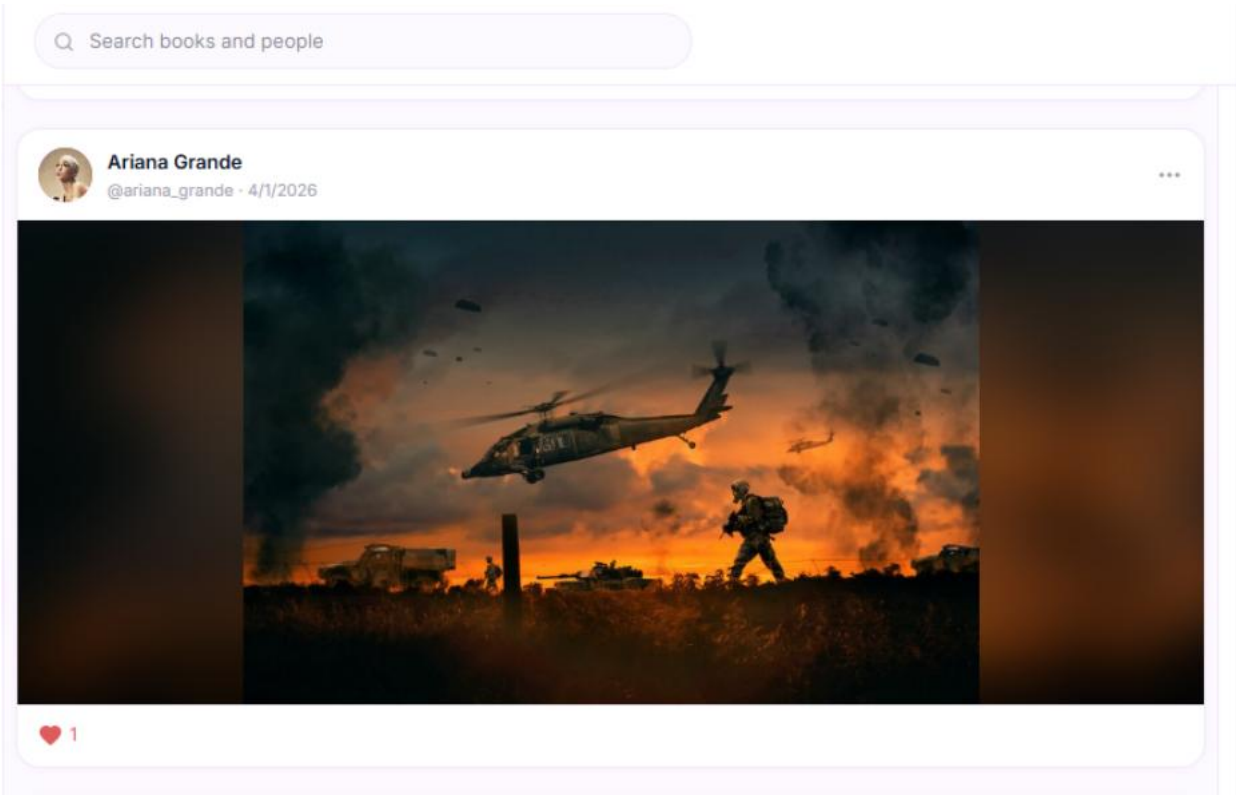


Figure 15: Like Books

5.4 Searching for Users and Books

1. Click the search bar at the top of the page
2. Start typing a user name or book title
3. A dropdown will appear with matching results
4. Click on a user to visit their profile, or a book to read it

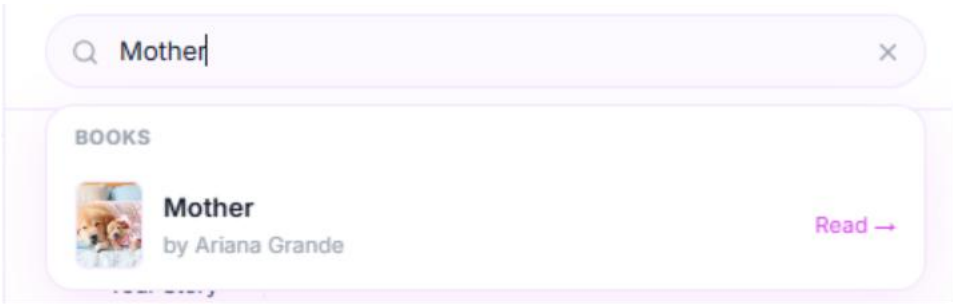


Figure 16: Search Bar - Book

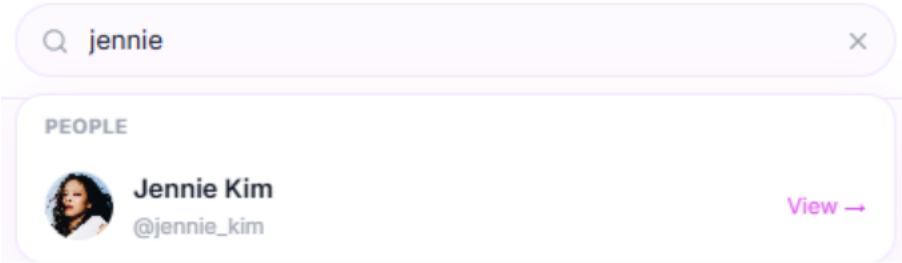


Figure 17: Search Bar - People

6. Creating AI Stories

6.1 From Prompt - Generate a Story Image

1. Click the + button next to Your Story in the stories strip at the top on Dashboard Page
2. The Create Story modal will open and stay on the From Prompt tab
3. Type your creative prompt (e.g. A cat wearing a wizard hat)
4. Choose your art style: Cartoon, Sketch, Storyboard, or Colorful
5. Click Generate Story; the AI will create your image
6. Click Save Story; it will appear in the stories strip

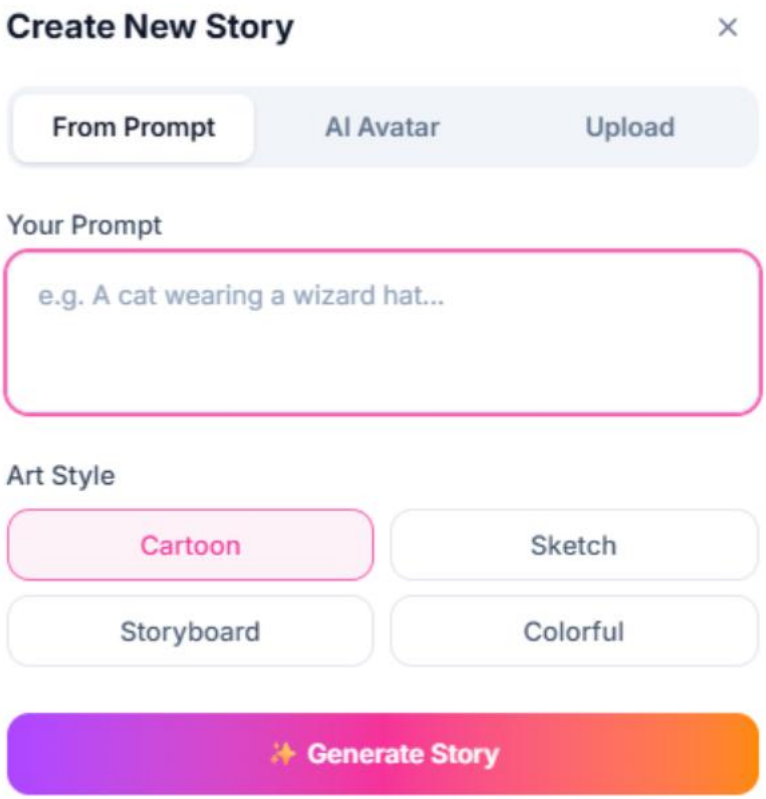


Figure 18: Generate Story – From prompt

6.2 AI Avatar- Generate Your Own Avatar

1. Click the + button next to Your Story
2. Click the AI Avatar tab
3. Click Upload Your Photo and select a clear photo of yourself
4. Choose your avatar style: Cartoon, Sketch, Storyboard, or Colorful
5. Click Generate Avatar and Gemini AI will analyze your photo and create a stylized avatar
6. Optionally click an action button (Winking, Heart Pose, etc.) or write a prompt and click Apply to edit your avatar
7. Click Save Avatar Story to share it

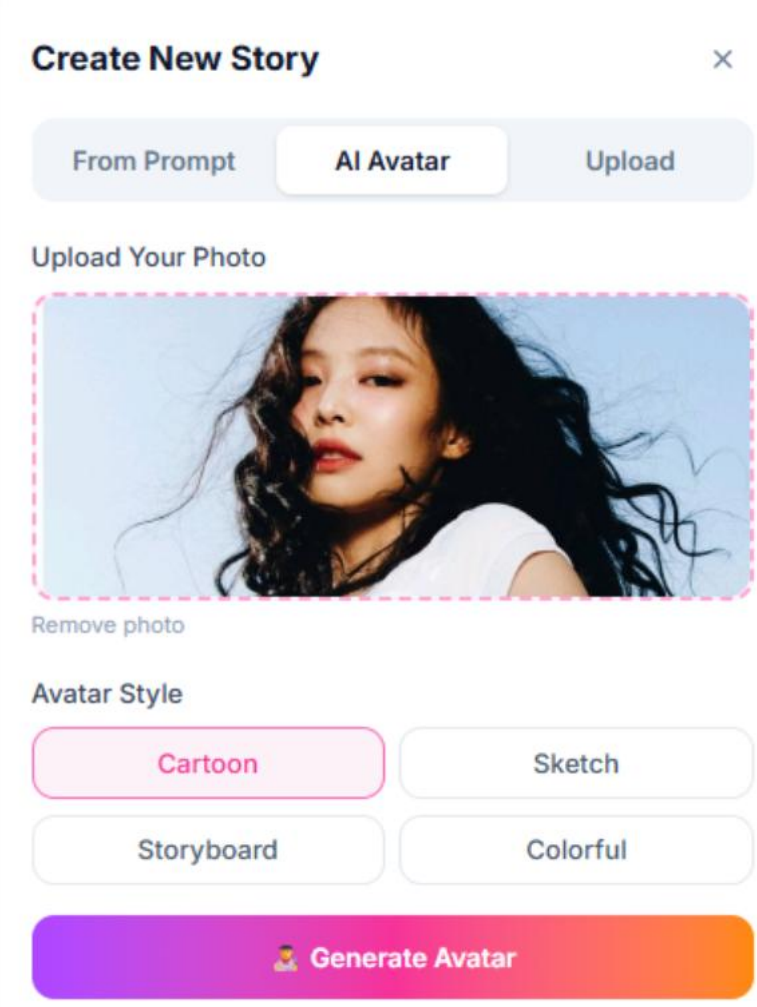


Figure 19: Generate Story – AI Avatar

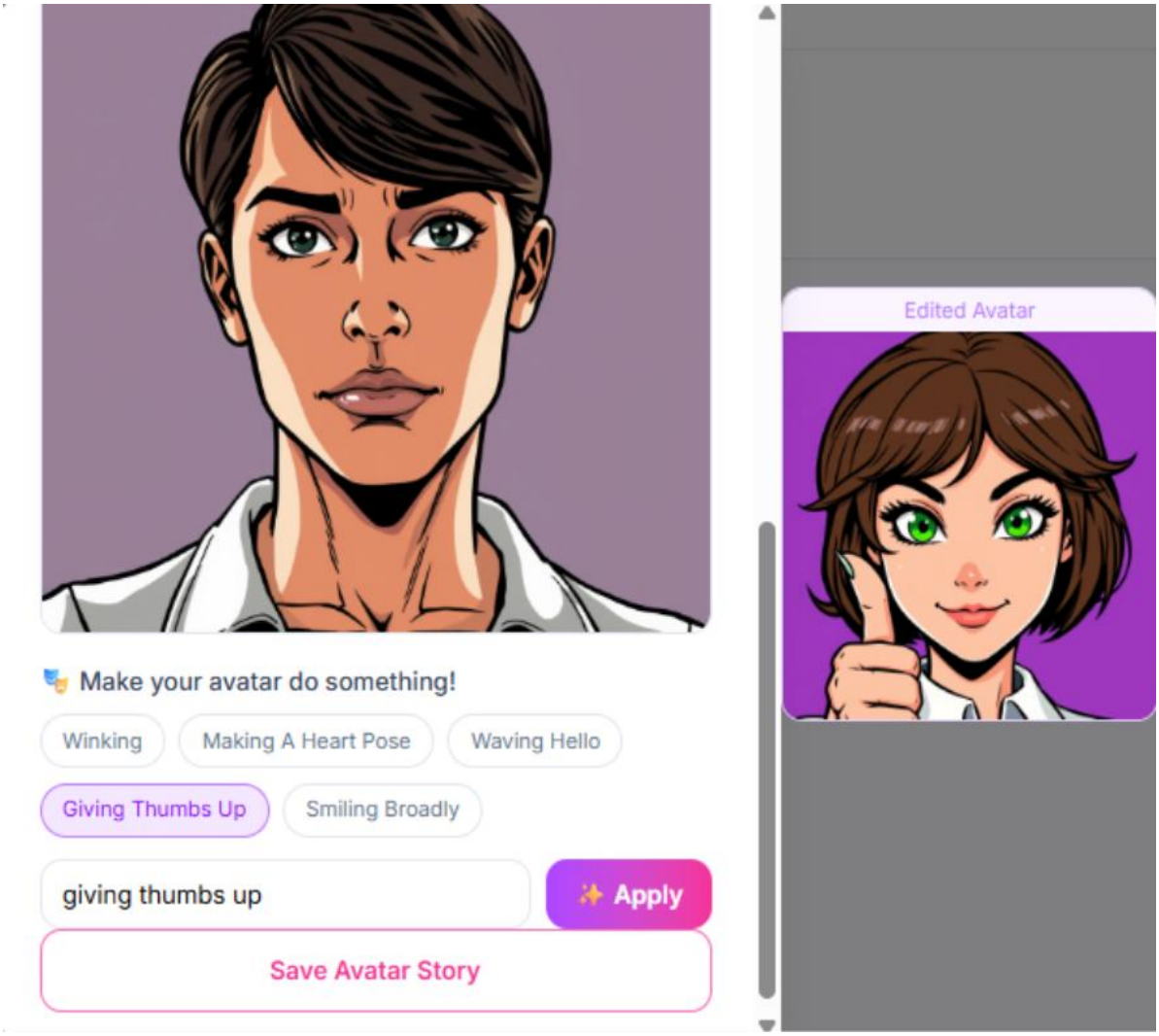


Figure 20: Edit Avatar

6.3 Upload Image

1. Click on Upload tap from the create story modal
2. Choose a photo from local device
3. Write a caption and click post story

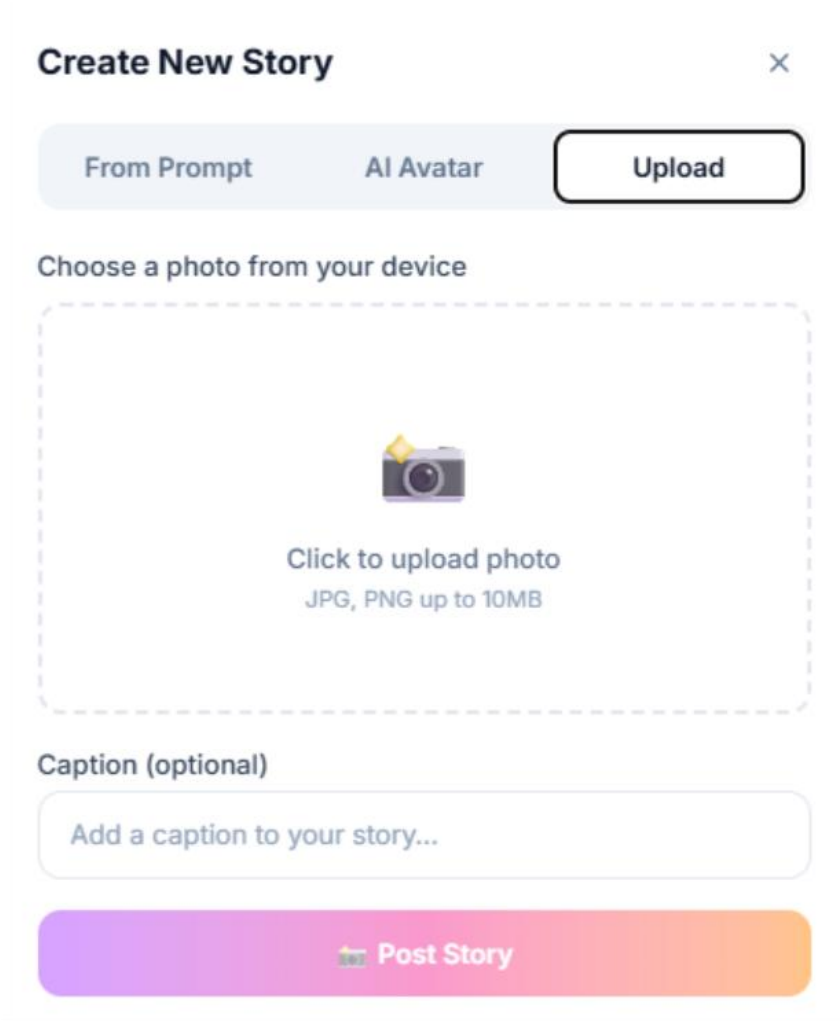


Figure 21: Story - Upload photo

7. Managing Your Profile

7.1 Viewing Your Profile

1. Click Profile in the left sidebar
2. You will see your name, bio, follower count, following count, and your books

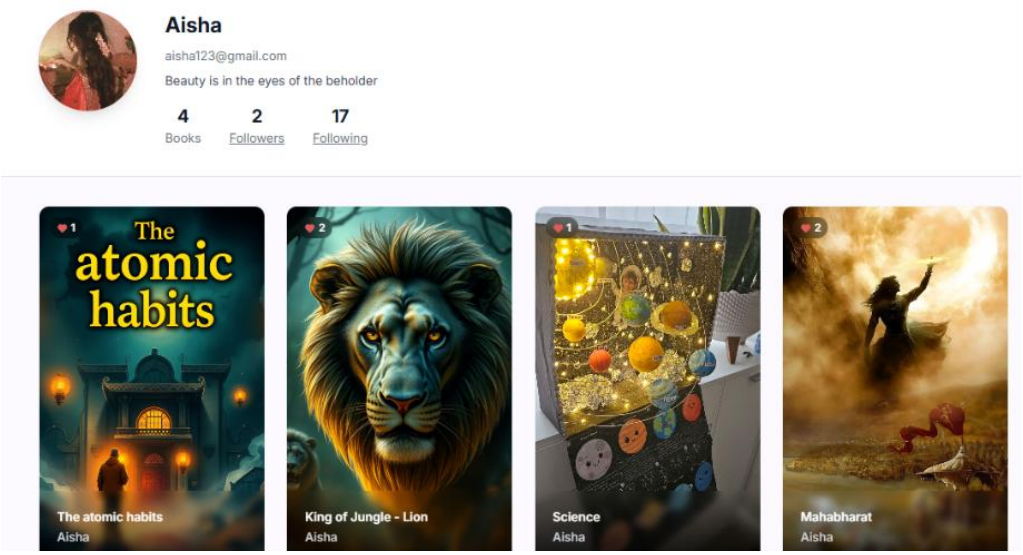


Figure 22: Profile Page

7.2 Updating Profile Settings

1. Click Settings in the left sidebar
2. Update your name, bio, or profile photo
3. Click Save Changes

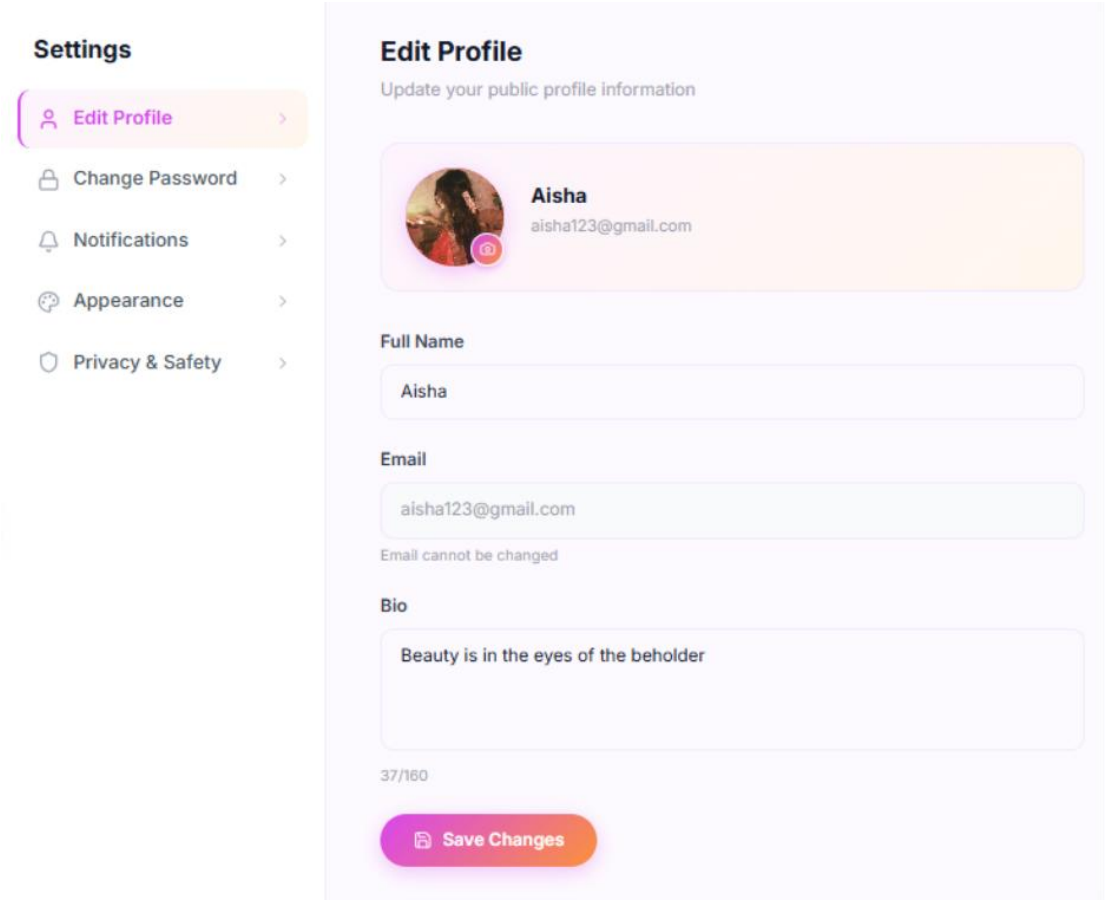


Figure 23: Edit Profile Page

7.3 Changing password

1. Write current password
2. Write a new password and confirm password
3. Click on Change password

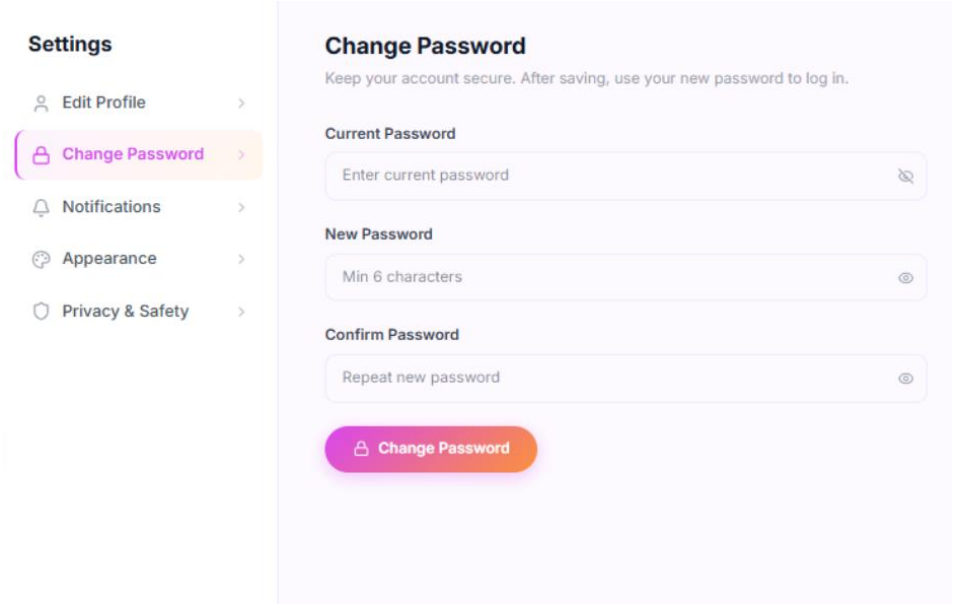


Figure 24: Change password

7.4 Settings Page

1. Open **Settings** from the navigation menu.
2. Click on **Notifications** in the sidebar.
3. Toggle notification options ON/OFF as needed.
4. Customize preferences for follower, likes, comments, and replies.
5. Click **Save Preferences** to apply changes.

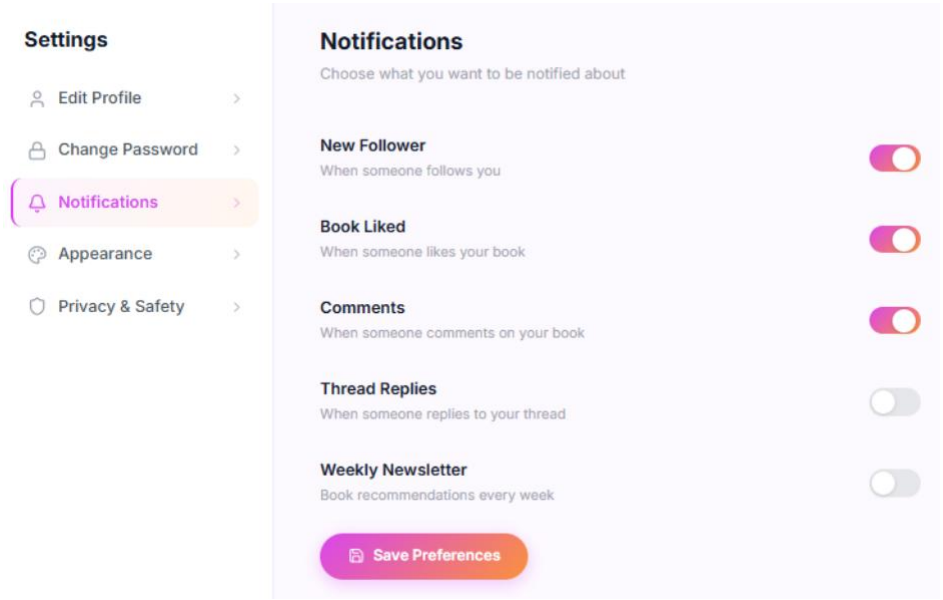


Figure 25: Notification Page

7.5 Appearance Page

1. Open **Settings** from the navigation menu.
2. Click on **Notifications** in the sidebar.
3. Toggle notification options ON/OFF as needed.
4. Customize preferences for follower, likes, comments, and replies.
5. Click **Save Preferences** to apply changes.

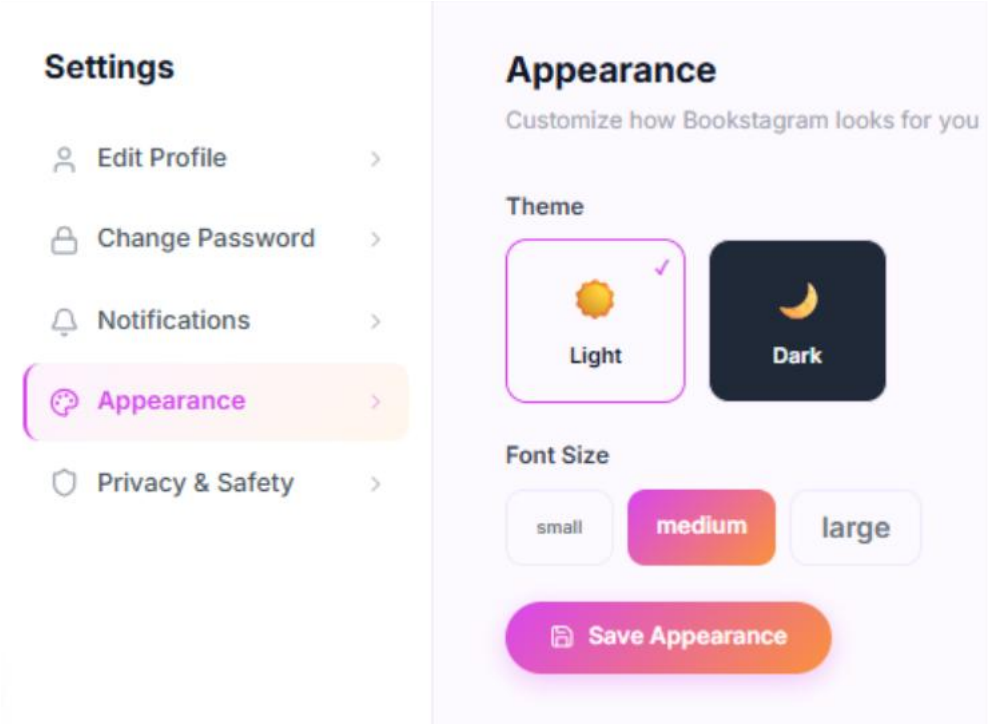


Figure 26: Appearance Page

7.6 Privacy & Safety

1. **Private Account;** Toggle on to restrict your content to approved followers only.
2. **Show Activity;** Toggle off to hide your last active status.
3. **Allow Messages;** Toggle off to limit messages to followers only.
4. **Log Out** — Tap to sign out of your account.
5. **Delete Account** — Tap to permanently remove your account.

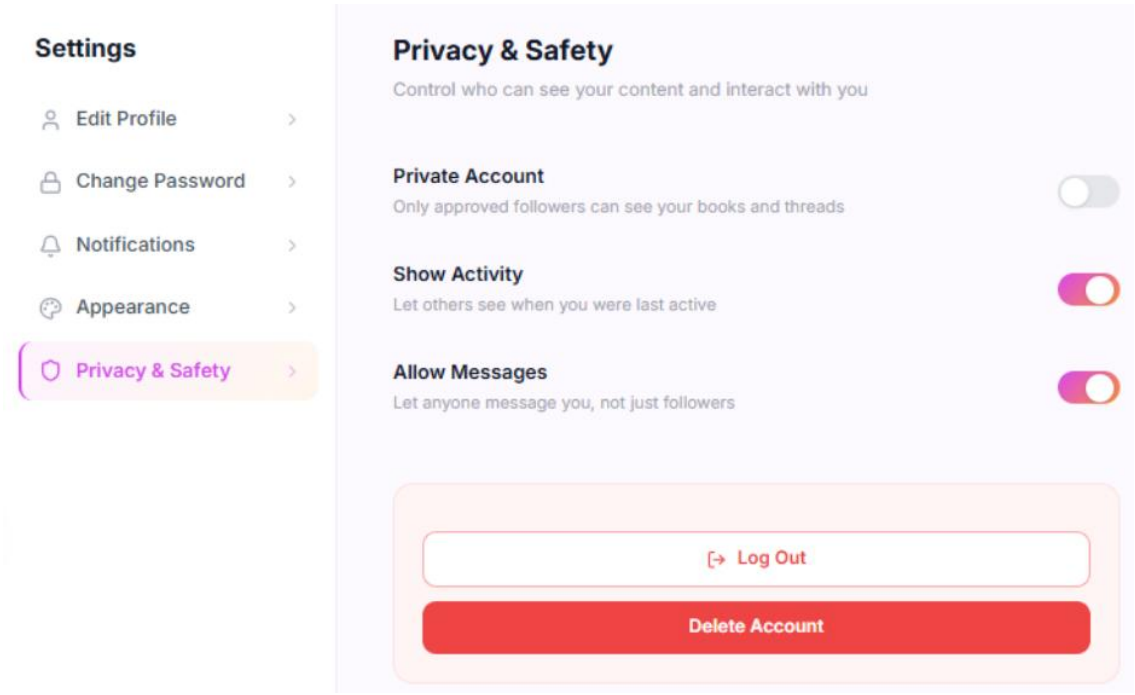


Figure 27: Privacy & Safety Page

8. Threads

Share your reading thoughts, book reviews, or short stories with the community.

1. Click on Threads from the left sidebar.
2. Write your thoughts or use the AI tool to improve your writing and add relevant hashtags.
3. Click on the image icon to upload images(up to 5 images allowed)
4. Click Post thread to publish.
5. You can **like** and **comment** on threads shared by others.

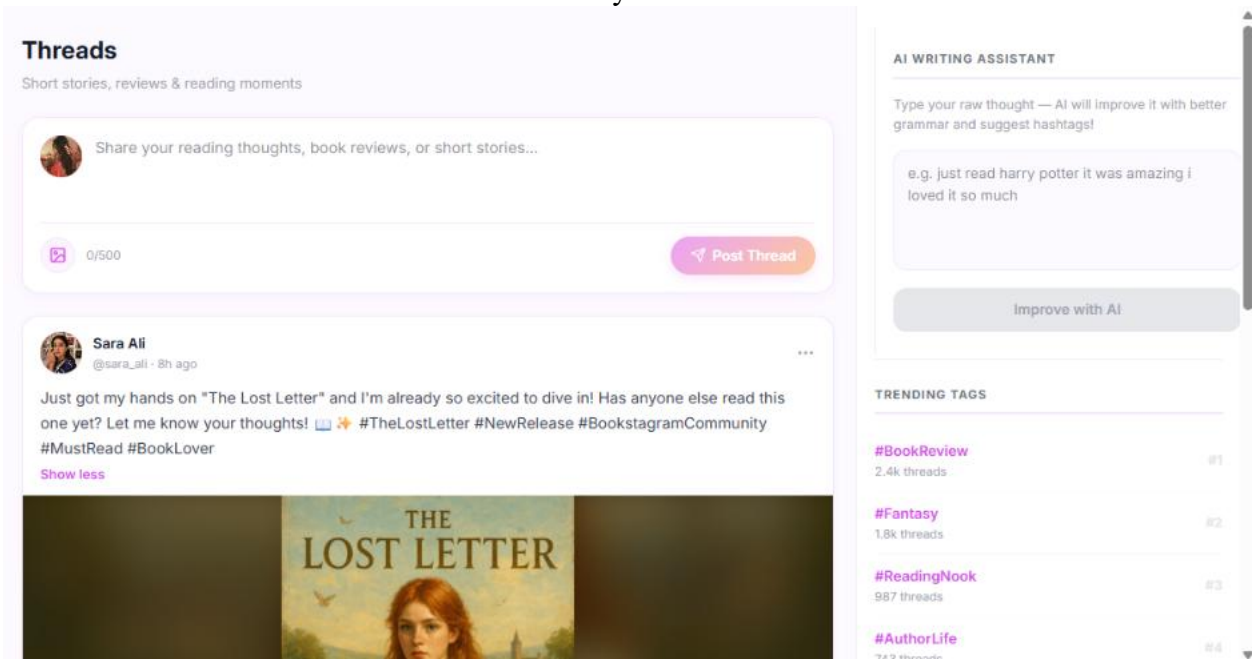


Figure 28: Threads page

9. Messages

1. Click to view the user's profile or go to **Message** from the sidebar
2. Click the **Message** button to start a conversation and send messages.
3. Click on **BookBot** to interact with our Bookstagram chatbot and ask it for book recommendations, reading suggestions, and more.

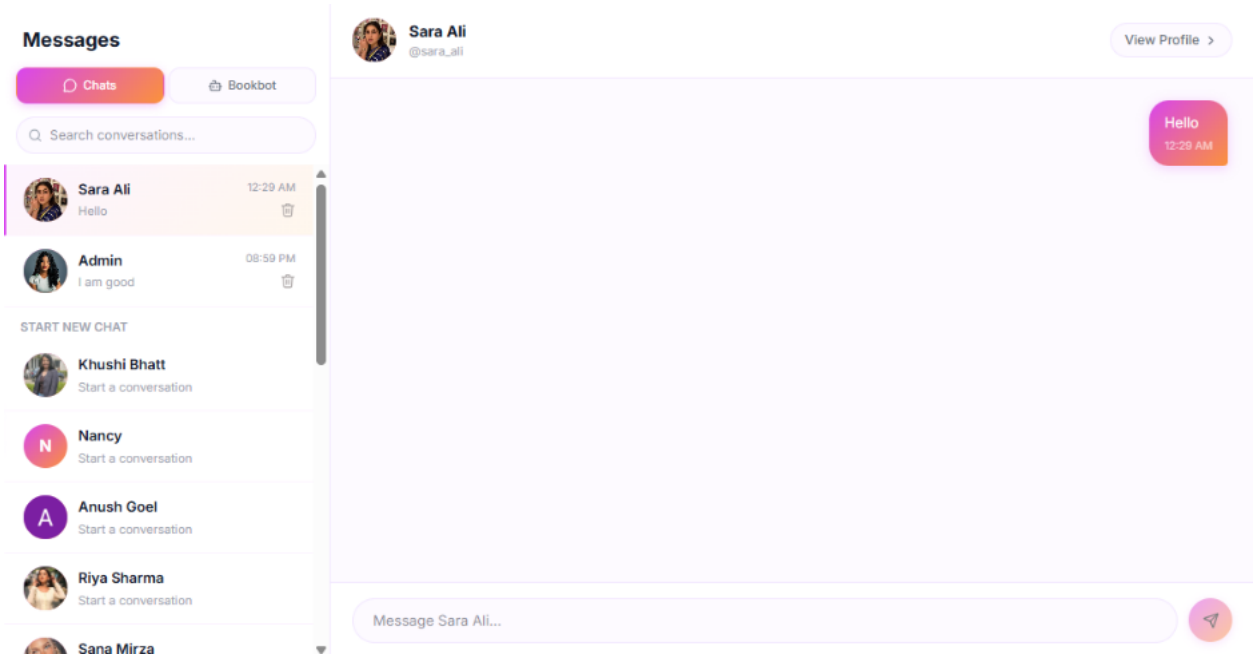


Figure 29: Messages Page

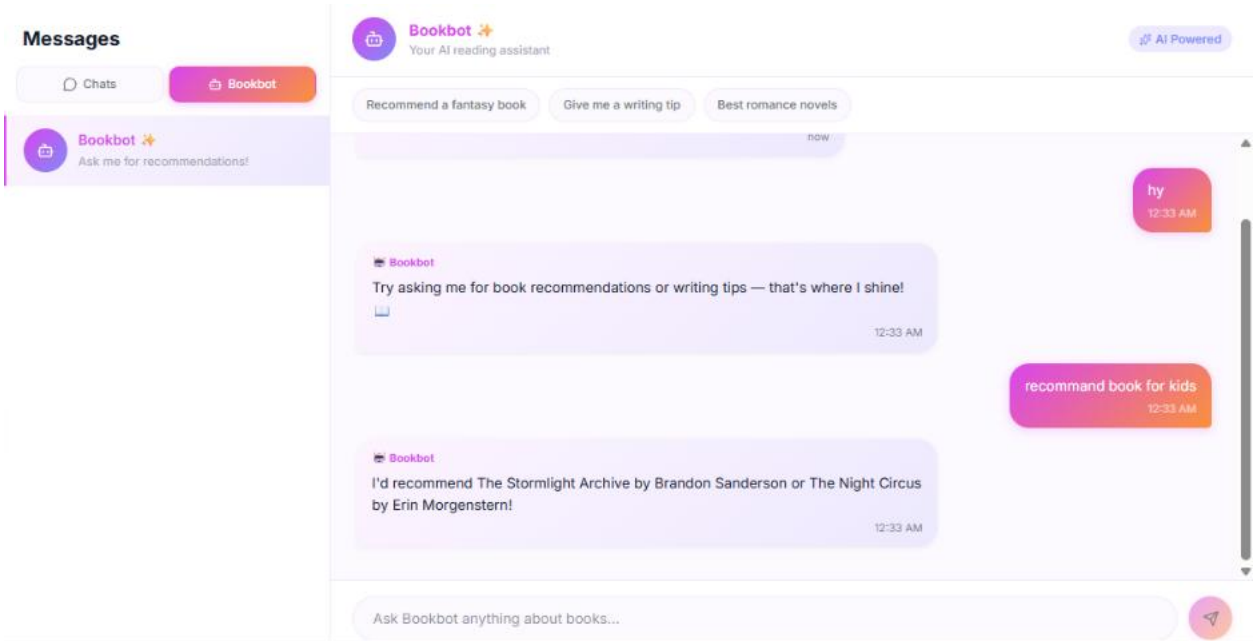


Figure 30: Messages Page - Bookbot

10. Survey

10.1 survey appears automatically when you visit your Profile Page after creating an eBook.

Step 1 of 3 - About You

1. Enter your **Age** in the text field.
2. Select your **Gender** — Male, Female, or Other.
3. Tap **Next** to continue.

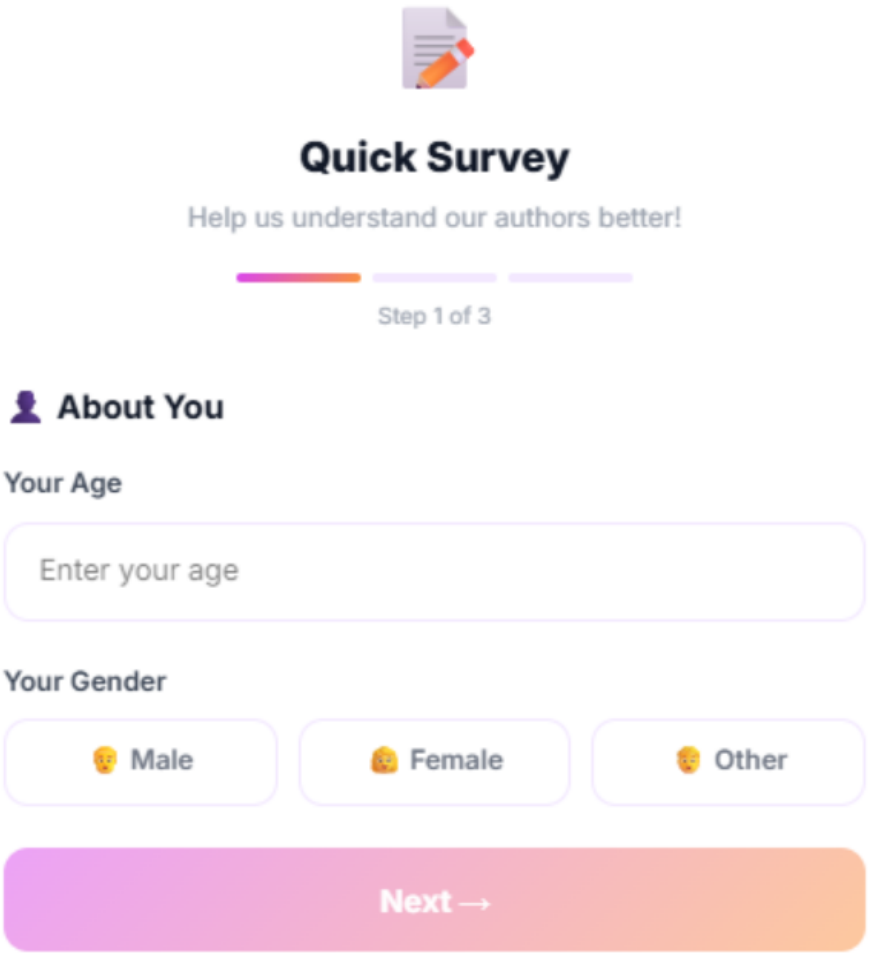


Figure 31: Quick Survey – Step 1

Step 2 of 3 - About Your Book

1. Select your book's **Genre** (e.g., Fantasy, Romance, Mystery, Sci-Fi, etc.).
2. Choose your **Target Audience** (Children, Teens, Young Adults, Adults, or All Ages).
3. Pick what **inspired** you to write (Personal Experience, Imagination, Research, etc.).
4. Tap **Next** to continue.

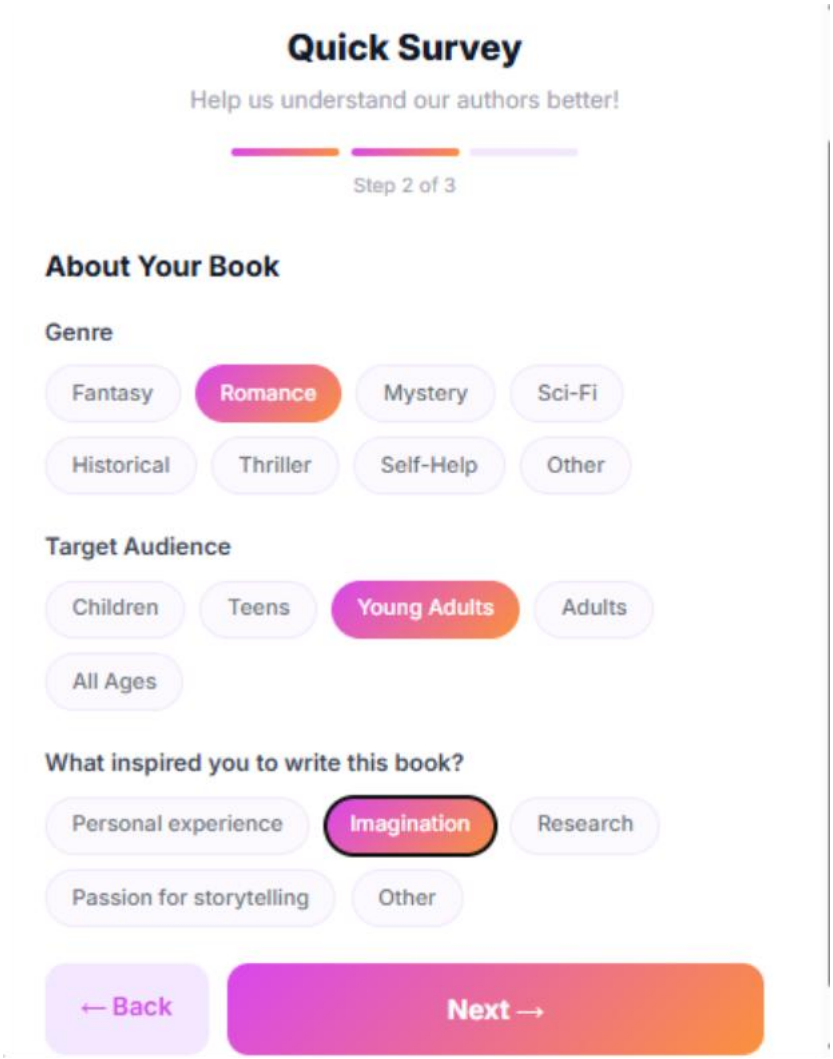


Figure 32: Quick Survey – Step 2

Step 3 of 3 - Your Experience

1. **Rate your experience** by tapping the stars (1–5).
2. Choose whether you'd **recommend Bookstagram** — Yes, Maybe, or No.
3. Tap **Submit** to complete the survey.

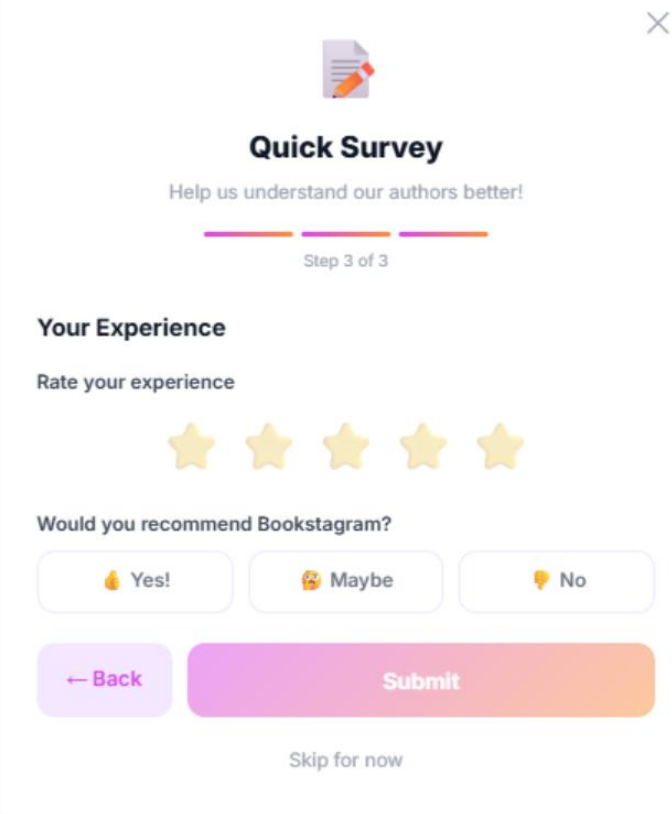


Figure 33: Quick Survey – Step 3

11. Logout

1. Go to the HomePage on the sidebar click logout button or on the top right click on the profile photo and click logout button.

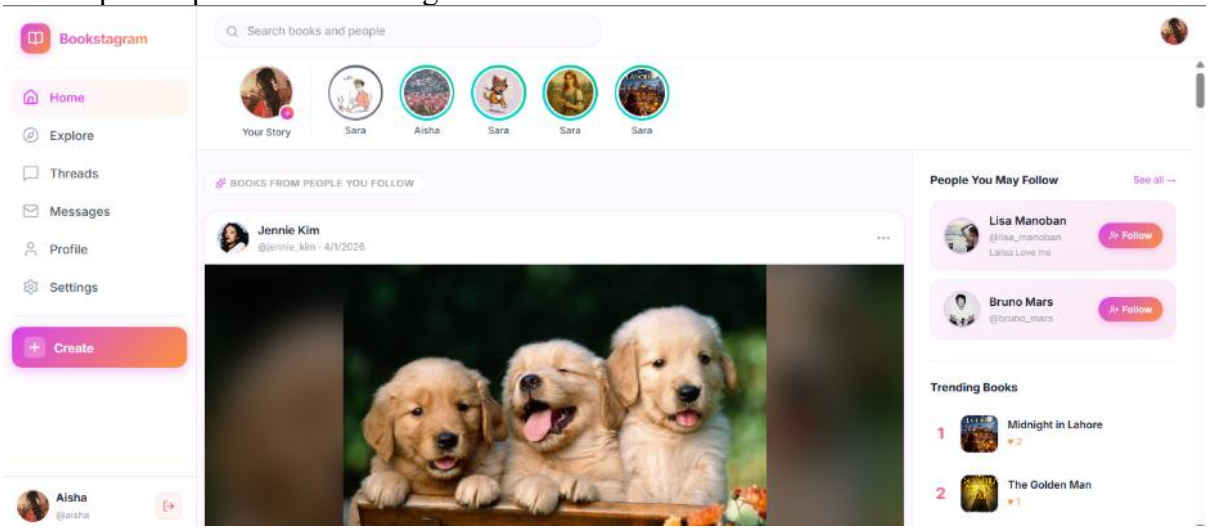


Figure 34: Logout button